GLOBAL TERRORISM DATABASE - DATA SLAYER

Il dataset assegnatoci raccoglie informazioni su attacchi terrostici avvenuti a partire dal 1970 fino agli anni recenti.

Per procedere all'analisi dei dati passeremo per 7 Fasi, indicizzate in tal modo:

Indice

- FASE 1 – Data Understanding
  - 1.1 Caricamento e ispezione
  - 1.2 Dizionario dei dati
  - 1.3 Obiettivi dell'analisi
- FASE 2 – Data Cleaning & Preparation
  - 2.1 Valori mancanti
    - 2.1.1 Identificazione NaN
  - 2.2 Duplicati
  - 2.3 Inconsistenze
    - 2.3.1 Date
    - 2.3.2 Vicinity / Doubterr
    - 2.3.3 Variabili binarie
  - 2.4 Outlier
    - 2.4.1 nperps
    - 2.4.2 nperpcap
    - 2.4.3 propvalue
    - 2.4.4 nkill e nwound
- FASE 3 – Analisi Esplorativa
  - 3.1 Analisi Univariata
    - 3.1.1 Paesi più colpiti
    - 3.1.2 Modalità di attacco
    - 3.1.3 Periodi critici
    - 3.1.4 Obiettivi
    - 3.1.5 Gruppi terroristici
  - 3.2 Analisi Multivariata
    - 3.2.1 Regione vs Bersaglio
    - 3.2.2 Attacco vs Gruppo
    - 3.2.3 Variabili numeriche
    - 3.2.4 Vittime per gruppo
- FASE 4 – Inferenza Statistica
  - 4.1 Domande inferenziali

    - 4.2 Domanda 1
        - 4.2.1 Campione
        - 4.2.2 Test
        - 4.2.3 Interpretazione
    - 4.3 Domanda 2
        - 4.3.1 Campione
        - 4.3.2 Test e IC 95%
        - 4.3.3 Interpretazione
    - 4.4 Domanda 3
        - 4.4.1 Campioni
        - 4.4.2 Ipotesi
        - 4.4.3 Risultati
        - 4.4.4 Conclusione
    - 4.5 Riepilogo
- FASE 5 – Analisi Statistica
    - 5.1 nkill
    - 5.2 ishostkid
    - 5.3 extended
- FASE 6 – Analisi Predittiva e ML
    - 6.1 Classificazione nkill
    - 6.2 Classificazione attacktype
    - 6.3 Classificazione gruppi
- FASE 7 – Unsupervised Learning
    - 7.1 Clustering letalità
    - 7.2 Clustering temporale
    - 7.3 KDE geografica
- Conclusioni
    - Prima parte: Analisi Esplorativa e Inferenza Statistica
    - Seconda parte: Modellazione Statistica e Machine Learning
    - Terza parte: Unsupervised Learning

Di seguito vengono definite le dipendenze del seguente notebook:

```
import numpy as np
import pandas as pd
import seaborn as sns
import statsmodels.api as sm
import statsmodels.formula.api as smf

import time

from matplotlib import pyplot as plt
from scipy.stats import chi2_contingency, mode, gaussian_kde
from scipy import stats
from turtle import color
```

```python
from statsmodels.formula.api import logit
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler,
OneHotEncoder,FunctionTransformer, MinMaxScaler
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import accuracy_score,
classification_report,confusion_matrix, roc_curve, roc_auc_score,
silhouette_score
from sklearn.compose import ColumnTransformer
from sklearn.naive_bayes import GaussianNB
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.mixture import GaussianMixture
from sklearn.neighbors import KernelDensity

# variabili globali per i grafici
# per una palette dark di 30 colori
dark_palette = sns.color_palette("dark", len(range(30)))
# per impostare darkblue come colore principale
darkblue_palette = sns.color_palette("dark", 1)


#funzioni statistiche e di conversione tipi
def vcramer(contingency_matrix):
    chi2,_,_,_ = chi2_contingency(contingency_matrix)
    n = contingency_matrix.values.sum()
    k = min(contingency_matrix.shape)
    return np.sqrt(chi2/(n*(k-1)))


def tocategory(df,columns):
    for col in columns:
        df[col] = df[col].astype('category')
    return df


def toint(df,columns):
    for col in columns:
        df[col] = df[col].astype('int')
    return df
```

FASE 1: Acquisizione e Comprensione del Dataset (Data Understanding)

I primi passi che si possono intraprendere sono:

- **Caricare e Ispezionare il dataset;**
- **Stilare il Dizionario dei Dati;**
- **Definire gli obiettivi dell'analisi dei dati.**

1.1: Carichiamo e ispezioniamo il dataset

```
link = "/kaggle/input/global-terrorism-database/globalterrorismdb.csv"
terrorism = pd.read_csv(link,encoding='latin1',low_memory=False)

terrorism.columns
```

```
Index(['eventid', 'iyear', 'imonth', 'iday', 'approxdate', 'extended',
       'resolution', 'country', 'country_txt', 'region',
       ...
       'addnotes', 'scite1', 'scite2', 'scite3', 'dbsource',
'INT_LOG',
       'INT_IDEO', 'INT_MISC', 'INT_ANY', 'related'],
      dtype='object', length=135)
```

```
terrorism.head()
```

```
        eventid  iyear  imonth  iday approxdate  extended resolution
country  \
0  197000000001   1970       7     2        NaN         0        NaN
58
1  197000000002   1970       0     0        NaN         0        NaN
130
2  197001000001   1970       1     0        NaN         0        NaN
160
3  197001000002   1970       1     0        NaN         0        NaN
78
4  197001000003   1970       1     0        NaN         0        NaN
101

          country_txt  region  ... addnotes scite1 scite2  scite3
dbsource  \
0  Dominican Republic       2  ...      NaN    NaN    NaN     NaN
PGIS
1              Mexico       1  ...      NaN    NaN    NaN     NaN
PGIS
2         Philippines       5  ...      NaN    NaN    NaN     NaN
PGIS
3              Greece       8  ...      NaN    NaN    NaN     NaN
PGIS
```

```
4                Japan        4  ...       NaN    NaN    NaN     NaN
PGIS

   INT_LOG  INT_IDEO INT_MISC INT_ANY  related
0        0         0        0       0      NaN
1        0         1        1       1      NaN
2       -9        -9        1       1      NaN
3       -9        -9        1       1      NaN
4       -9        -9        1       1      NaN

[5 rows x 135 columns]
```

```
print(terrorism.info())
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 170350 entries, 0 to 170349
Columns: 135 entries, eventid to related
dtypes: float64(53), int64(24), object(58)
memory usage: 175.5+ MB
None
```

Dimensioni Dataset:

- Numero di righe: 170.350 (ogni riga rappresenta un attacco terroristico)
- Numero di colonne: 135 (variabili descrittive dell'evento)

Tipi di dati:

- float64: 53 variabili numeriche continue (es. numero di feriti, coordinate)
- int64: 24 variabili numeriche discrete (es. anno, mese, giorno)
- object: 58 variabili testuali o categoriche (es. nome del gruppo, tipo di attacco)

Dando una prima occhiata al dataset notiamo una consistente presenza di osservazioni e colonne. Verificheremo la loro consistenza in segiito.

1.2: Dizionario dei Dati

Per garantire che l'analisi dei dati sia coerente e consapevole, il Codebook del Global Terrorism Database (GTD) sotto riportato sarà uno strumento fondamentale:

https://www.start.umd.edu/sites/default/files/2024-10/Codebook.pdf

Dalla documentazione otteniamo informazioni utili per stilare il dizionario dei dati, come il tipo e la scala delle variabili, i valori restituiti e la suddivisione in varie sezioni:

I) Identificativo e Data

- **eventid — numerica discreta, scala ratio** (intero, identificativo unico).

- **iyear, imonth, iday — numeriche discrete, scala intervalli** (anno, mese (1-12), giorno (1-31) o 0/NA se mancante).
- **approxdate — testuale** (stringa, data approssimativa quando giorno/mese non noti).
- **extended — categorica binaria, scala nominale** (0/1: evento esteso su più giorni).
- **resolution — testuale / categorica, scala nominale** (stringa che descrive come l'evento è stato risolto).
- **country (ID) — numerica discreta, scala nominale** (codice paese).
- **country_txt — testuale** (nome paese corrispondente al codice).
- **region (ID) — numerica discreta, scala nominale** (codice regione).
- **region_txt — testuale** (nome della regione).

II) Informazioni sull'Incidente

- **incident — categorica binaria/testuale, scala nominale** (0/1 o descrizione: se l'evento è considerato un incidente registrato).
- **specificity — numerica discreta, scala ordinale** (grado di specificità della localizzazione).
- **vicinity — categorica binaria/testuale, scala nominale** (0/1 o descrizione: indica se l'attacco è nella città/prossimità).
- **location — testuale** (descrizione della località).
- **summary, addnotes — testuali** (descrizioni libere).
- **crit1, crit2, crit3 — categoriche binarie, scala nominale** (0/1: criteri di inclusione GTD).
- **doubtterr — categorica, scala nominale** (dubbio sulla natura terroristica).
- **alternative, alternativetype — categoriche, scala nominale** (classificazione alternativa se "doubt terrorism").
- **multiple — categorica binaria, scala nominale** (0/1: evento multiplo/coordinato).
- **success — categorica binaria, scala nominale** (0/1: attacco riuscito).
- **suicide — categorica binaria, scala nominale** (0/1: suicidio coinvolto).

- **attacktype1, attacktype1_txt; attacktype2/3, attacktype2_txt/3_txt — categoriche, scala nominale** (codici e descrizioni del tipo di attacco; valori codificati come es. Assalto armato, Attentato esplosivo, Incendio, Sequestro, ecc.).

III) Localizzazione dell'Incidente

- **provstate — testuale** (nome provincia/stato).
- **city — testuale** (nome città; può essere "Unknown").
- **latitude, longitude — numeriche continue, scala ratio** (coordinate geografiche; NA se non note).
- **location** — vedi sezione II.
- **region, region_txt; country, country_txt** — vedi sezione I.
- **specificity — categorica, scala ordinale** (precisione delle coordinate; valori codificati come es. city centroid, neighborhood, country-level, ecc.).

IV) Informazioni sull'Attacco

- **success, suicide** — vedi sezione II.
- **attacktype1/2/3, attacktype*_txt** — vedi sezione II.
- **targtype1/2/3, targtype*_txt** — **categoriche, scala nominale** (tipo di bersaglio; valori codificati come es. Civili, Forze di polizia, Governi, Industria, Religiosi, ecc.).
- **target1/2/3 — testuali** (nome o descrizione del bersaglio).
- **natlty1/2/3, natlty*_txt** — **numeriche discrete, scala nominale / testuali** (codice e nome della nazionalità del bersaglio; NA se non applicabile).

V) Informazioni sulle Armi

- **weaptype1/2/3, weaptype*_txt** — **categoriche, scala nominale** (tipo principale arma; valori codificati come es. Esplosivi, Firearms, Incendiari, Chimico, Biological, ecc.).
- **weapsubtype1/2/3, weapsubtype*_txt** — **categoriche, scala nominale** (sottotipo arma valori codificati come es. bomba a mano, fucile automatico, pipe bomb, ecc.).
- **weapdetail — testuale** (dettagli liberi sull'arma o modalità d'uso).

VI) Informazioni sul Bersaglio/Vittima

- **targtype1/2/3, targtype*_txt** — vedi sezione IV.
- **target1/2/3** — vedi sezione IV.
- **natlty1/2/3, natlty*_txt** — vedi sezione IV.

VII) Informazioni sui Responsabili

- **gname, gsubname — testuali** (nome del gruppo terroristico responsabile e sottogruppo; valori codificati come es. LTTE, PKK, Taliban, ISIL, FARK, ecc).

- **guncertain1/2/3 — categorica binaria, scala nominale** (0/1: indica il livello di incertezza nell'assegnazione del gruppo).
- **individual — categorica/testuale, scala nominale** (flag o descrizione se si tratta di individuo/i non affiliati).
- **nperps, nperpcap — numeriche discrete, scala ratio** (numero di perpetratori segnalati/idenrificati; numero perpetratori catturati).
- **claimed — categorica binaria, scala nominale** (0/1: rivendicazione presente).
- **claimmode/2/3, claimmode*_txt (e claim2/3 e relativi claimmode)** — **categoriche, scala nominale** (modalità di rivendicazione; valori codificati come es. comunicato stampa, video, tramite intermediario, ecc.).

VIII) Vittime e Conseguenze

- **nkill, nkillus, nkillter — numeriche discrete, scala ratio** (numero morti totali; numero morti cittadini USA morti; numero terroristi/colpevoli morti).
- **nwound, nwoundus, nwoundte — numeriche discrete, scala ratio** (numero feriti totali; numero cittadini USA feriti; numero terroristi feriti).
- **property, propextent, propextent_txt — categoriche/testuali, scala nominale** (se c'è danno a proprietà; estensione del danno; descrizione).
- **propvalue — numerica continua, scala ratio** (valore stimato danni materiali, NA se non disponibile).
- **propcomment — testuale** (note sui danni).
- **ishostkid — categorica binaria, scala nominale** (flag relativi rapimenti/ostaggi).
- **nhostkid, nhostkidus — numeriche discrete, scala ratio** (numero di rapimenti/ostaggi; numero di rapimenti/ostaggi USA).
- **nhours, ndays — numeriche discrete, scala ratio** (durata dell'evento in ore/giorni; NA se non nota).
- **ransom, ransomamt, ransomamtus, ransomnote, ransompaid, ransompaidus — categoriche/numeriche/testuali , scala nominale o ratio** a seconda del campo (informazioni su richieste di riscatto, importi e pagamenti).
- **hostkidoutcome, hostkidoutcome_txt — categoriche, scala nominale** (esito del rapimento; valori codificati come es. liberati, uccisi, rimangono ostaggi, ecc.).

IX) Informazioni aggiuntive e Fonti

- **motive — testuale** (descrizione libera della presunta motivazione).
- **addnotes** — vedi sezione II.
- **scite1, scite2, scite3 — testuali** (citazioni delle fonti principali usate per la voce).

- **dbsource — testuale/categorica, scala nominale** (indicazione della fonte di origine del record/ente collezionista).
- **related — testuale/numerico, scala ratio** (ID correlati ad altri eventid che indicano se l'evento fa parte di un'azione coordinata).

(per ulteriori dettagli consultare il codebook)

Dal dizionario possiamo porci delle domande e definire gli obiettivi strada facendo.

1.3: Definizione degli obiettivi dell'analisi dei dati

Ecco le domande sul nostro dataset che saranno oggetto di analisi, seguendo il modello delle **5 Ws + H di Problem Solving** e **Analisi dei Dati** secondo il **pattern** suggerito da **Modern Analyst** (fonte: https://www.modernanalyst.com/Careers/InterviewQuestions/tabid/128/ID/5829/What-are-the-5-W-questions-plus-H.aspx):

- **Quali sono i continenti o i paesi che subiscono più/meno attacchi terrostici? (Chi?/Dove?)**
- **C'è una correlazione tra i luoghi attaccati e il bersaglio dell'attacco? (Che cosa?)**
- **In quali periodi c'è stata più concentrazione di attacchi? (Quando?)**
- **Quali sono i principali obiettivi di un attacco terroristico? (Perché?)**
- **Quali sono le modalità più comuni in cui avvengono? (Come?)**
- **Quanto sono letali i vari gruppi terroristici e quante vittime hanno causato? (Quanto?)**

FASE 2: Pulizia e Preprocessing (Data Cleaning & Preparation)

2.1: Gestione Valori Mancanti (NaN)

2.1.1: Identificare le variabili con dati mancanti e quantificare la loro incidenza

Attraverso la stampa delle prime 5 righe, si può osservare come ci siano delle colonne con dei valori nulli.

Possiamo verificare la percentuale di valori nulli per ogni colonna e magari, escludere alcune di esse perchè irrilevante al data analysis.

```
nan_percentage = terrorism.isna().mean().sort_values(ascending=False)

columns_to_drop = nan_percentage[nan_percentage > 0.9].index
print("Possibili colonne da rimuovere o ignorare:")
print(columns_to_drop)
```

```
print("Totale colonne con percentuale alta di valori
nulli:",len(columns_to_drop))
```

```
Possibili colonne da rimuovere o ignorare:
Index(['gsubname3', 'weapsubtype4_txt', 'weapsubtype4', 'weaptype4',
       'weaptype4_txt', 'claimmode3', 'claimmode3_txt', 'gsubname2',
'claim3',
       'guncertain3', 'gname3', 'divert', 'attacktype3',
'attacktype3_txt',
       'ransomnote', 'ransompaidus', 'ransomamtus', 'claimmode2',
       'claimmode2_txt', 'ransompaid', 'corp3', 'targsubtype3',
       'targsubtype3_txt', 'natlty3_txt', 'natlty3', 'target3',
       'targtype3_txt', 'targtype3', 'ransomamt', 'weapsubtype3_txt',
       'weapsubtype3', 'claim2', 'weaptype3_txt', 'weaptype3',
'guncertain2',
       'gname2', 'resolution', 'kidhijcountry', 'nhours', 'compclaim',
       'gsubname', 'attacktype2', 'attacktype2_txt', 'ndays',
'approxdate',
       'corp2', 'nreleased', 'targsubtype2', 'targsubtype2_txt',
'natlty2',
       'natlty2_txt', 'target2', 'hostkidoutcome_txt',
'hostkidoutcome',
       'targtype2_txt', 'targtype2', 'weapsubtype2',
'weapsubtype2_txt',
       'weaptype2', 'weaptype2_txt', 'nhostkidus', 'nhostkid',
'claimmode_txt',
       'claimmode'],
      dtype='object')
Totale colonne con percentuale alta di valori nulli: 64
```

Possiamo osservare che molte di queste colonne presentano una percentuale elevata di valori nulli. Tuttavia, è utile verificare quanti valori non nulli stiamo effettivamente scartando. Ad esempio, prendendo in considerazione la colonna ****claimmode****, che ha una percentuale di valori nulli superiore al 90%, ci accorgiamo che tra le colonne con il maggior numero di nulli, è comunque una di quelle con il numero più alto di valori presenti.

```
valuenotnull = terrorism['claimmode'].notna().sum()
print("Valori non nulli : {}".format(valuenotnull))
```

```
Valori non nulli : 16248
```

Possiamo osservare che, di fronte a oltre ****170.000 osservazioni****, avere poco più di ****15.000**** valori validi in alcune colonne rende queste informazioni poco rappresentative. Per questo motivo, al fine di ridurre la complessità del dataset e semplificare l'analisi, è ragionevole eliminare queste colonne. Inoltre, molte di queste colonne contengono **informazioni ridondanti** o estremamente dettagliate, che possono essere approssimate o già espresse da altre colonne principali. Ad esempio, le informazioni riguardanti i gruppi terroristici, rivendicazioni, bersagli secondari e armi multiple contengono dati rari che non influenzano le statistiche generali.

Eliminazione colonne nulle

```
# Elimina le colonne dal DataFrame
terrorism_cleaned = terrorism.drop(columns=columns_to_drop)

# Visualizza le prime righe del dataset pulito
terrorism_cleaned.head()
```

```
        eventid  iyear  imonth  iday  extended  country
country_txt  \
0  197000000001   1970       7     2         0       58  Dominican
Republic
1  197000000002   1970       0     0         0      130
Mexico
2  197001000001   1970       1     0         0      160
Philippines
3  197001000002   1970       1     0         0       78
Greece
4  197001000003   1970       1     0         0      101
Japan

   region                   region_txt provstate  ... addnotes  scite1
\
0       2  Central America & Caribbean       NaN  ...      NaN     NaN

1       1                North America       NaN  ...      NaN     NaN

2       5               Southeast Asia    Tarlac  ...      NaN     NaN

3       8               Western Europe    Attica  ...      NaN     NaN

4       4                    East Asia       NaN  ...      NaN     NaN


   scite2  scite3  dbsource INT_LOG INT_IDEO  INT_MISC  INT_ANY
related
0     NaN     NaN      PGIS       0        0         0        0
NaN
1     NaN     NaN      PGIS       0        1         1        1
NaN
2     NaN     NaN      PGIS      -9       -9         1        1
NaN
3     NaN     NaN      PGIS      -9       -9         1        1
NaN
4     NaN     NaN      PGIS      -9       -9         1        1
NaN

[5 rows x 71 columns]
```

2.2: Gestione Duplicati

```
# Visualizza le righe duplicate (senza rimuoverle)
duplicates = terrorism_cleaned[terrorism_cleaned.duplicated()]
print(f"Numero di righe duplicate trovate: {duplicates.shape[0]}")

Numero di righe duplicate trovate: 0
```

Come confermato dal codice, tramite l'utilizzo della funzione duplicated(), il dataset è privo di righe duplicate.

2.3: Correzione Inconsistenze e Formattazione

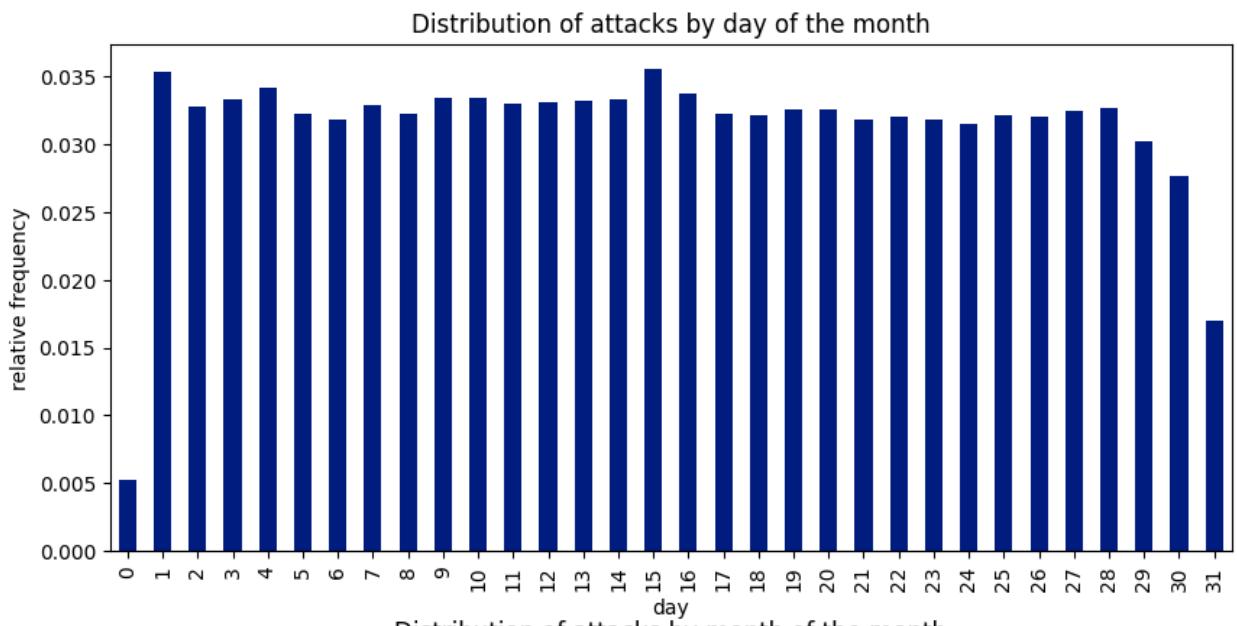
Una delle parti più importanti del Data Wrangling è la gestione delle inconsistenze.

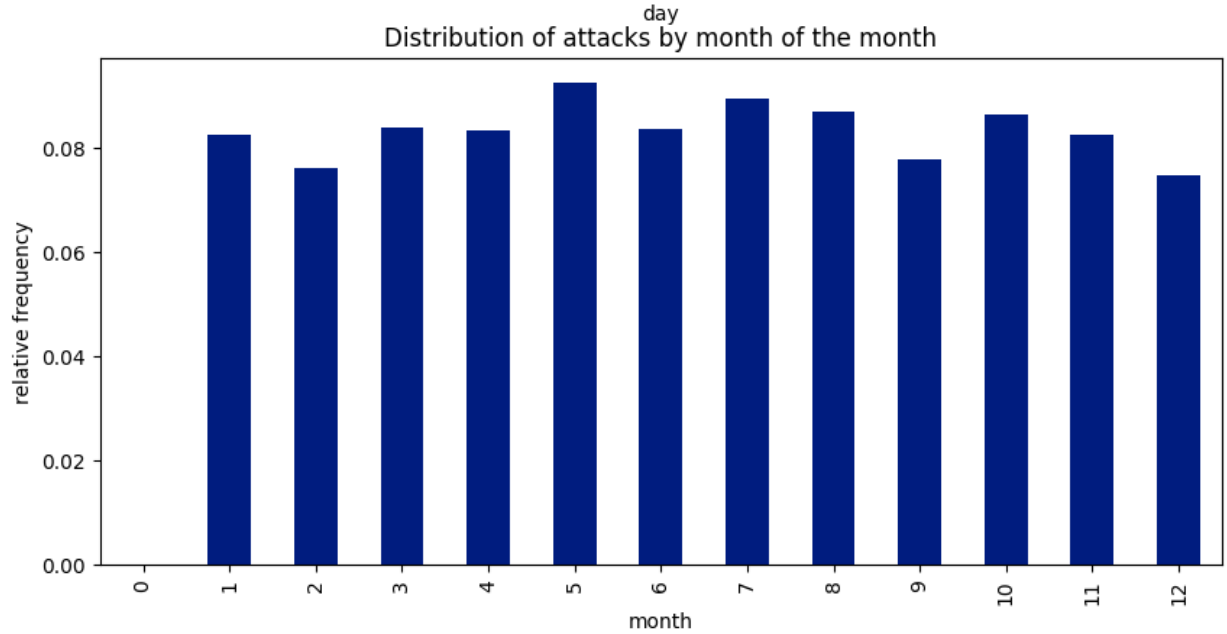
2.3.1 iday, imonth, iyear: inconsistenze e nuova feature date

Dall'analisi del dataset si nota un valore anomalo: lo **0** utilizzato nelle feature **imonth** e **iday**. Questo potrebbe causare problemi nel momento in cui si voglia unire giorno, mese e anno in una colonna di tipo datetime. Per questo motivo, è fondamentale risolvere questa incongruenza, sostituendo i valori 0 con un giorno valido oppure rimuovere l'evento che non ha un mese definito. Si effettua quindi una rapida analisi per verificare la quantità di valori anomali e quale strada intraprendere.

```
# selezioniamo in ordine crescente le frequenze assolute di giorno e
mese
from ctypes import sizeof


day_counter =
terrorism_cleaned['iday'].value_counts(normalize=True).sort_index()
month_counter =
terrorism_cleaned['imonth'].value_counts(normalize=True).sort_index()


plt.figure(figsize=(10,10))

plt.subplot(2,1,1)
day_counter.plot.bar(color=darkblue_palette)
plt.xlabel("day")
plt.ylabel("relative frequency")
plt.title("Distribution of attacks by day of the month")

plt.subplot(2,1,2)
month_counter.plot.bar(color=darkblue_palette)
plt.xlabel("month")
plt.ylabel("relative frequency")
plt.title("Distribution of attacks by month of the month")

plt.show()
```

```
day_counter = terrorism_cleaned['iday'].value_counts().sort_index()
month_counter =
terrorism_cleaned['imonth'].value_counts().sort_index()

print("Numero osservazioni day = 0:", day_counter[day_counter.index ==
0])
print("Numero osservazioni month = 0:",
month_counter[month_counter.index == 0])
```

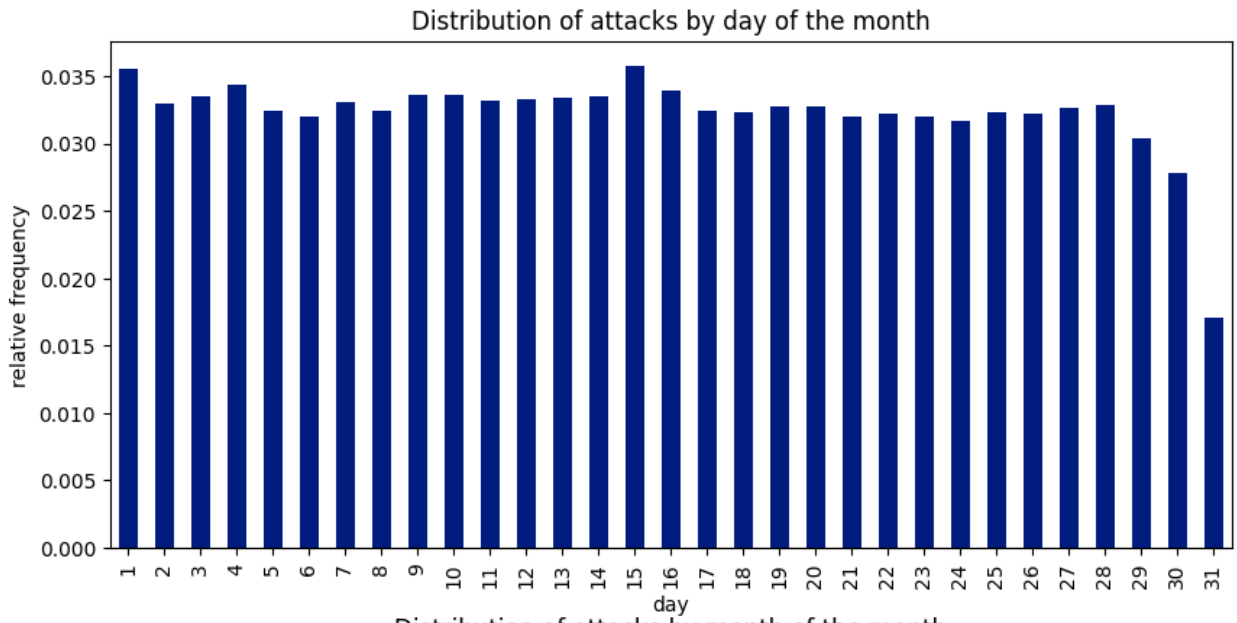


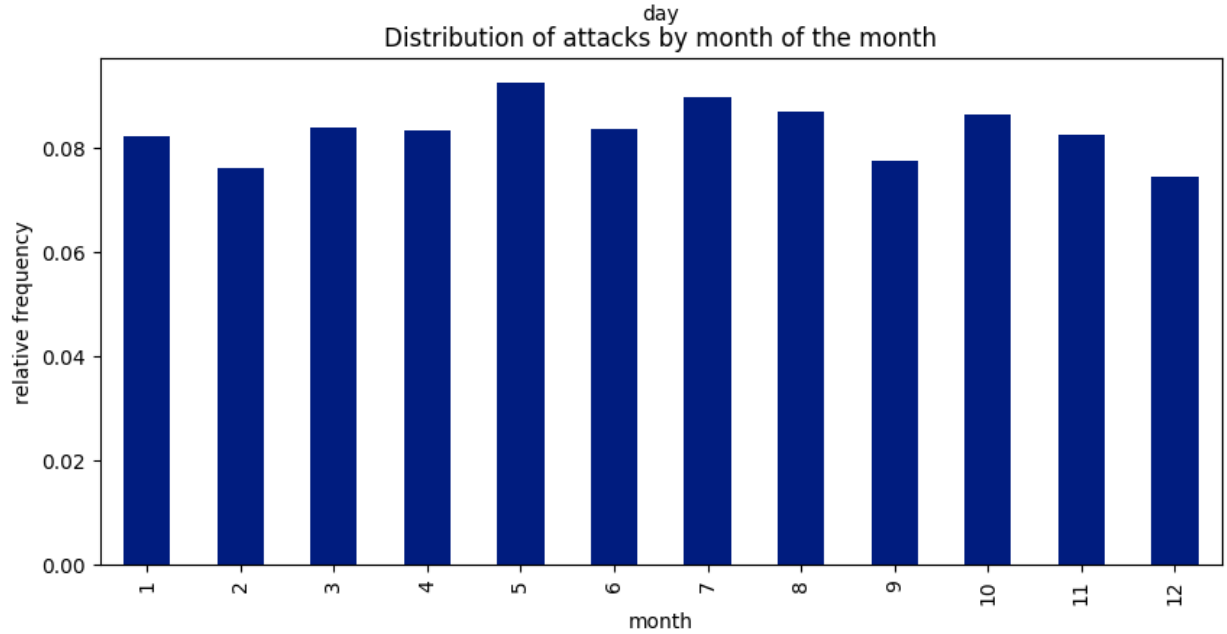


```
Numero osservazioni day = 0: iday
0    891
Name: count, dtype: int64
Numero osservazioni month = 0: imonth
0    20
Name: count, dtype: int64
```

Possiamo osservare che il numero di 0 presenti nella colonna **idays** e **imonth** è irrilevante. Quindi possiamo scartare le **900 osservazioni** coinvolte.

```
#eliminiamo i valori 0
terrorism_cleaned = terrorism_cleaned[terrorism_cleaned['iday'] !=0]
terrorism_cleaned = terrorism_cleaned[terrorism_cleaned['imonth'] !=0]

#Verifichiamo che non ci siano più occorrenze di 0 nelle due feature

day_counter =
terrorism_cleaned['iday'].value_counts(normalize=True).sort_index()
month_counter =
terrorism_cleaned['imonth'].value_counts(normalize=True).sort_index()

plt.figure(figsize=(10,10))
plt.subplot(2,1,1)
day_counter.plot.bar(color=darkblue_palette)
plt.xlabel("day")
plt.ylabel("relative frequency")
plt.title("Distribution of attacks by day of the month")

plt.subplot(2,1,2)
month_counter.plot.bar(color=darkblue_palette)
plt.xlabel("month")
plt.ylabel("relative frequency")
plt.title("Distribution of attacks by month of the month")




plt.show()
```





Adesso che il valore **0** non è più di ostacolo per la creazione della variabile di tipo **datetime**, possiamo procedere come segue:

```
terrorism_cleaned.rename(columns={'iyear':'year','imonth':'month','ida
y':'day'}, inplace=True)

terrorism_cleaned['date'] =
pd.to_datetime(terrorism_cleaned[['year','month','day']],
errors='coerce')
#Stampiamo le date > 25/12/2015
print("Stampa dei primi 10 attacchi terroristici che si sono
verificati dopo il 25/12/2025")
```

```
print(terrorism_cleaned[terrorism_cleaned['date'] > '2015-12-25']
['date'].head(10))
```

```
Stampa dei primi 10 attacchi terroristici che si sono verificati dopo
il 25/12/2025
156637   2015-12-26
156638   2015-12-26
156639   2015-12-26
156640   2015-12-26
156641   2015-12-26
156642   2015-12-26
156643   2015-12-26
156644   2015-12-26
156645   2015-12-26
156646   2015-12-26
Name: date, dtype: datetime64[ns]
```

Per rendere più interpretabile la variabile temporale, abbiamo convertito il formato **'date'** in **'giorni della settimana'**. Attraverso l'utilizzo della funzione **'.dt.day_name()'**, ogni osservazione presente nella variabile date è stata trasformata nel corrispondente giorno. La nuova variabile, è stata denominata **'weekday'**, permette di analizzare l'andamento degli eventi in funzione del giorno della settimana. Così possiamo facilmente individuare dei pattern legati alla settimana.

```
terrorism_cleaned['weekday'] = terrorism_cleaned['date'].dt.day_name()
print(terrorism_cleaned['weekday'].head(10))
```

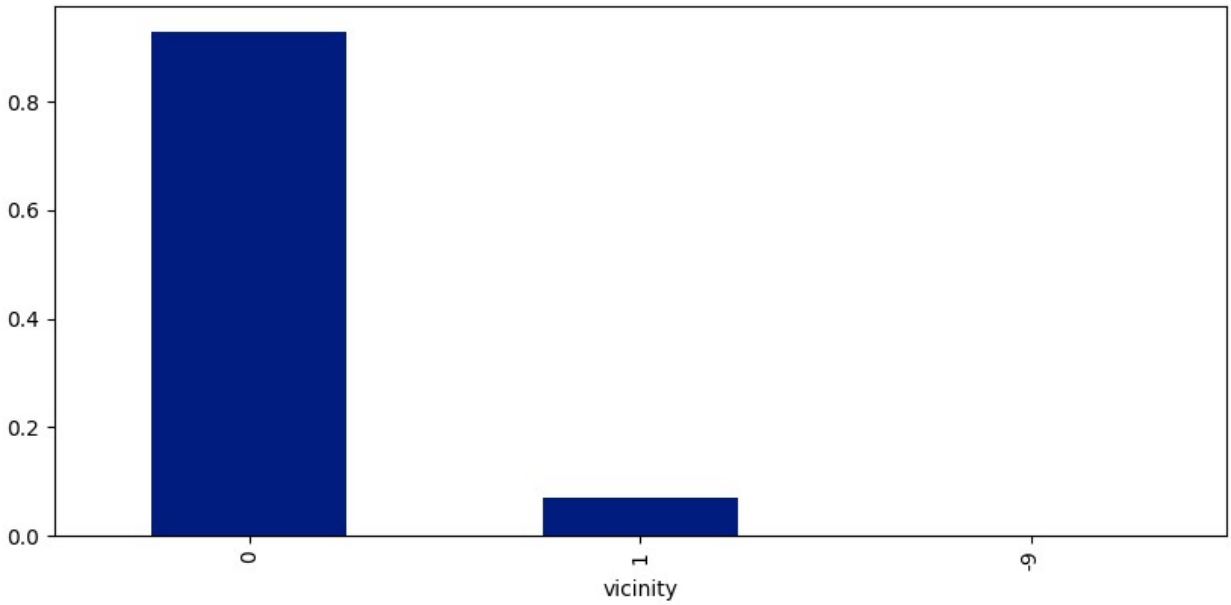
```
0     Thursday
5     Thursday
6       Friday
7       Friday
8       Friday
9     Saturday
10    Thursday
11     Tuesday
12    Thursday
13      Friday
Name: weekday, dtype: object
```

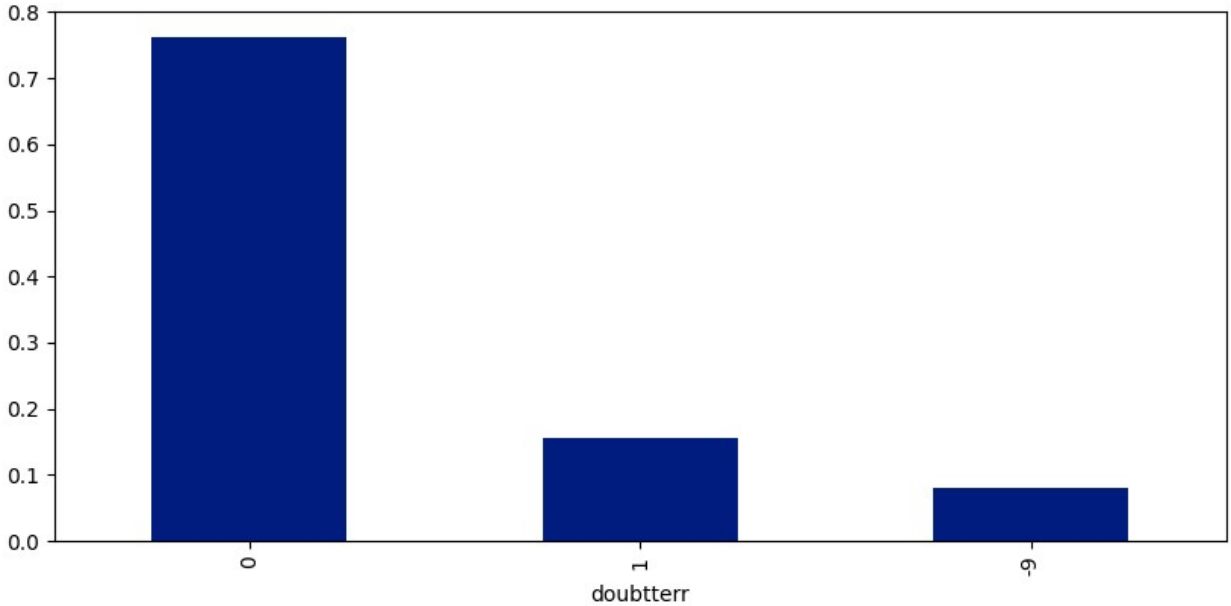
2.3.2 vicinity, doubterr: inconsistenze

Un'ulteriore incoerenza presente nel dataset riguarda la presenza di valori anomali in colonne che dovrebbero essere binarie. In particolare, nelle colonne ****vicinity**** e ****doubtterr****, oltre ai valori 0 e 1, è presente il valore ****-9****. La documentazione del Global Terrorism Database (GTD) indica che il dato non era disponibile o non veniva ancora raccolto nel periodo di riferimento (prima del 1997). Verifichiamo la quantità di valori presenti per ogni colonna, attraverso dei barplot, e scegliamo una strategia per gestire queste anomalie.

```
vicinity_count=
terrorism_cleaned['vicinity'].value_counts(normalize=True)
```

```
doubtterr_count=
terrorism_cleaned['doubtterr'].value_counts(normalize=True)

plt.figure(figsize=(10,10))
plt.subplot(2,1,1)
vicinity_count.plot.bar(color=darkblue_palette)

plt.subplot(2,1,2)
doubtterr_count.plot.bar(color=darkblue_palette)
plt.show()

print(vicinity_count)
print()
print(doubtterr_count)
```






```
vicinity
 0    0.930077
 1    0.069722
-9    0.000201
Name: proportion, dtype: float64

doubtterr
 0    0.763011
 1    0.156469
-9    0.080521
Name: proportion, dtype: float64
```

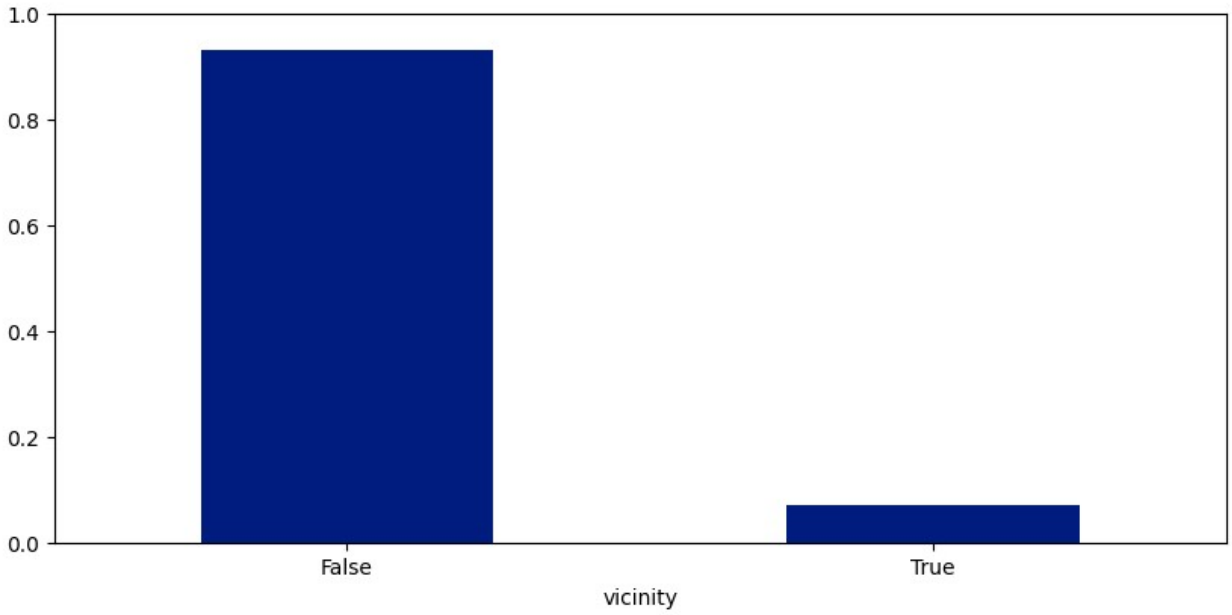
Possiamo osservare che:

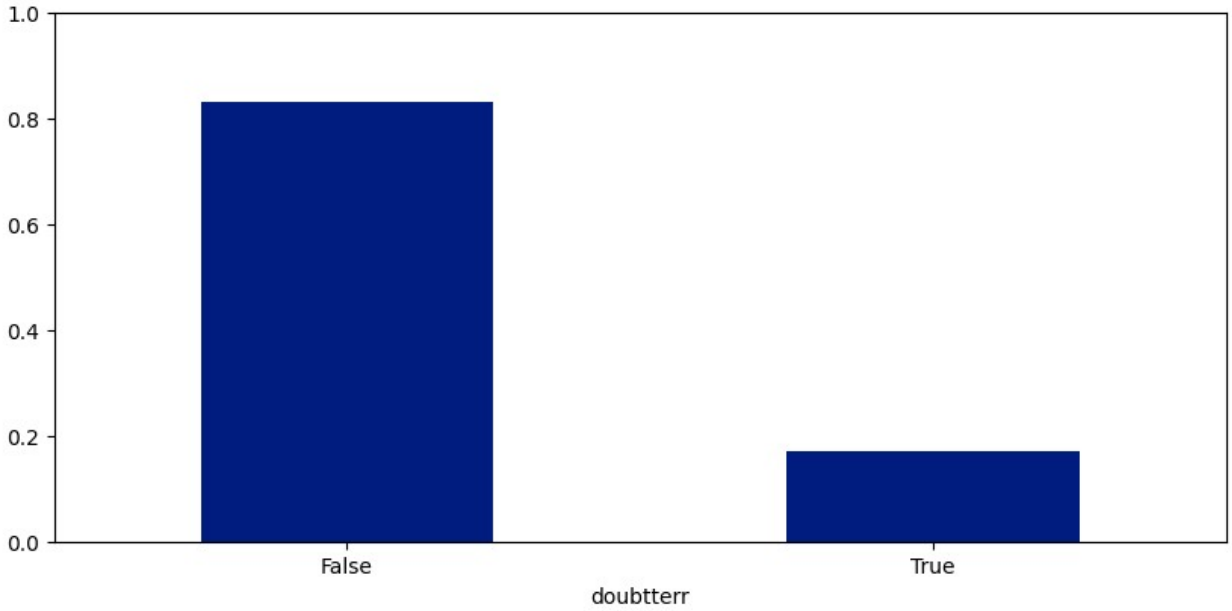
- ****vicinity**** contiene circa lo ****0,02%**** delle osservazioni con valore pari a ****-9****;
- ****doubtterr**** contiene circa l'****8%**** delle osservazioni con valore pari a ****-9****.

Dati questi risultati, è opportuno adottare strategie diverse:

- per ****vicinity****, poiché la quantità di valori anomali è trascurabile, possiamo rimuovere le osservazioni corrispondenti, in quanto non influiranno in modo significativo sull'analisi complessiva;
- per ****doubtterr****, invece, la presenza di un numero consistente di valori pari a -9 suggerisce di convertirli in NaN. In questo modo manterremo la ****colonna binaria (0 e 1)**** e tratteremo i casi mancanti come dati non disponibili, evitando di introdurre distorsioni nell'analisi.

```
terrorism_cleaned =
terrorism_cleaned[~terrorism_cleaned['vicinity'].isin([-9])]
terrorism_cleaned['doubtterr'] =
terrorism_cleaned['doubtterr'].replace(-9,np.nan)


terrorism_cleaned['vicinity'] =
terrorism_cleaned['vicinity'].astype('boolean')
terrorism_cleaned['doubtterr'] =
terrorism_cleaned['doubtterr'].astype('boolean')


vicinity_count=
terrorism_cleaned['vicinity'].value_counts(normalize=True)
doubtterr_count=
terrorism_cleaned['doubtterr'].value_counts(normalize=True)

plt.figure(figsize=(10,10))
plt.subplot(2,1,1)
vicinity_count.plot.bar(color=darkblue_palette)
plt.xticks(rotation=0)
plt.ylim(0,1)


plt.subplot(2,1,2)
doubtterr_count.plot.bar(color=darkblue_palette)
plt.xticks(rotation=0)
plt.ylim(0,1)
plt.show()
```





2.3.3 Ulteriori variabili binarie: Soglia di rimozione/sostituzione inconsistenze

Poiché anche **altre variabili binarie** presentano valori che indicano **dati sconosciuti**, applichiamo lo **stesso metodo di sostituzione** utilizzato in precedenza. In questo caso si adotta una **soglia del 10%**: se la percentuale di valori sconosciuti risulta **inferiore**, le osservazioni **si rimuovono**; altrimenti **i valori anomali si sostituiscono con NaN**.

```
coltofix =
['claimed','ransom','property','ishostkid','INT_LOG','INT_IDEO','INT_M
ISC','INT_ANY']
```

```python
coltodrop = []

for col in coltofix:
    terrorism_cleaned[col] = terrorism[col].replace(-9,np.nan)
    numnan = terrorism_cleaned[col].isna().sum()
    total = len(terrorism_cleaned[col])
    percnan = numnan/total
    print(f"La percentuale di NaN in {col} è: {percnan:.2%}")
    if percnan <= 0.1:
        coltodrop.append(col)
        print(f"Le row in {col} contenenti le osservazioni con valori
nulli sono state rimosse, perchè non superano la soglia.")
    else:
        print(f"Le row in {col} contenenti le osservazioni con valori
nulli sono state mantenute, perchè superano la soglia.")
    terrorism_cleaned[col] = terrorism_cleaned[col].astype('boolean')

terrorism_cleaned = terrorism_cleaned.dropna(subset=coltodrop)
print(f"Osservazioni rimanenti: {len(terrorism_cleaned)}")
```

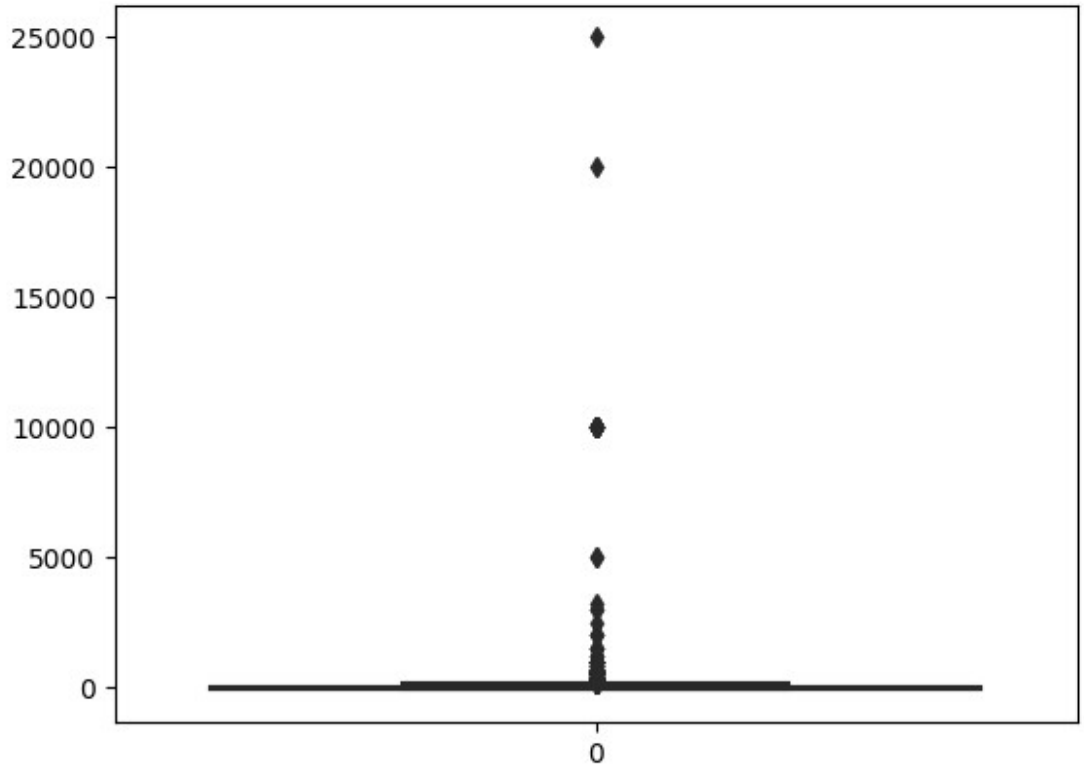
```
La percentuale di NaN in claimed è: 39.55%
Le row in claimed contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
La percentuale di NaN in ransom è: 56.16%
Le row in ransom contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
La percentuale di NaN in property è: 11.51%
Le row in property contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
La percentuale di NaN in ishostkid è: 0.26%
Le row in ishostkid contenenti le osservazioni con valori nulli sono
state rimosse, perchè non superano la soglia.
La percentuale di NaN in INT_LOG è: 51.37%
Le row in INT_LOG contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
La percentuale di NaN in INT_IDEO è: 51.45%
Le row in INT_IDEO contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
La percentuale di NaN in INT_MISC è: 0.28%
Le row in INT_MISC contenenti le osservazioni con valori nulli sono
state rimosse, perchè non superano la soglia.
La percentuale di NaN in INT_ANY è: 46.45%
Le row in INT_ANY contenenti le osservazioni con valori nulli sono
state mantenute, perchè superano la soglia.
Osservazioni rimanenti: 168508
```

2.4: Gestione Outlier (Valori Anomali)

2.4.1 nperps

Analizzando il dataset, possiamo notare delle incongruenze per la colonna **nperps**, che contiene importanti informazioni sul numero di autori che hanno compiuto l'atto terroristico. Come nei precendenti casi è presente **-99**,**-9** e inoltre **0**, che indicano un valore ignoto. Inoltre, sono presenti anche outlier che possono essere verificati attraverso dei boxplot.

```
sns.boxplot(terrorism_cleaned['nperps'], color='darkblue')
plt.show()
print(terrorism_cleaned['nperps'].describe())

print()

unknown_val =
len(terrorism_cleaned[terrorism_cleaned['nperps'].isin([-99,-9,0])])
print("I valori sconosciuti sono: {}".format(unknown_val))
```




```
count    98391.000000
mean       -64.001108
std        228.635776
min        -99.000000
25%        -99.000000
```

```
50%        -99.000000
75%          1.000000
max      25000.000000
Name: nperps, dtype: float64

I valori sconosciuti sono: 72610
```

Dall'analisi descrittiva ottenuta con il metodo **.describe()**, si osserva che la variabile *nperps*, che indica il numero di autori coinvolti in ciascun attacco, presenta diverse anomalie. In particolare, i valori **-99**, **-9** e **0** vengono utilizzati come valori per rappresentare dati mancanti, ma in realtà dovrebbero essere trattati come missing values (NaN). Inoltre, la presenza di valori estremamente elevati (ad esempio fino a **25.000**) suggerisce probabili errori di inserimento o outlier non realistici.

Per correggere questi problemi, con l'aiuto del GTD si decide di:

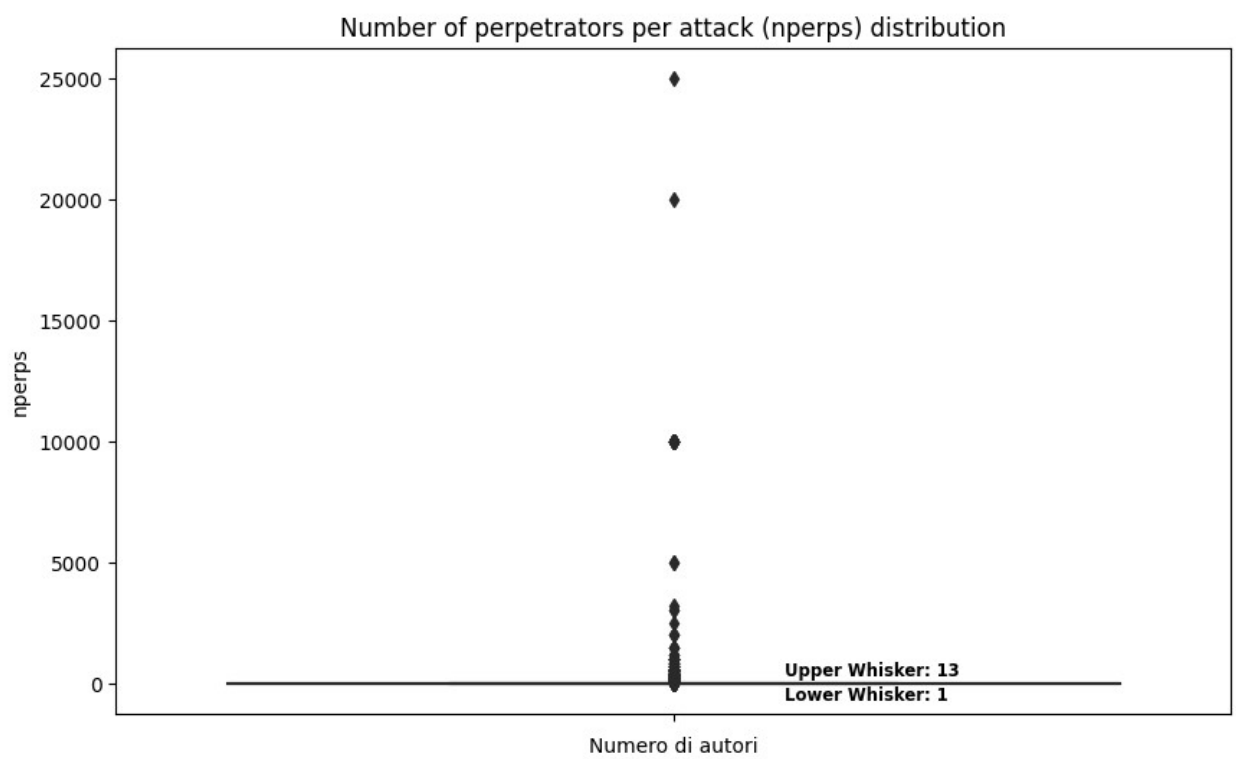
- Sostituire i valori minori di 1 con NaN (poiché non è plausibile che un attacco terroristico coinvolga meno di un autore) e creare una nuova feature chiamata ***nperps_missing*** contenente valori booleani che indicano se il valore di nperps è NaN o assume un valore.
- Mostrare nuovamente boxplot e informazioni ottenute attraverso .describe().

N.B. : Si crea la tabella **nperps_missing** in modo tale che quando si fa analisi dei dati, si prendano solo i dati che sono disponibili.

```
terrorism_cleaned['nperps'] = terrorism_cleaned['nperps'].replace([-
99,-9,0],np.nan)
terrorism_cleaned['nperps_missing'] =
terrorism_cleaned['nperps'].isna()



Q1 = terrorism_cleaned['nperps'].quantile(0.25)
Q3 = terrorism_cleaned['nperps'].quantile(0.75)
IQR = Q3-Q1


lowb = Q1-1.5*IQR
upb = Q3+1.5*IQR


vwtnan = terrorism_cleaned['nperps'].dropna()
lowwhis = vwtnan[vwtnan >= lowb].min()
upwhis = vwtnan[vwtnan <= upb].max()


print(upwhis)


plt.figure(figsize=(10,6))
sns.boxplot(y=terrorism_cleaned['nperps'], color='darkblue', whis=1.5)
```

```
plt.title('Number of perpetrators per attack (nperps) distribution')

plt.text(0.1, upwhis+500, f'Upper Whisker: {int(upwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')
plt.text(0.1, lowwhis-500, f'Lower Whisker: {int(lowwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')


plt.xlabel('Numero di autori')
plt.show()

print(terrorism_cleaned['nperps'].describe())
```

```
13.0
```




```
count    25781.000000
mean        34.056437
std        431.770415
min          1.000000
25%          1.000000
50%          2.000000
75%          6.000000
max      25000.000000
Name: nperps, dtype: float64
```

```
validvalue = (~terrorism_cleaned['nperps_missing']) &
(terrorism_cleaned.loc[~terrorism_cleaned['nperps_missing'], 'nperps']
<= upwhis)
#rende vera la condizione per tutti i valori minori di 13(non nulli)
nanvalue = terrorism_cleaned['nperps_missing'] #è vera la condizione
per tutti i valori NAN

valselected = validvalue|nanvalue #la condizione è vera per tutti i
valori minori 13 e i valori nulli


terrorism_cleaned = terrorism_cleaned[valselected] #esclude tutti i
valori maggiori di 13
```

2.4.2 nperpcap

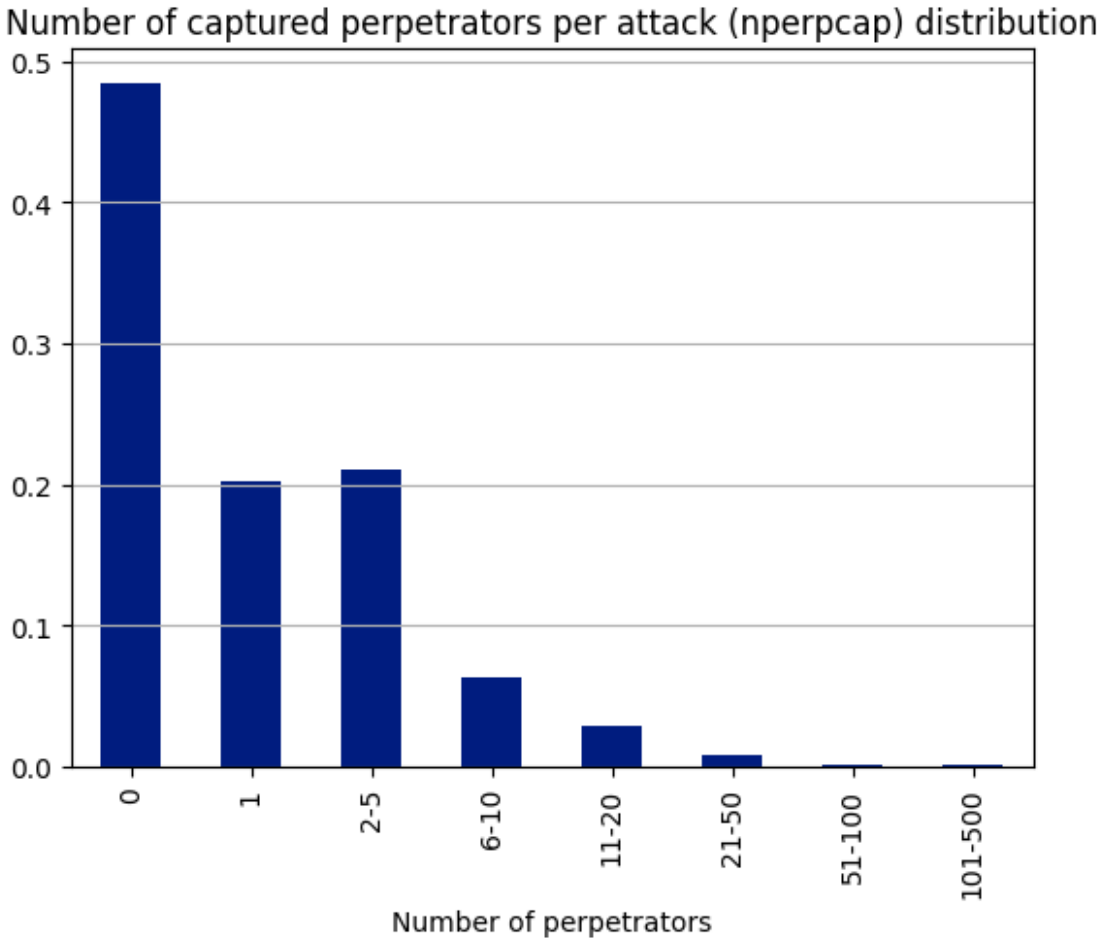
Dato che sono state apportate delle modifiche alla colonna ***nperps***, è necessario intervenire anche sulla colonna ***nperpcap***, che rappresenta il numero di autori catturati, per mantenere la coerenza tra le due variabili. Anche in questa feature troviamo delle inconsistenze date da valori sconosciuti, però in questo caso lo **0** si preserva dato che possono esserci autori non catturati.

```
unknown = terrorism_cleaned['nperpcap'].isin([-99,-9])
unknown = unknown.sum()

print(f"Numero di valori che assumono i valori -99 e -9: {unknown}")
print(f"I valori sono circa il
{unknown/len(terrorism_cleaned)*100:.2f}% del totale.")

print("Dato che sono pochi possiamo anche eliminarli del tutto.")
terrorism_cleaned =
terrorism_cleaned[~terrorism_cleaned['nperpcap'].isin([-99,-9])]

Numero di valori che assumono i valori -99 e -9: 1426
I valori sono circa il 0.87% del totale.
Dato che sono pochi possiamo anche eliminarli del tutto.
```

Ora possiamo verificare dove si concentrano i valori e infine come nella feature **nperps** rimuovere i valori maggiori a 13.

```
terrorism_cleaned['nperpcap_missing'] =
terrorism_cleaned['nperpcap'].isna()

bins =[0,1,2,5,10,20,50,100,500]
labels = ['0', '1', '2-5', '6-10', '11-20', '21-50', '51-100','101-
500']

captured_bins = pd.cut(terrorism_cleaned['nperpcap'], bins=bins,
labels=labels, right=True)
```

```
captured_bins.value_counts(normalize=True).sort_index().plot.bar(color
=darkblue_palette)
plt.title('Number of captured perpetrators per attack (nperpcap)
distribution')
plt.xlabel('Number of perpetrators')
plt.grid(axis='y')
plt.show()
```



Attraverso il seguente barplot come alcuni valori si possono scartare perchè degli outlier. Seguendo lo stesso principio del caso precedente, si tolgono tutti i valori maggiori di 13, perchè irrilevanti durante l'analisi.

```
validvalue = (~terrorism_cleaned['nperpcap_missing']) &
(terrorism_cleaned.loc[~terrorism_cleaned['nperpcap_missing'],
'nperpcap'] <= upwhis)

nanvalue = terrorism_cleaned['nperpcap_missing']
```

```
valselected = validvalue|nanvalue
terrorism_cleaned = terrorism_cleaned[valselected]
```

2.4.3 propvalue

Un'altra variabile numerica che contiene dei valori inconsistenti è **propvalue**. Applicheremo lo stesso approccio visto per le variabili numeriche precedenti.

```
propvalue = terrorism_cleaned['propvalue']
invalidprop = propvalue.isin([-99,-9]).sum()
print(f"Numero di elementi non validi:{invalidprop} circa il
{invalidprop/len(propvalue):.2%}")
nanprop = propvalue.isna().sum()
print(f"Numero di valori mancanti: {nanprop} circa il
{nanprop/len(propvalue):.2%}")
print(propvalue.describe())
```

```
Numero di elementi non validi:24040 circa il 14.80%
Numero di valori mancanti: 128741 circa il 79.25%
count    3.369800e+04
mean     2.097057e+05
std      1.631811e+07
min     -9.900000e+01
25%     -9.900000e+01
50%     -9.900000e+01
75%      2.000000e+03
max      2.700000e+09
Name: propvalue, dtype: float64
```

Vediamo come i valori non validi alterano le statistiche riassuntive. Possiamo sostituirli con NaN per ottenere statistiche e grafici che non siano influenzati da tali valori.

```
terrorism_cleaned['propvalue'] =
terrorism_cleaned['propvalue'].replace([-99,-9],np.nan)
terrorism_cleaned['propvalue_missing'] =
terrorism_cleaned['propvalue'].isna()

propvalue = terrorism_cleaned['propvalue']




Q1 = terrorism_cleaned['propvalue'].quantile(0.25)
Q3 = terrorism_cleaned['propvalue'].quantile(0.75)
IQR = Q3-Q1


lowb = Q1-1.5*IQR
upb = Q3+1.5*IQR
```

```
vwtnan = terrorism_cleaned['propvalue'].dropna()
lowwhis = vwtnan[vwtnan >= lowb].min()
upwhis = vwtnan[vwtnan <= upb].max()




plt.figure(figsize=(10,6))
sns.boxplot(y=terrorism_cleaned['propvalue'], color='darkblue',
whis=1.5)


plt.title('Distribution of Property Value')


plt.text(0.1, upwhis+50000000, f'Upper Whisker: {int(upwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')
plt.text(0.1, lowwhis-50000000, f'Lower Whisker: {int(lowwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')


plt.xlabel('Property Value (USD)')
plt.show()

print(terrorism_cleaned['propvalue'].describe())
```

```
count    9.658000e+03
mean     7.319366e+05
std      3.047578e+07
min      0.000000e+00
25%      5.000000e+03
50%      2.200000e+04
75%      5.000000e+04
max      2.700000e+09
Name: propvalue, dtype: float64
```

Dopo aver sostituito i valori mancanti e aver analizzato la variabile **propvalue** tramite boxplot e statistiche descrittive, emergono alcuni aspetti critici:

- la **deviazione standard** è molto **elevata**
- il **valore massimo** è di **diversi ordini di grandezza maggiore rispetto ai valori tipici**, circa **2.7 miliardi** su valori tipici che sono compresi tra **0** e **116500**.

I whisker calcolati tramite la formula dell'IRQ risultano:

- **whisker inferiore: 0**
- **whisker superiore: 116.500**

Dato che deviazione standard altri parametri statistici sono altamente influenzati da questi outlier, possiamo rimuoverli, al fine di avere statistiche più **affidabili** e **interpretabili**.

```
validvalue = (~terrorism_cleaned['propvalue_missing']) &
(terrorism_cleaned.loc[~terrorism_cleaned['propvalue_missing'],
'propvalue'] <= upwhis)

nanvalue = terrorism_cleaned['propvalue_missing']

valselected = validvalue|nanvalue
terrorism_cleaned = terrorism_cleaned[valselected]
```

2.4.4: nkill e nwound

```
coltofix = ['nkill','nwound']

coltodrop = []

for col in coltofix:
    numnan = terrorism_cleaned[col].isna().sum()
    total = len(terrorism_cleaned[col])
    percnan = numnan/total
    print(f"La percentuale di NaN in {col} è: {percnan:.2%}")
    if percnan <= 0.1:
        coltodrop.append(col)
        print(f"Le row in {col} contenenti le osservazioni con valori
nulli sono state rimosse, perchè non superano la soglia.")
    else:
        print(f"Le row in {col} contenenti le osservazioni con valori
nulli sono state mantenute, perchè superano la soglia.")

terrorism_cleaned = terrorism_cleaned.dropna(subset=coltodrop)
print(f"Osservazioni rimanenti: {len(terrorism_cleaned)}")

for col in coltodrop:
    Q1 = terrorism_cleaned[col].quantile(0.25)
    Q3 = terrorism_cleaned[col].quantile(0.75)
    IQR = Q3-Q1
    upwhis = Q3+1.5*IQR
    lowwhis = Q1-1.5*IQR
    terrorism_cleaned =
terrorism_cleaned[terrorism_cleaned[col].between(lowwhis,upwhis)]
```

```
La percentuale di NaN in nkill è: 5.58%
Le row in nkill contenenti le osservazioni con valori nulli sono state
rimosse, perchè non superano la soglia.
La percentuale di NaN in nwound è: 8.75%
Le row in nwound contenenti le osservazioni con valori nulli sono
state rimosse, perchè non superano la soglia.
Osservazioni rimanenti: 146785
```

Fase 3: Analisi Esplorativa

3.1: Analisi Univariata

Un passo importante per l'analisi esplorativa è sicuramente quello dell'**analisi univariata**, per calcolare gli **indici statistici** ed eventuali **visualizzazioni** delle variabili numeriche, e poi mostrare la **distribuzione** delle categorie nel caso delle variabili categoriche.

Per fare ciò, prima di tutto visualizziamo con uno script opportuno quali sono tutte le variabili dei tre tipi di dati visti nel punto **1.1** (**float64**, **int64** e **object**), sapendo che float64 si riferisca alle variabili numeriche continue, int64 alle variabili numeriche discrete e object alle variabili categoriche:

```
print(terrorism_cleaned.dtypes.to_string())
```

```
eventid                        int64
year                           int64
month                          int64
day                            int64
extended                       int64
country                        int64
country_txt                   object
region                         int64
region_txt                    object
provstate                     object
city                          object
latitude                     float64
longitude                    float64
specificity                  float64
vicinity                     boolean
location                      object
summary                       object
crit1                          int64
crit2                          int64
crit3                          int64
doubtterr                    boolean
alternative                  float64
alternative_txt               object
multiple                       int64
success                        int64
suicide                        int64
attacktype1                    int64
attacktype1_txt               object
targtype1                      int64
targtype1_txt                 object
targsubtype1                 float64
targsubtype1_txt              object
corp1                         object
target1                       object
```

```
natlty1                      float64
natlty1_txt                   object
gname                         object
motive                        object
guncertain1                  float64
individual                     int64
nperps                       float64
nperpcap                     float64
claimed                      boolean
weaptype1                      int64
weaptype1_txt                 object
weapsubtype1                 float64
weapsubtype1_txt              object
weapdetail                    object
nkill                        float64
nkillus                      float64
nkillter                     float64
nwound                       float64
nwoundus                     float64
nwoundte                     float64
property                     boolean
propextent                   float64
propextent_txt                object
propvalue                    float64
propcomment                   object
ishostkid                    boolean
ransom                       boolean
addnotes                      object
scite1                        object
scite2                        object
scite3                        object
dbsource                      object
INT_LOG                      boolean
INT_IDEO                     boolean
INT_MISC                     boolean
INT_ANY                      boolean
related                       object
date                  datetime64[ns]
weekday                       object
nperps_missing                  bool
nperpcap_missing                bool
propvalue_missing               bool
```

3.1.1: Paesi colpiti da piu' attacchi terroristici

Possiamo già rispondere alla prima domanda, cioè identificare i **paesi che subiscono più attacchi terroristici e quelli che ne subiscono meno**. Dato che il numero totale di paesi è molto elevato, è preferibile selezionare solo quelli con il maggior/minor numero di attacchi, così da

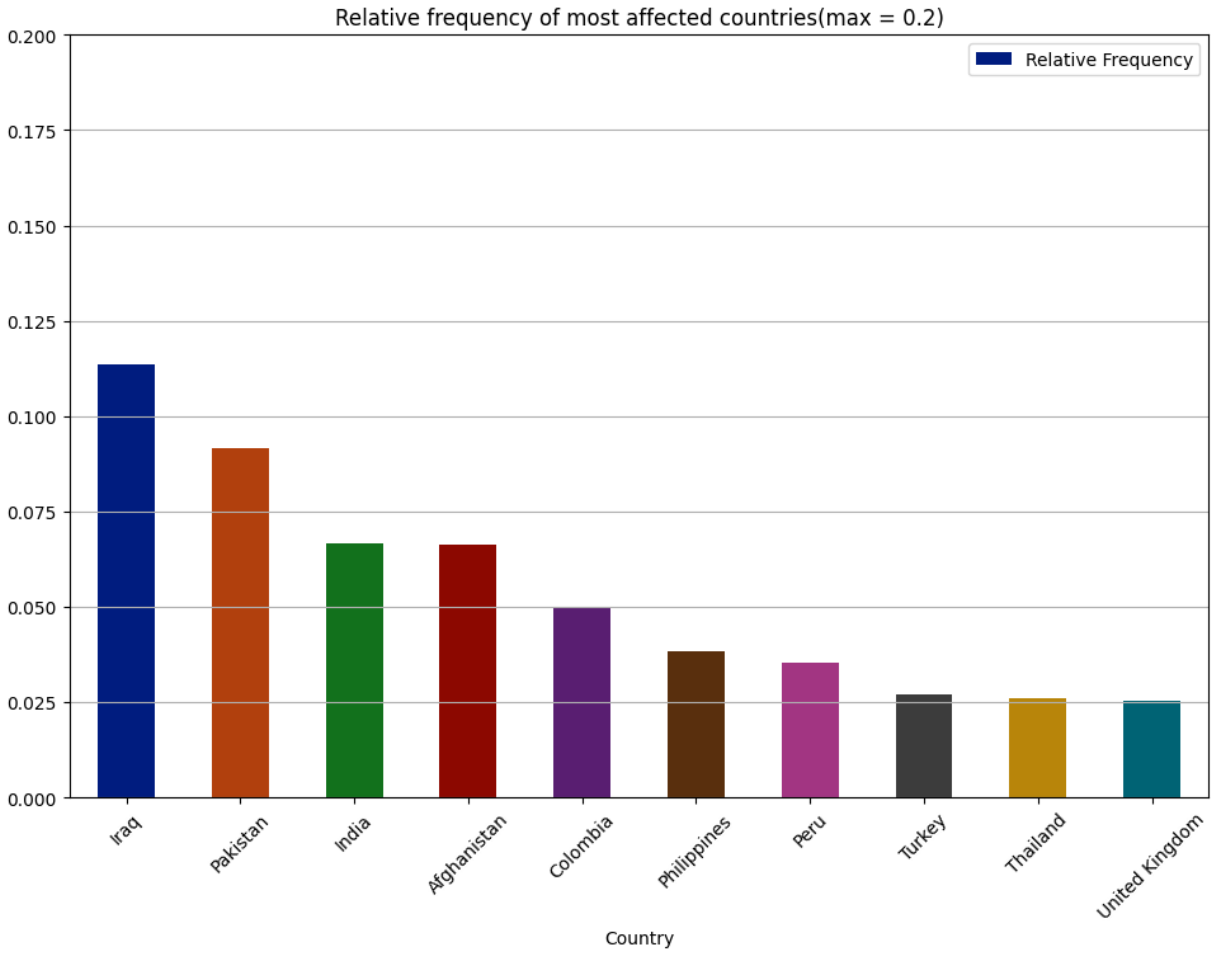
ottenere grafici leggibili e facilmente interpretabili, evitando sovraffollamento e confusione. Possiamo già verificare il numero di paesi diversi all'interno del dataset nel seguente modo.

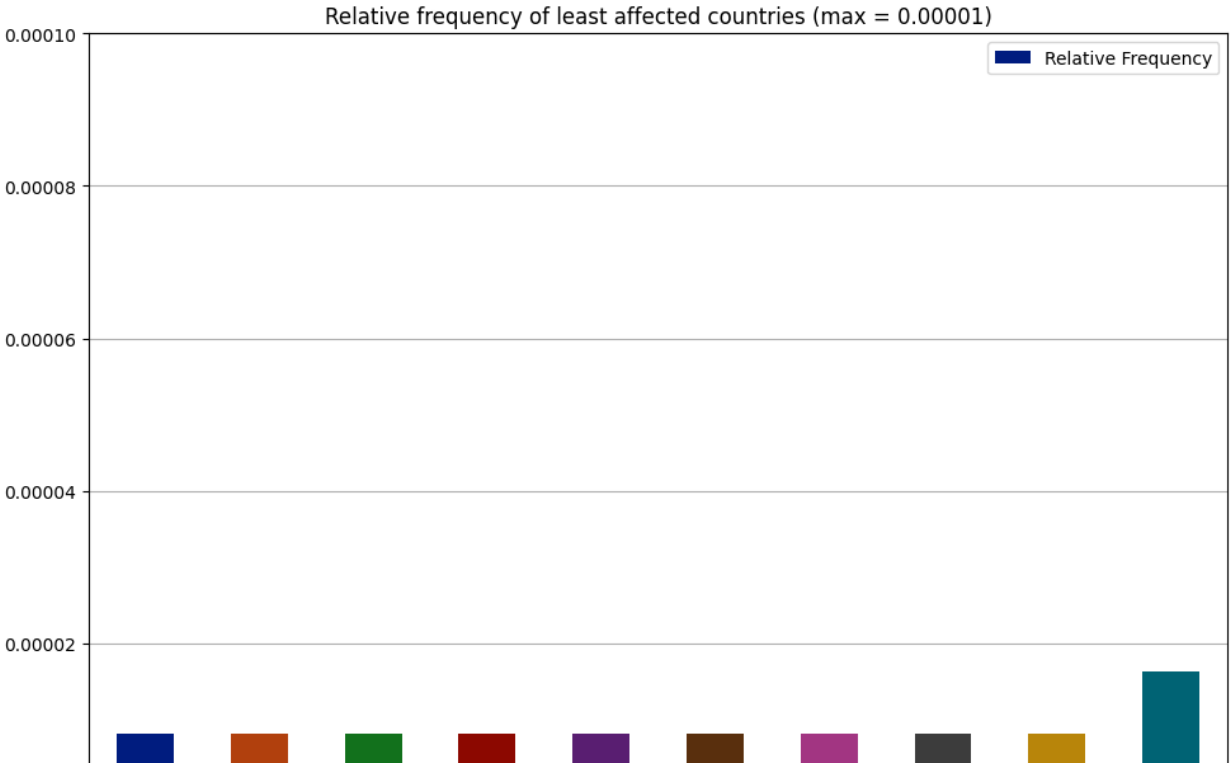
```
num_country = terrorism_cleaned['country_txt'].nunique()
print(f"Il numero di paesi presenti all'interno del dataset è
{num_country}")
```

```
Il numero di paesi presenti all'interno del dataset è 198
```

```
country_abs = terrorism_cleaned['country_txt'].value_counts()
country_rel = country_abs/len(terrorism_cleaned)


freq_abs_rel = pd.DataFrame({
    'Country': country_abs.index,
    'Absolute Frequency':country_abs.values,
    'Relative Frequency': country_rel.values})

leastaffected = freq_abs_rel.sort_values(by='Absolute
Frequency',ascending=True).head(10)
mostaffected = freq_abs_rel.sort_values(by='Absolute
Frequency' ,ascending=False).head(10)

print("Frequenza assoluta e relativa dei paesi più attaccati:")
print(mostaffected)
print()

print("Frequenza assoluta e relativa dei paesi meno attaccati:")
print(leastaffected)


fig, axes = plt.subplots(2,1, figsize=(10,16))

mostaffected.plot.bar(x='Country',y='Relative Frequency', ax =
axes[0], color=dark_palette)
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=45)
axes[0].set_title("Relative frequency of most affected countries(max =
0.2)")
axes[0].set_ylim(0,0.20)
axes[0].grid(axis='y')


leastaffected.plot.bar(x='Country',y='Relative Frequency', ax =
axes[1], color=dark_palette)
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=45)
axes[1].set_title("Relative frequency of least affected countries (max
= 0.00001)")
axes[1].set_ylim(0,0.0001)
axes[1].grid(axis='y')
plt.tight_layout()
plt.show()
```

```
Frequenza assoluta e relativa dei paesi più attaccati:
          Country  Absolute Frequency  Relative Frequency
0            Iraq               13880            0.113437
1        Pakistan               11200            0.091534
2           India                8162            0.066705
3     Afghanistan                8128            0.066427
4        Colombia                6123            0.050041
5     Philippines                4671            0.038175
6            Peru                4331            0.035396
7          Turkey                3295            0.026929
8        Thailand                3200            0.026153
9  United Kingdom                3117            0.025474

Frequenza assoluta e relativa dei paesi meno attaccati:
                            Country  Absolute Frequency  Relative
Frequency
197                          Brunei                   1
0.000008
189  People's Republic of the Congo                   1
0.000008
190               Wallis and Futuna                   1
0.000008
191                    New Hebrides                   1
0.000008
196             St. Kitts and Nevis                   1
0.000008
193                        Dominica                   1
0.000008
195                    Vatican City                   1
0.000008
192                     North Korea                   1
0.000008
194                         Vanuatu                   1
0.000008
188                        Barbados                   2
0.000016
```

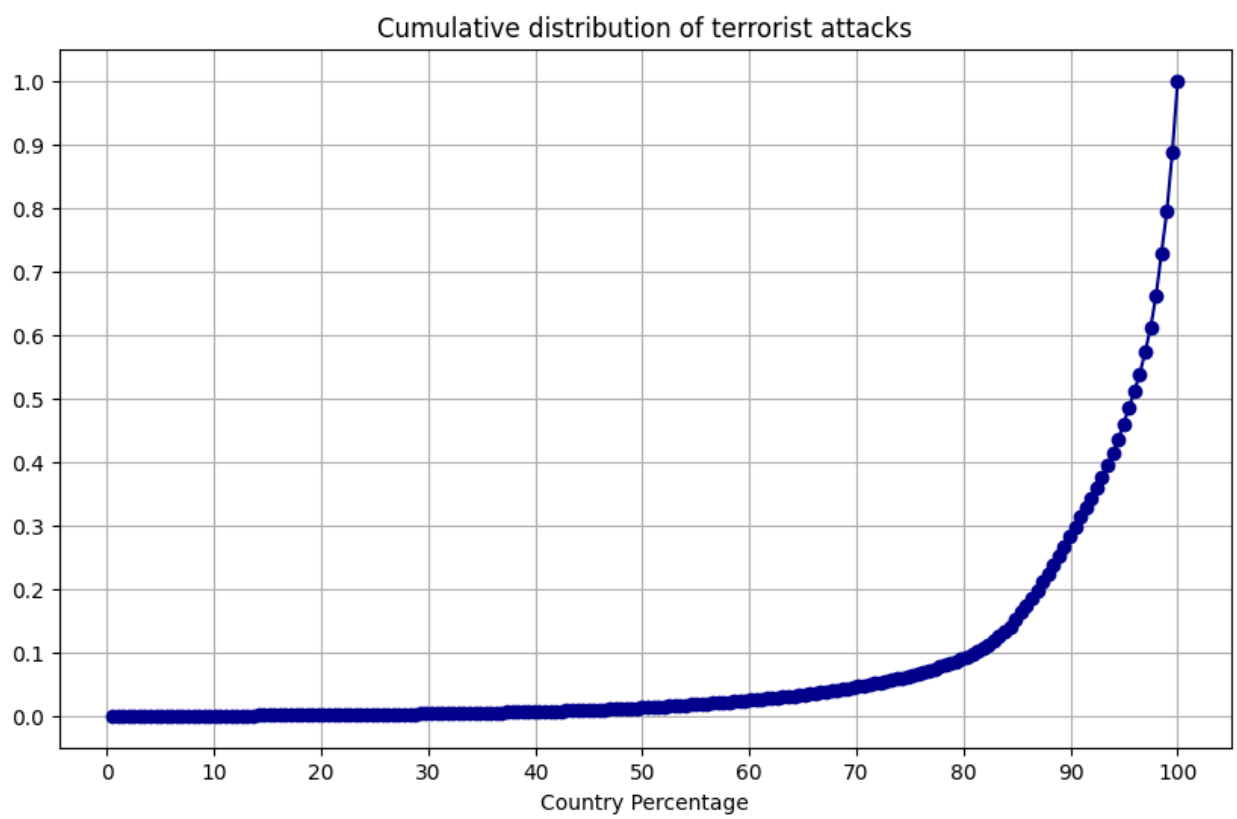

Relative frequency of most affected countries(max = 0.2)
Relative Frequency
0.200
0.175
0.150
0.125
0.100
0.075
0.050
0.025
0.000
Iraq
Pakistan
India
Afghanistan
Colombia
Philippines
Peru
Turkey
Thailand
United Kingdom
Country


Relative frequency of least affected countries (max = 0.00001)
Relative Frequency
0.00010
0.00008
0.00006
0.00004
0.00002

Dai grafici emerge un forte squilibrio nella distribuzione degli attacchi terroristici.

- Alcuni paesi sono **altamente colpiti**, ad esempio: *Iraq(circa 13,5%)*, *Pakistan(circa 8%)* e *Afghanistan(circa 7%)*.
- Altri paesi come *Vatican City (0.00006%)*, *North Korea(0.00006%)* o *South Vietnam(0.00006%)* hanno subito **pochissimi attacchi terroristici**, con percentuali trascurabili. Possiamo vedere se c'è uno squilibrio nella distribuzione attraverso la distribuzione cumulativa empirica. Se la curva sale molto rapidamente nei primi paesi, significa che pochi paesi concentrano la maggior parte degli attacchi, evidenziando una distribuzione fortemente sbilanciata.

```
ecdf = country_rel.sort_values().cumsum()
xperc = np.arange(1,num_country+1)/ num_country * 100
plt.figure(figsize=(10,6))

plt.plot(xperc,ecdf, marker='o',linestyle='-', color='darkblue')
plt.grid()
plt.xlabel("Country Percentage")
plt.xticks(np.arange(0,101,10))
plt.yticks(np.arange(0,1.10,0.1))
plt.title("Cumulative distribution of terrorist attacks")
plt.show()
```



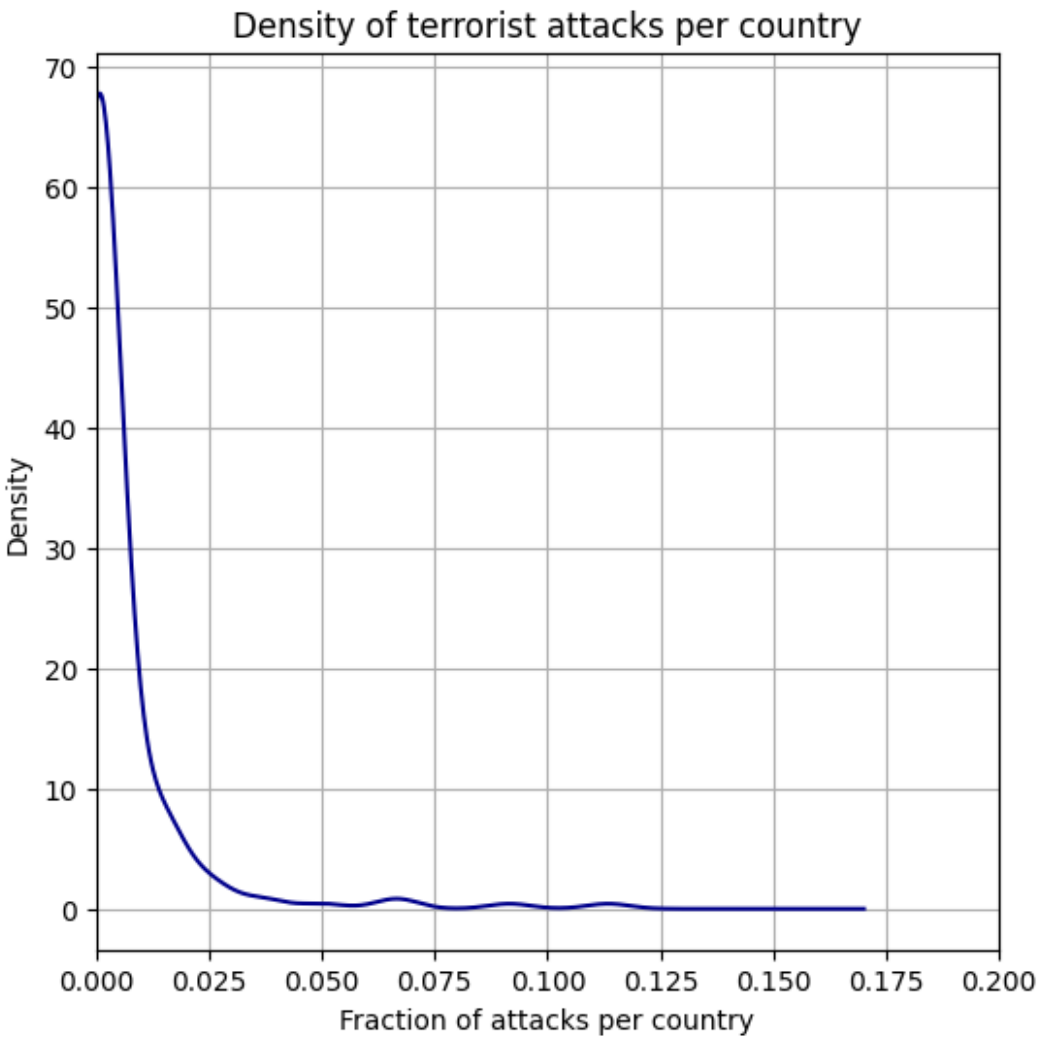
La curva mostra una **forte concentrazione di attacchi terroristici in pochi paesi**, mentre la maggior parte del mondo è relativamente poco colpita. A sostegno di ciò, osserviamo che **l'80% dei paesi subisce meno del 10% degli attacchi complessivi**.

Le possibili cause di questo squilibrio includono:

- politiche interne impopolari nei paesi più colpiti;
- presenza di diverse religioni all'interno di uno stesso paese;
- dati potenzialmente non veritieri in paesi soggetti a dittature o controlli autoritari.

Verifichiamo quindi come sono distribuiti i dati attraverso la kernel density estimate.

```
plt.figure(figsize=(6,6))
country_rel.plot(kind='density', color='darkblue')
plt.xlabel("Fraction of attacks per country")
plt.ylabel("Density")
plt.xlim(0, 0.2)
plt.title("Density of terrorist attacks per country")
plt.grid(True)
plt.show()

print()
print()
skew = country_rel.skew()
kurt = country_rel.kurtosis()
print(f"Skewness: {skew:.2f}")
print(f"Kurtosis: {kurt:.2f}")
```




```
Skewness: 4.97
Kurtosis: 29.48
```

La curva del density plot mostra che la maggior parte dei paesi ha una frazione di attacchi terroristici molto bassa, quasi 0%. Man mano che la frazione aumenta, i paesi coinvolti sono pochissimi, appiattendosi dopo che x = 0.14, ossia confermando che nessun paese supera il 14% della frazione totale degli attacchi.

Attraverso gli strumenti acquisiti durante il corso, possiamo effettivamente confermare che tutti i valori si concentrano intorno allo **0-5%**. Possiamo quindi confermare quantitativamente attraverso:

- **Skewness = 5.67** → molto positiva, confermando che la distribuzione è fortemente sbilanciata verso destra. Pochi paesi hanno valori elevati.

- **Kurtosis = 40.79** → molto elevata, indicando una distribuzione **leptocurtica**, ossia che la curva è molto appuntita vicino a zero e con code lunghe a destra.

3.1.2: Modalita' di attacchi terroristici piu' frequenti

Una seconda domanda a cui possiamo rispondere è quali sono le **modalità più comuni in cui avvengono questi attacchi?** Verifichiamo innanzitutto il numero di modalità proposte dal GTD.

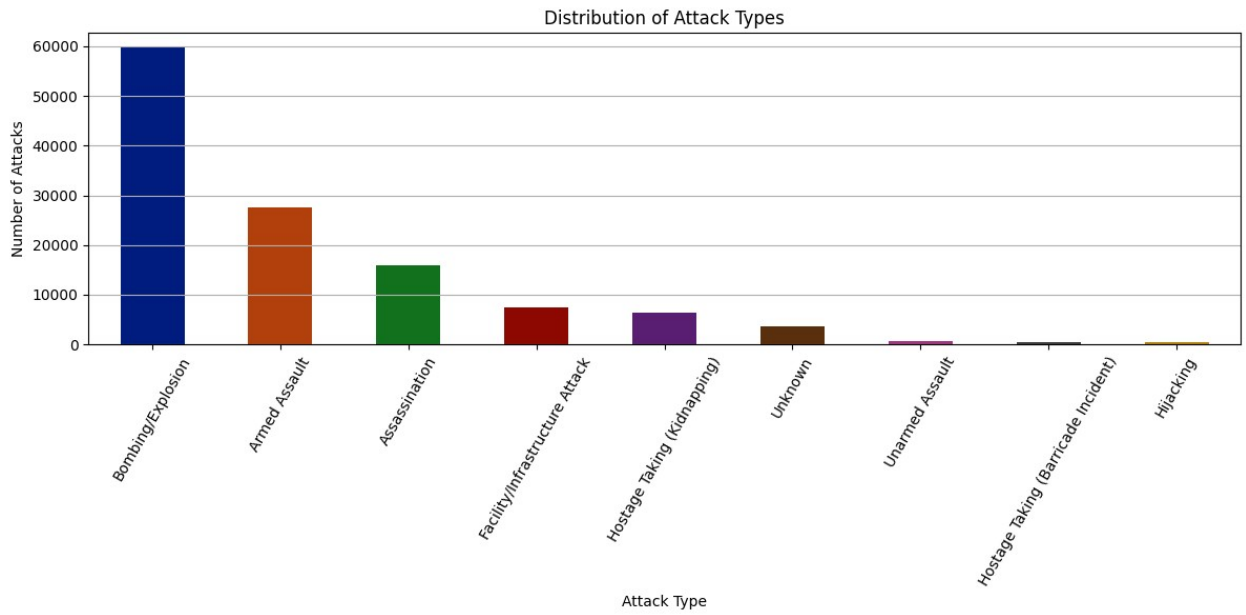
```
num_attacktypes = terrorism_cleaned['attacktype1_txt'].nunique()
print(f"I tipi di attacco presenti all'interno del dataset è
{num_attacktypes}")
```

```
I tipi di attacco presenti all'interno del dataset è 9
```

Possiamo elencare in decrescenza tali modalità per individuare quelle più adottate rappresentando successivamente il tutto con un grafico a barre.

```
# Frequenza assoluta dei tipi di attacco
attack_counts = terrorism_cleaned['attacktype1_txt'].value_counts()

freq_abs = pd.DataFrame({
    'Tipo d\'attacco': attack_counts.index,
    'Frequenza Assoluta':attack_counts.values})

print("Frequenza assoluta dei tipi di attacco in ordine descescente:")
print(freq_abs.sort_values(by='Frequenza Assoluta' ,ascending=False))
print()

# Plot
attack_counts.plot.bar(figsize=(12, 6), color=dark_palette)
plt.title("Distribution of Attack Types")
plt.ylabel("Number of Attacks")
plt.xlabel("Attack Type")
plt.xticks(rotation=60)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```

```
Frequenza assoluta dei tipi di attacco in ordine descescente:
                        Tipo d'attacco  Frequenza Assoluta
0                    Bombing/Explosion               59663
1                        Armed Assault               27481
2                        Assassination               15927
3       Facility/Infrastructure Attack                7447
4          Hostage Taking (Kidnapping)                6476
5                              Unknown                3720
6                      Unarmed Assault                 629
7  Hostage Taking (Barricade Incident)                 543
8                             Hijacking                473
```



Il barplot soprastante indica che gli attacchi terroristici osservati si affidano in modo sistematico a **ordigni esplosivi**, perché sono relativamente facili da reperire, hanno un forte impatto mediatico e possono colpire infrastrutture e civili in massa.

3.1.3: Periodi con maggiore concentrazione di attacchi

Per rispondere alla domanda: **"in quali periodi c'è stata più concentrazione di attacchi?"** utilizziamo la variabile **date** già creata nel dataset. L'approccio è il seguente:

- distribuiamo la quantità di attacchi per anno a partire dalla colonna date;
- Calcoliamo la frequenza assoluta degli attacchi in ciascun intervallo;
- Calcoliamo la frequenza relativa (%) rispetto al totale;
- Individuiamo i periodi con percentuali più rilevanti, cioè quelli che concentrano la maggior parte degli attacchi
- rappresentiamo graficamente le frequenze assolute e relative di attacchi per anno;

```
freq_abs =
terrorism_cleaned['date'].dt.year.value_counts().sort_index()


freq_rel = freq_abs / len(terrorism_cleaned)


freq_rel_perc = freq_abs / len(terrorism_cleaned)

summary = pd.DataFrame({
    'Anno': freq_abs.index,
    'Frequenza assoluta': freq_abs.values,
    'Frequenza relativa': freq_rel_perc.values.round(2)
})
```

```
summary_sorted = summary.sort_values(by='Frequenza relativa',
ascending=False)

print("Periodi annuali con maggiore concentrazione di attacchi:")
print(summary_sorted.head(10))  # Mostriamo i 10 anni più rilevanti
```

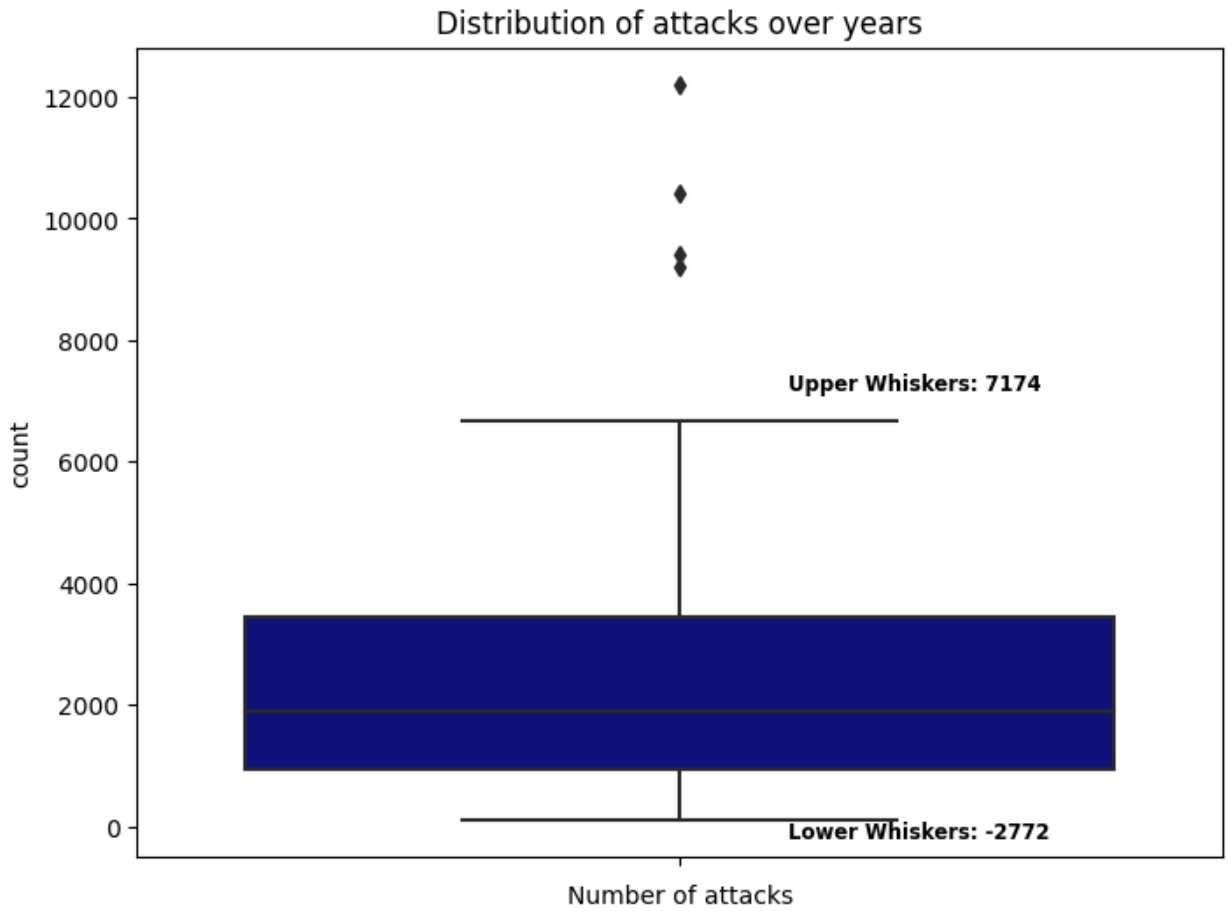
```
Periodi annuali con maggiore concentrazione di attacchi:
    Anno  Frequenza assoluta  Frequenza relativa
43  2014               12209                0.10
44  2015               10411                0.09
45  2016                9420                0.08
42  2013                9217                0.08
41  2012                6679                0.05
22  1992                4154                0.03
20  1990                3255                0.03
21  1991                3809                0.03
37  2008                3645                0.03
38  2009                3508                0.03
```

Tra il 2011 e il 2016 si osserva una forte **concentrazione di attacchi terroristici**, con il 2014 come anno di massimo impatto (circa il 10% degli eventi totali). Scorgiamo anche un periodo **tra il 1990 e il 1992** con una concentrazione di attacchi non di poco conto.

Intanto, possiamo continuare ad effettuare l'analisi con i dati che abbiamo a disposizione. Attraverso l'utilizzo del **boxplot** possiamo vedere le informazioni statische più rilevanti.

```
attforyear = terrorism_cleaned['year'].value_counts()

Q1 = attforyear.quantile(0.25)
Q3 = attforyear.quantile(0.75)

IQR = Q3-Q1

lowwhis = Q1-1.5*IQR
upwhis = Q3+1.5*IQR

plt.figure(figsize=(8,6))

sns.boxplot(y=attforyear, color='darkblue', whis=1.5)

plt.title('Distribution of attacks over years')
plt.text(0.1, upwhis+100, f'Upper Whiskers: {int(upwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')
plt.text(0.1, -100, f'Lower Whiskers: {int(lowwhis)}',
            horizontalalignment='left', size='small',
color='black',weight = 'bold', va='center')
```

```
plt.xlabel('Number of attacks')
plt.show()

print(attforyear.describe())
```

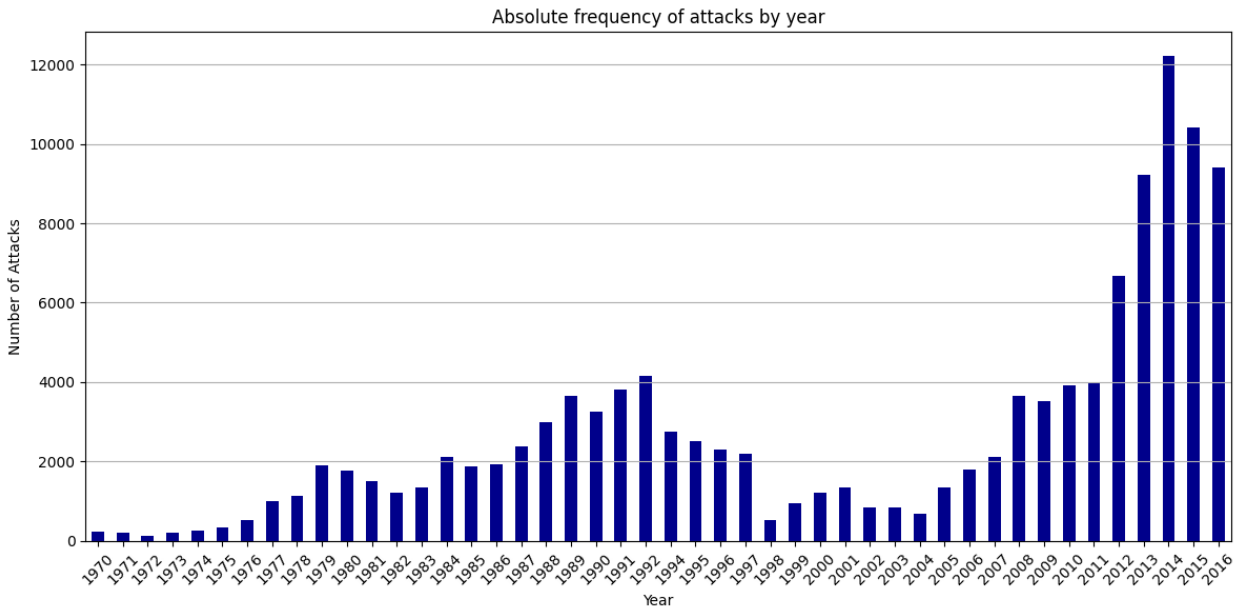



```
count       46.000000
mean      2659.978261
std       2766.598405
min        117.000000
25%        958.000000
50%       1900.000000
75%       3444.750000
max      12209.000000
Name: count, dtype: float64
```

La **mediana** pari a **1900** può rappresentare un valore centrale per la distribuzione: **metà** degli anni ha registrato **meno di 1900 attacchi**, l'altra **metà di più**. Questo la rende un **indicatore robusto** per distinguere gli anni **"ordinari"** da quelli **"critici"**

È possibile costruire un **grafico temporale** che metta in evidenza l'evoluzione annuale degli attacchi, facilitando l'identificazione delle fasi **più critiche**.

```
plt.figure(figsize=(12,6))
freq_abs.plot.bar(color='darkblue')
plt.title("Absolute frequency of attacks by year")
plt.ylabel("Number of Attacks")
plt.xlabel("Year")
plt.xticks(rotation=45)
plt.grid(axis='y')
plt.tight_layout()
plt.show()
```



Il grafico mostra chiaramente che i picchi di attacchi terroristici si concentrano nei periodi che **superano la soglia della mediana** di **1900** attacchi annui. In particolare:

- **Dal 1984 al 1997**, con costante valore sopra la mediana;
- **Dal 2011 al 2016**, culminando nel 2014 oltre 12.000 eventi, il valore massimo assoluto.

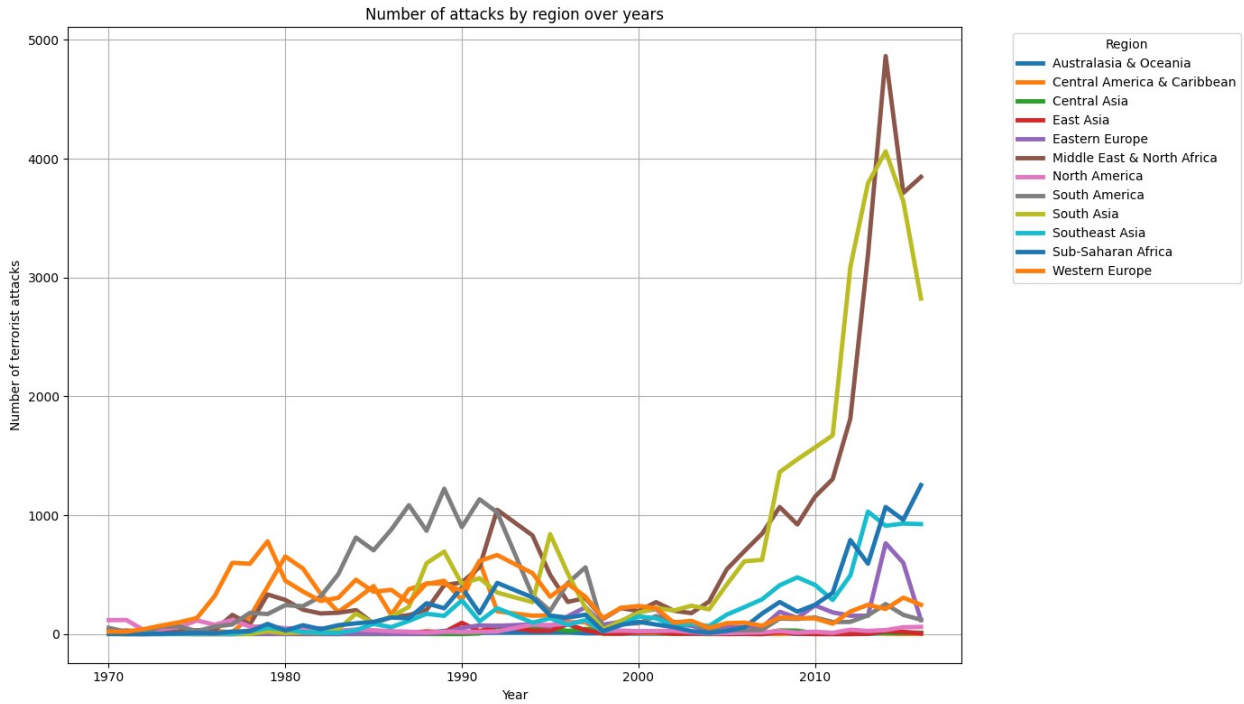
Osservando il seguente grafico, possiamo osservare come sia presente un'**inconsistenza**. Gli attacchi con **year = 1993** sono assenti sul nostro dataset. A seguito di una ricerca approfondita nel **CODEBOOK**, abbiamo scoperto che i dati relativi al 1993 sono stati **perduti durante una fase di compilazione**. Possiamo sfruttare questa informazione per eventuali approfondimenti futuri.

In conclusione abbiamo evidenziato due periodi di particolare **instabilità geopolitica**, in cui l'attività di gruppi armati ha raggiunto livelli altissimi, il secondo dei quali senza precedenti (fino al 2016).

Analizziamo come gli attacchi terroristici si sono distribuiti nel tempo nei diversi continenti.

```
region_counts = terrorism_cleaned.groupby(['year', 'region_txt'])
['eventid'].count().unstack(fill_value=0)

region_counts.plot(figsize=(14,8),linewidth=3.5)
```

```
plt.title("Number of attacks by region over years")
plt.xlabel("Year")
plt.ylabel("Number of terrorist attacks")
plt.grid()
plt.legend(title="Region", bbox_to_anchor=(1.05,1), loc='upper left')
plt.tight_layout()
plt.show()
```



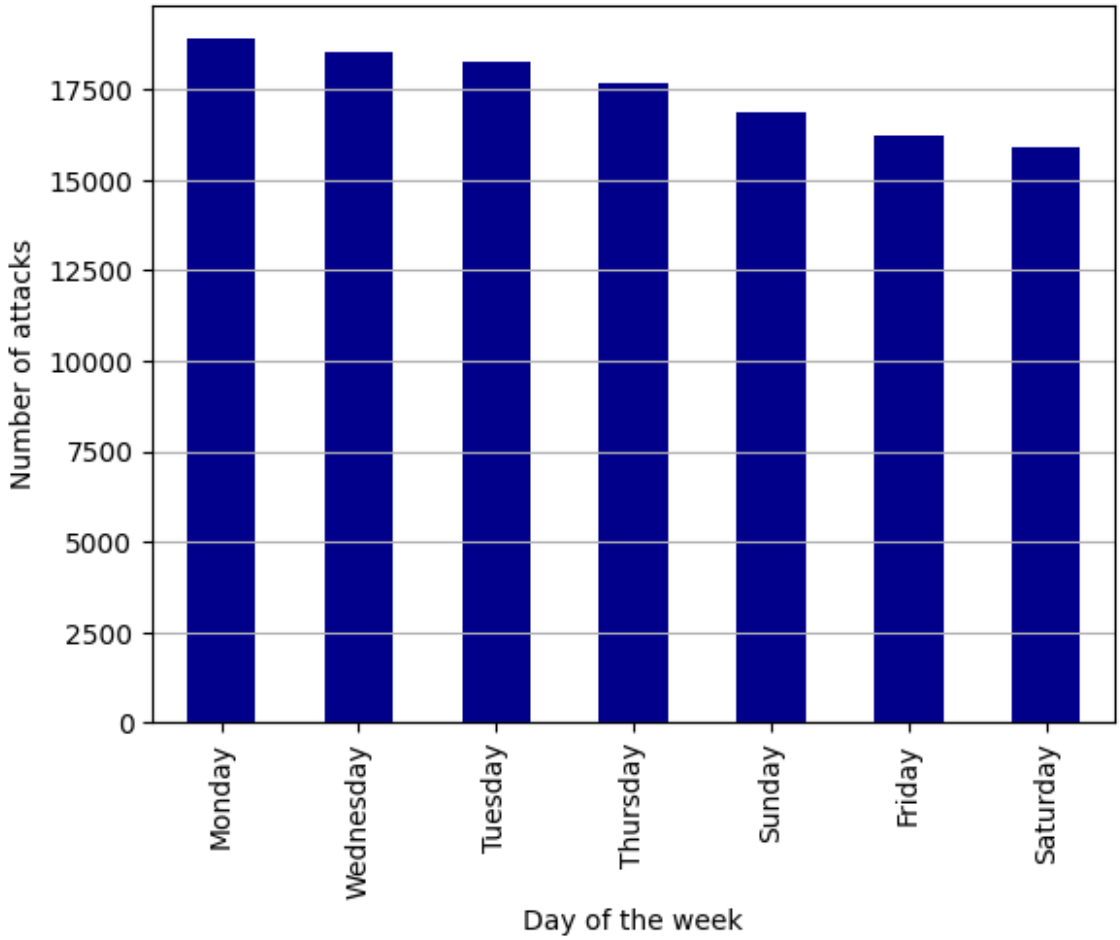
Dal grafico si possono attingere molte informazioni; infatti, possiamo dedurre che:

- nel ventennio che va **dal 1980 al 2000**, la regione più soggetta ad attacchi era ***'South America'***;
- negli **ultimi anni** c'è un trend che trova un **picco nel 2014-2015**, ovvero le due regioni di interesse erano ***'Middle East & North Africa'*** e ***'South Asia'***;
- un altro trend in salita riguarda gli attacchi nella regione ***'Sub-Saharan Desert'***.

Un'ulteriore prospettiva sempre legata alla temporalità degli attacchi, potrebbe riguardare la valutazione su come il giorno della settimana possa influire la frequenza.

```
attinweek = terrorism_cleaned['weekday'].value_counts()
attinweek.plot(kind='bar', color='darkblue')
plt.xlabel("Day of the week")
plt.ylabel("Number of attacks")
plt.grid(axis='y')
plt.show()
```

```
print(attinweek)
```




```
weekday
Monday       18905
Wednesday    18532
Tuesday      18258
Thursday     17706
Sunday       16854
Friday       16217
Saturday     15887
Name: count, dtype: int64
```

Nel complesso, i **dati sono distribuiti quasi equamente tra loro**, si nota solo che alcune giornate, in particolare l'**inizio della settimana**, registrano un **numero maggiore di attacchi**.

3.1.4: Principali obiettivi di un attacco terroristico

Una terza domanda a cui è importante dare uno sguardo è: **Quali sono i principali obiettivi di un attacco terroristico?**

Basandoci sulla variabile **targtype1_txt** verifichiamo quanti valori assume in modo da analizzare con chiarezza.

```
num_targtype1 = terrorism_cleaned['targtype1_txt'].nunique()
print(f"Il numero di tipi di attacco presenti all'interno del dataset
è {num_targtype1}")
```

```
Il numero di tipi di attacco presenti all'interno del dataset è 22
```

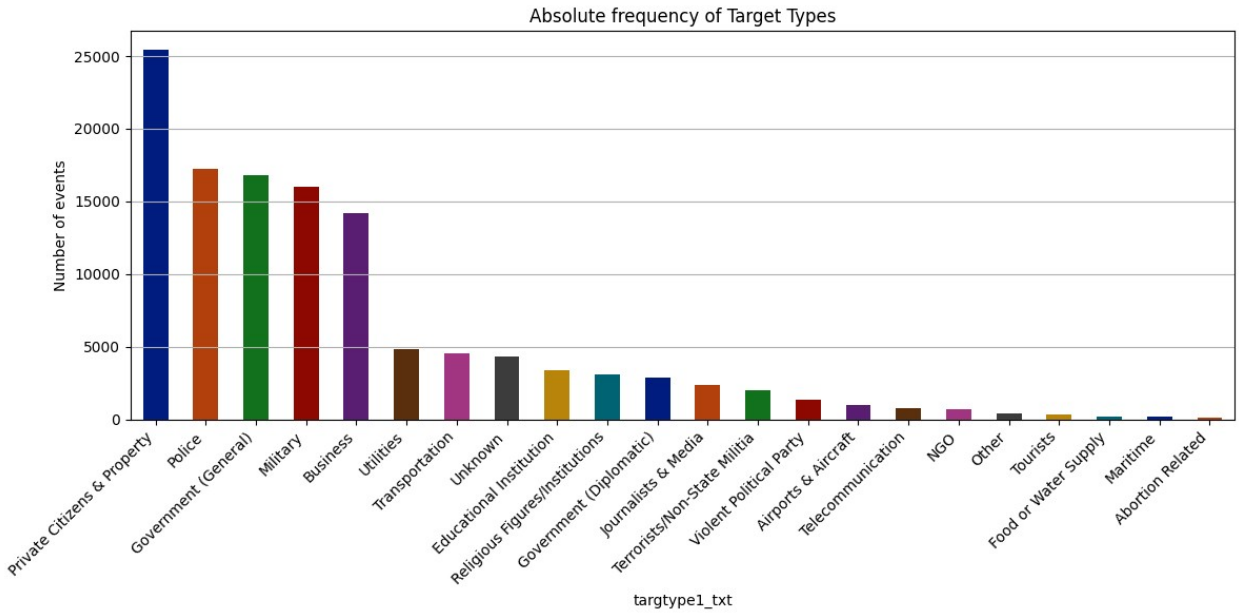
Possiamo procedere in questo modo:

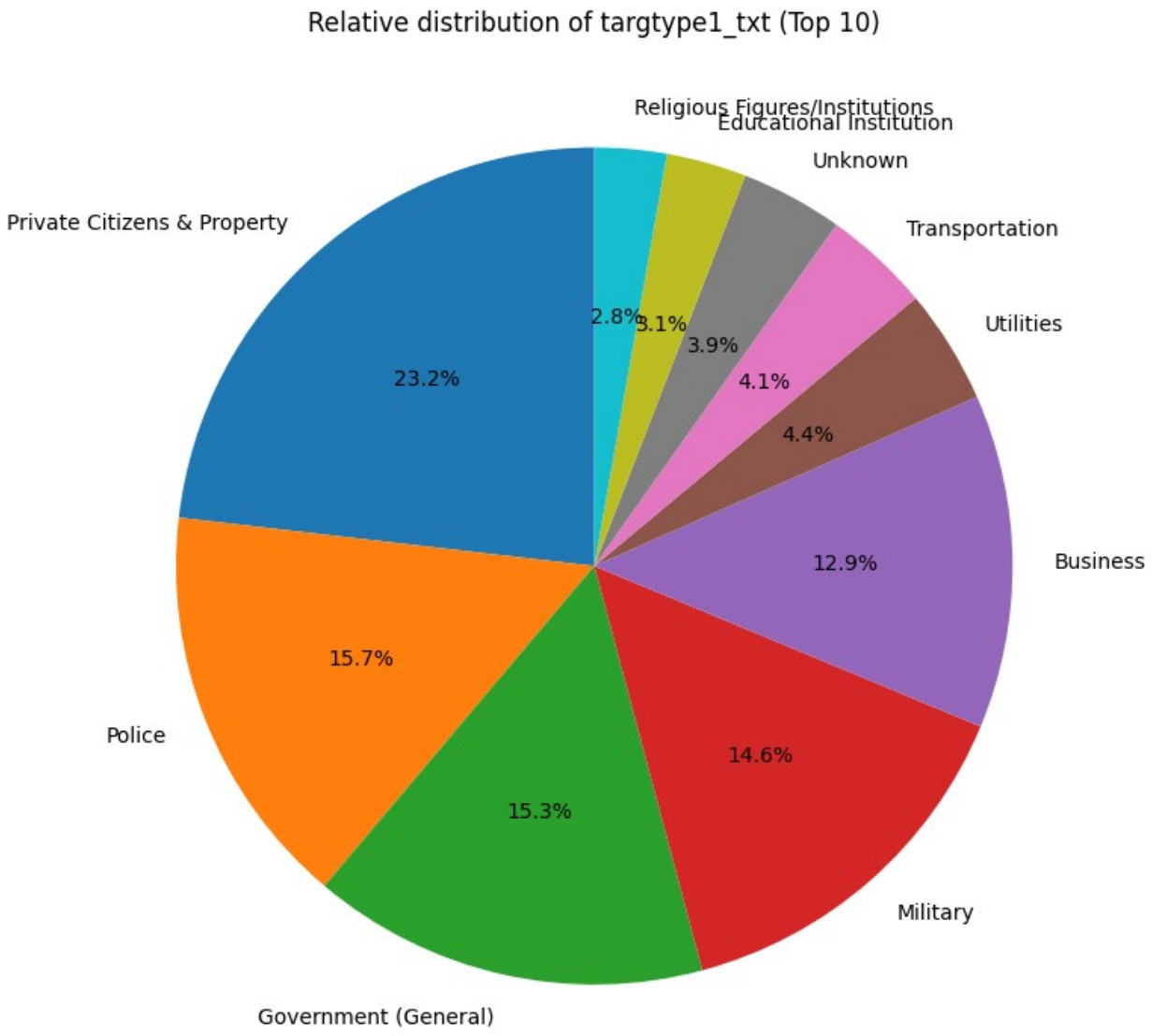
- calcolando le frequenze assolute e relative, elencandole in una tabella;
- rappresentando la frequenza assoluta di tutti i valori su un grafico a barre;
- rappresentando le 10 frequenze relative più rilevanti su un grafico a torta.

```
freq_abs = terrorism_cleaned['targtype1_txt'].value_counts()


freq_rel = freq_abs / freq_abs.sum()

freq_abs_rel = pd.DataFrame({
    'Obiettivo': freq_abs.index,
    'Frequenza Assoluta':freq_abs.values,
    'Frequenza Relativa': freq_rel.values})

print("Frequenza assoluta e relativa dei paesi più attaccati:")
print(freq_abs_rel.sort_values(by='Frequenza
Assoluta' ,ascending=False))
print()


plt.figure(figsize=(12,6))
freq_abs.plot(kind='bar', color=dark_palette)
plt.title('Absolute frequency of Target Types')
plt.ylabel('Number of events')
plt.xlabel('targtype1_txt')
plt.xticks(rotation=45, ha='right')
plt.grid(axis='y')
plt.tight_layout()
plt.show()


plt.figure(figsize=(8,8))
freq_rel.head(10).plot(kind='pie', autopct='%1.1f%%', startangle=90)
plt.title('Relative distribution of targtype1_txt (Top 10)')
plt.ylabel('')
plt.tight_layout()
plt.show()
```

```
Frequenza assoluta e relativa dei paesi più attaccati:
                         Obiettivo  Frequenza Assoluta  Frequenza
```

```
Relativa
0      Private Citizens & Property                25449
0.207986
1                            Police                17245
0.140938
2             Government (General)                 16831
0.137554
3                          Military                16022
0.130943
4                          Business                14164
0.115758
5                         Utilities                 4865
0.039760
6                    Transportation                 4533
0.037047
7                           Unknown                 4338
0.035453
8          Educational Institution                 3402
0.027803
9   Religious Figures/Institutions                 3058
0.024992
10         Government (Diplomatic)                 2868
0.023439
11             Journalists & Media                 2391
0.019541
12    Terrorists/Non-State Militia                 2040
0.016672
13         Violent Political Party                 1360
0.011115
14             Airports & Aircraft                 1020
0.008336
15               Telecommunication                  787
0.006432
16                             NGO                  732
0.005982
17                           Other                  377
0.003081
18                         Tourists                 306
0.002501
19            Food or Water Supply                  227
0.001855
20                         Maritime                 198
0.001618
21                 Abortion Related                 146
0.001193
```


Absolute frequency of Target Types
Number of events
25000
20000
15000
10000
5000
0
Private Citizens & Property
Police
Government (General)
Military
Business
Utilities
Transportation
Unknown
Educational Institution
Religious Figures/Institutions
Government (Diplomatic)
Journalists & Media
Terrorists/Non-State Militia
Violent Political Party
Airports & Aircraft
Telecommunication
NGO
Other
Tourists
Food or Water Supply
Maritime
Abortion Related
targtype1_txt



Il **grafico a barre** mostra chiaramente che i bersagli più colpiti in termini assoluti sono **i civili e le proprietà private**, seguiti da **forze di polizia**, **governi** e **militari**. Questi quattro gruppi da soli rappresentano la maggior parte degli attacchi registrati (con un **picco di 25000 bersagli civili**), evidenziando una strategia terroristica mirata a **destabilizzare l'ordine pubblico e le istituzioni statali**.

Il **grafico a torta**, basato sulle frequenze relative, conferma questa tendenza:

- I **civili** costituiscono oltre il **20%** degli attacchi totali;
- Le forze dell'ordine e il governo insieme superano il **27%**, indicando una pressione diretta sulle strutture di controllo.

3.1.5 gruppi terroristici che hanno commesso piu' attacchi

Riguardo ai gruppi terroristici, in merito alla domanda **Quanto?**, possiamo utilizzare la variabile categoriale **gname** per trovare indici statistici e rappresentazioni a noi utili.

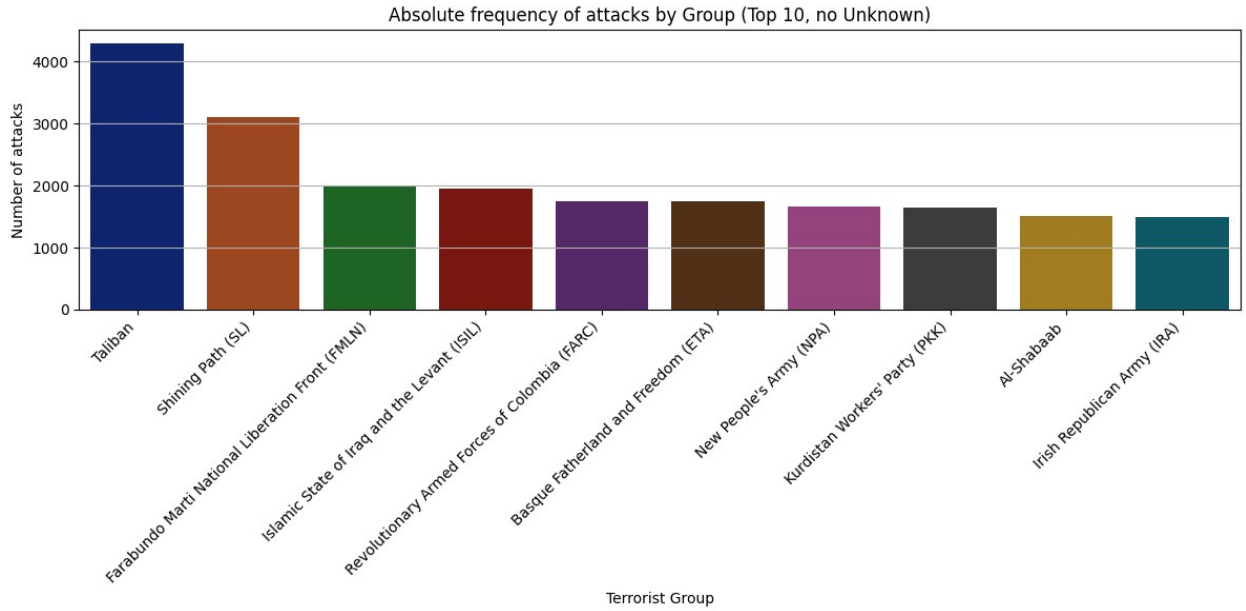
```
group_freq = terrorism_cleaned['gname'].value_counts().reset_index()
group_freq.columns = ['gname', 'freq_abs']


totale_attacchi = group_freq['freq_abs'].sum()
group_freq['freq_rel'] = (group_freq['freq_abs'] /
totale_attacchi).round(2)


print("Gruppi terroristici con più attacchi:")
print(group_freq.head(10))
```

```
Gruppi terroristici con più attacchi:
                                                gname  freq_abs
freq_rel
0                                            Unknown     61536
0.50
1                                            Taliban      4294
0.04
2                                  Shining Path (SL)      3099
0.03
3  Farabundo Marti National Liberation Front (FMLN)      1980
0.02
4       Islamic State of Iraq and the Levant (ISIL)      1959
0.02
5     Revolutionary Armed Forces of Colombia (FARC)      1744
0.01
6                Basque Fatherland and Freedom (ETA)      1739
0.01
7                           New People's Army (NPA)      1667
0.01
8                    Kurdistan Workers' Party (PKK)      1652
0.01
9                                         Al-Shabaab      1514
0.01
```

La tabella mostra i 10 gruppi terroristici con il maggior numero di attacchi registrati nel dataset, o meglio, 9 con la dominanza di oltre 60 mila **unknown**, circa il 50% del totale degli attacchi. Mentre i gruppi **Taliban** e **Shining** Path sono **nettamente più attivi rispetto agli altri**, sono comunque molto **distanti** dal volume di attacchi sconosciuti Dal punto di vista analitico e visivo, è fondamentale **escludere il gruppo "Unknown"** prima di procedere alla rappresentazione su grafico a barre per evitare schiacciamenti.

```
terrorism_filtered = terrorism_cleaned[terrorism_cleaned['gname'] !=
'Unknown']
```

```
group_freq_no_unknown =
terrorism_filtered['gname'].value_counts().reset_index()
group_freq_no_unknown.columns = ['gname', 'frequenza_assoluta']


df_top = group_freq_no_unknown.head(10).copy()


plt.figure(figsize=(12,6))
#df_top.plot.bar(x='gname',y='frequenza_assoluta',color=dark_palette,
legend=False) #grafico schiacciato
ax = sns.barplot(
    x='gname',
    y='frequenza_assoluta',
    data=df_top,
    palette='dark'
)
plt.xticks(rotation=45, ha='right')
plt.title("Absolute frequency of attacks by Group (Top 10, no
Unknown)")
plt.ylabel("Number of attacks")
plt.xlabel("Terrorist Group")
plt.grid(axis='y')

plt.tight_layout()
plt.show()
```



Dopo aver escluso i valori unknown, notiamo dal grafico:

- **Taliban** è nettamente il gruppo più attivo, con **oltre 3.500 attacchi**;

- Seguono **SL** e **FMLN** con circa **2.000–3.000 attacchi** ciascuno;
- Gli altri gruppi si attestano tra **1.000 e 1.700 attacchi**, mostrando sul 50% conosciuto una presenza **significativa** se considerati insieme, ma **meno intensa dell'1-2%** se presi singolarmente.

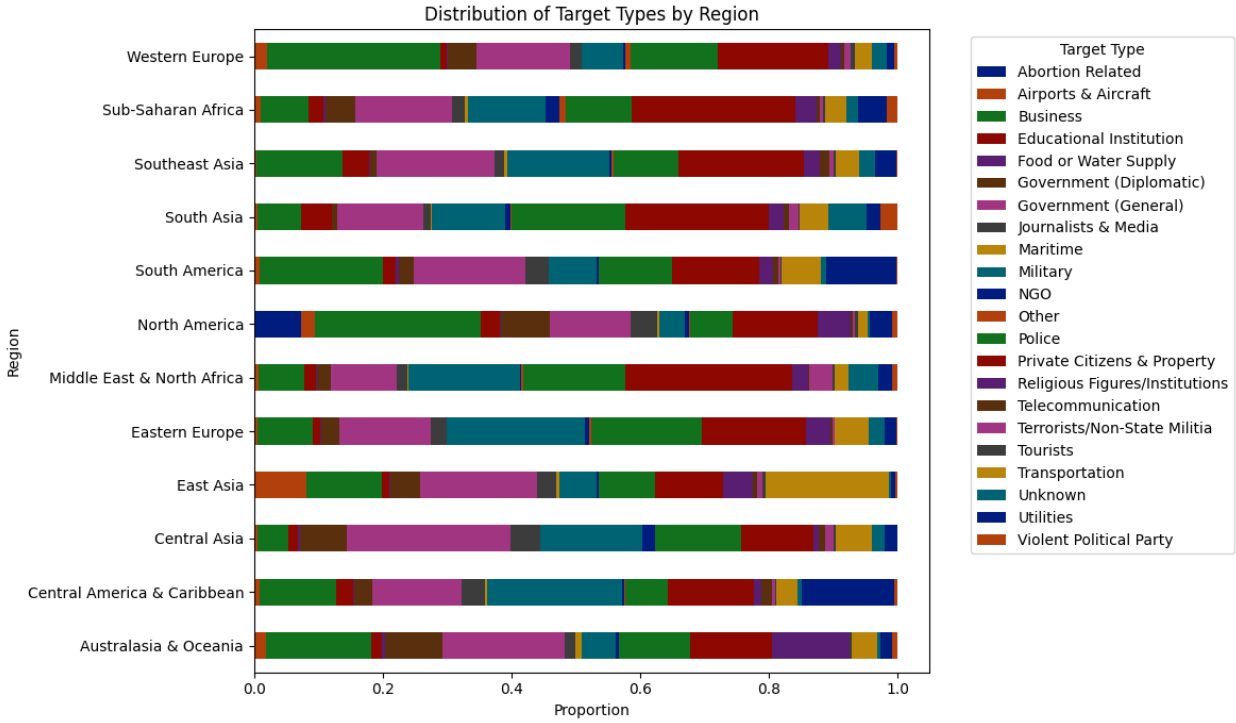
Una direzione estremamente interessante per approfondire l'analisi multivariata sarebbe quella di incrociare la frequenza degli attacchi con il **numero di vittime** provocate (nkill) **per ciascun gruppo**.

3.2: Analisi Multivariata

3.2.1: Regione e Bersaglio (Categorica vs. Categorica)

Rispondendo alla domanda del **Che Cosa?** riguardo una correlazione tra i luoghi attaccati e il bersaglio di ciascun attacco, si può visualizzare su un grafico a barre impilate il risultato dell'**Analisi Multivariata**, in aggiunta alla relativa **Tabella di Contigenza** nonché i valori del risultato del **Test Chi-Quadrato**, **V di Cramer** e **p-value**:

```
con_tab = pd.crosstab(terrorism_cleaned['region_txt'],
terrorism_cleaned['targtype1_txt'])

chi2, pvalue, _, expected = chi2_contingency(con_tab)
v = vcramer(con_tab)

contingency_prop = con_tab.div(con_tab.sum(axis=1), axis=0)


contingency_prop.plot(kind='barh', stacked=True, figsize=(12, 7),
color=dark_palette)
plt.title("Distribution of Target Types by Region")
plt.xlabel("Proportion")
plt.ylabel("Region")
plt.legend(title="Target Type", bbox_to_anchor=(1.05, 1), loc='upper
left')
plt.tight_layout()
plt.show()

print(f"Chi-Quadrato: {chi2:.2f}")
print(f"p-value: {pvalue:.4f}")
print(f"V di Cramér: {v:.4f}")
```

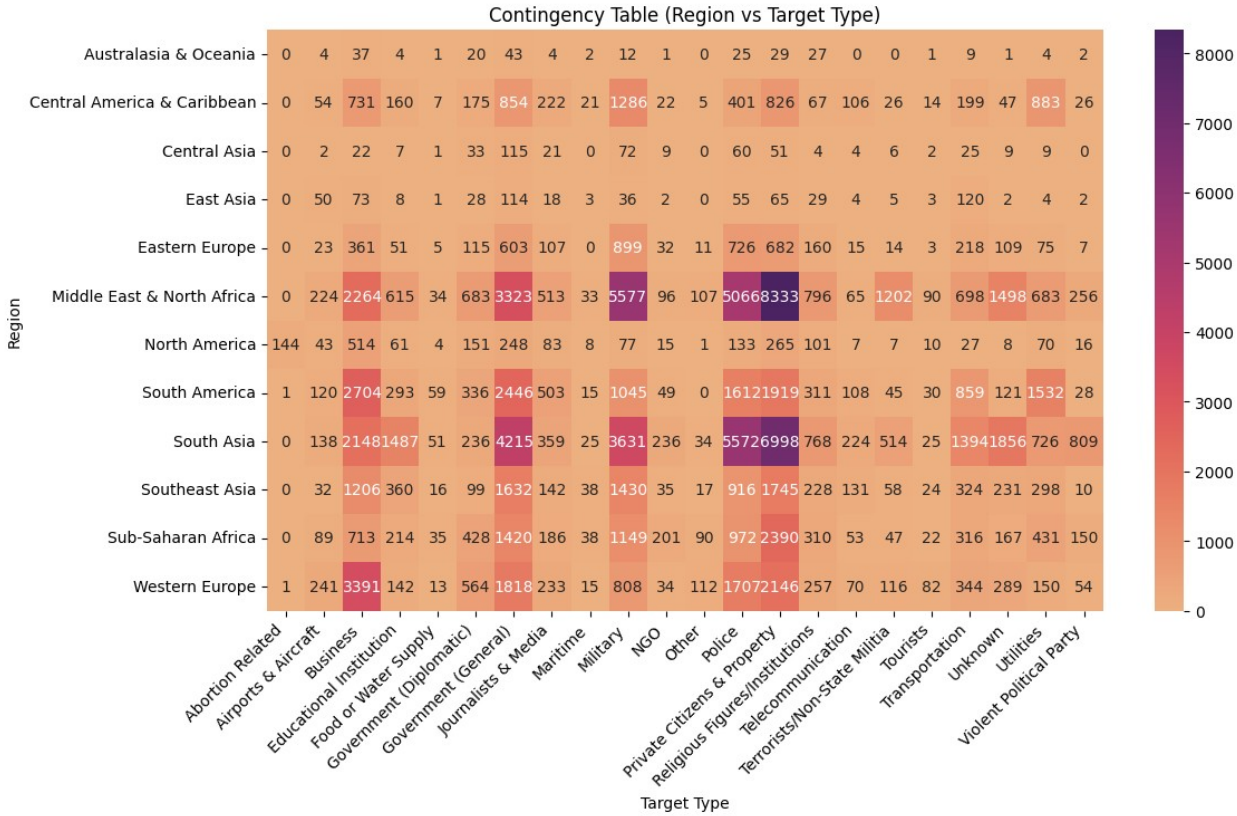



```
Chi-Quadrato: 32141.83
p-value: 0.0000
V di Cramér: 0.1545
```

Possiamo osservare che il **Chi-quadrato** indica che c'è una relazione significativa e in contemporanea il **p-value** smentisce l'ipotesi nulla di indipendenza, come se avessimo una **relazione forte**. Tuttavia la **V di Cramér** ci dice che la relazione è **debole**.

Vista l'incoerenza tra i dati statistici **verifichiamo l'affidabilità del Chi-quadrato** controllando che i valori sulla tabella di contingenza non siano inferiori a 5.

```python
# Heatmap della tabella di contingenza
plt.figure(figsize=(12, 8))
sns.heatmap(con_tab, cmap="flare", annot=True, fmt="d", cbar=True)

plt.title("Contingency Table (Region vs Target Type)")
plt.xlabel("Target Type")
plt.ylabel("Region")
plt.tight_layout()
plt.xticks(rotation=45, ha='right')
plt.show()

# Conta valori <= 5
num_problematic = (con_tab.values <= 5).sum()
total_values = con_tab.size

print(f"Valori <= 5: {num_problematic} su {total_values} totali")
```




```
Valori <= 5: 49 su 264 totali
```

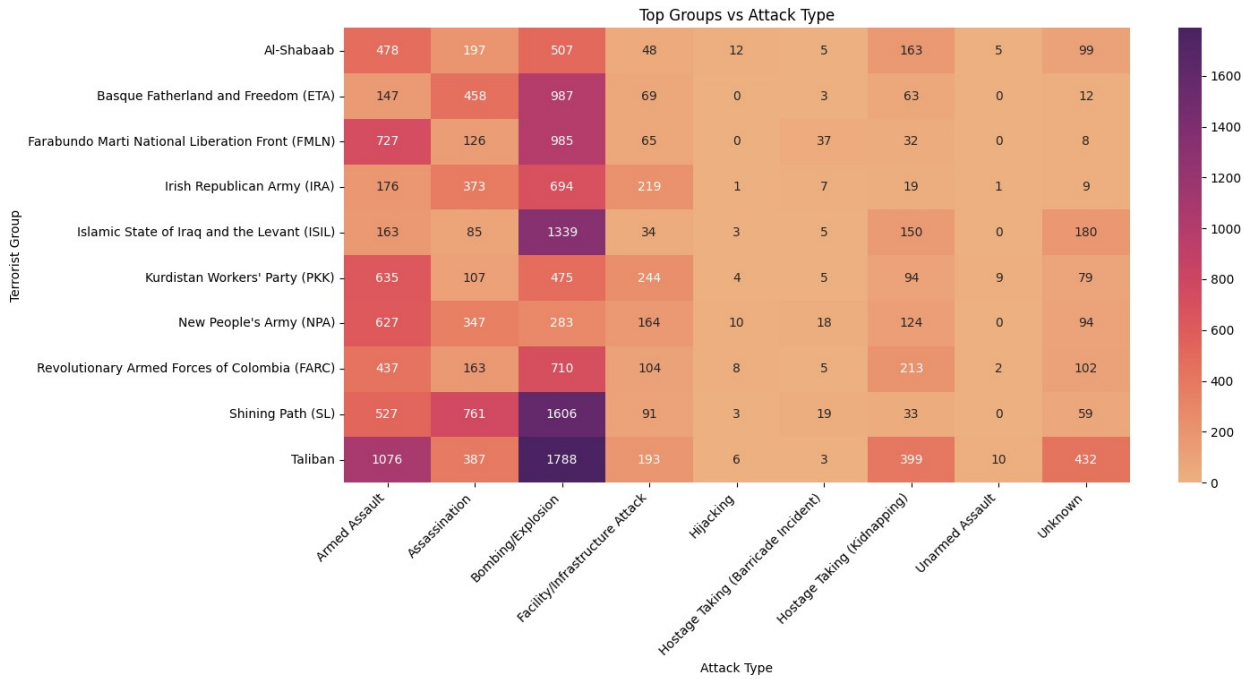
Dalla presenza di valori inferiori a 5 nella tabella di contingenza, che conferma l'inaffidabilità del Chi-quadrato, e la V di Cramér bassa, possiamo affermare che la regione **influisce** sul tipo di bersaglio, ma **non** lo determina in **modo netto**.

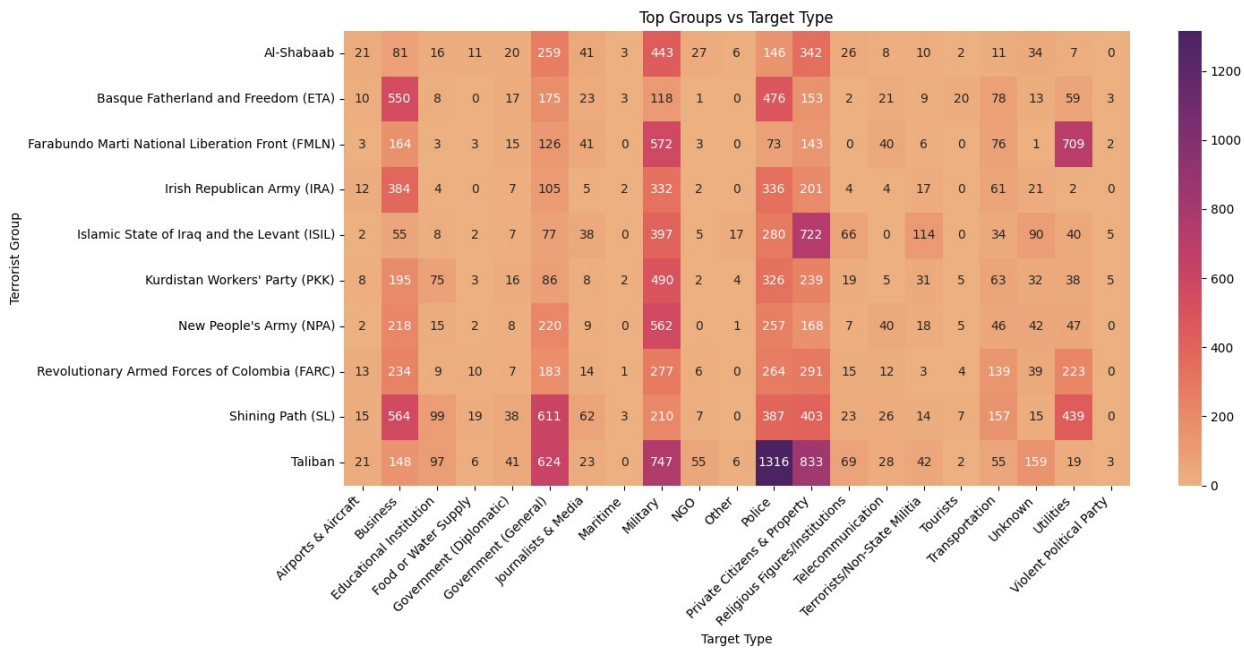
3.2.2: Modalita' d'attacco e gruppi terroristici (Categorica vs. Categorica)

Le ***modalità d'attacco*** e i ***gruppi terroristici*** tendono a essere collegati tra loro. Per capire quali tipi di attacco i gruppi prediligono e chi prendono di mira, abbiamo tolto i gruppi segnati come **'Unknown'**, che sono la maggior parte e non ci aiutano a capire le responsabilità. L'analisi si concentra quindi sui 10 gruppi più attivi, così i risultati sono più chiari e facili da interpretare.

```
filtered_groups = terrorism_cleaned[terrorism_cleaned['gname'] !=
'Unknown']['gname']

topgroups =
filtered_groups.value_counts(normalize=True).sort_values(ascending=False).head(10)

filtered =
terrorism_cleaned[terrorism_cleaned['gname'].isin(topgroups.index)]
```

```
favattgrp = pd.crosstab(filtered['gname'],
filtered['attacktype1_txt'])
favtargrp = pd.crosstab(filtered['gname'], filtered['targtype1_txt'])


fig, axes = plt.subplots(2,1 , figsize=(15,15))

sns.heatmap(favattgrp, annot=True, fmt='d', cmap='flare', ax=axes[0])
axes[0].set_title('Top Groups vs Attack Type')
axes[0].set_xlabel('Attack Type')
axes[0].set_xticklabels(axes[0].get_xticklabels(), rotation=45,
ha='right')
axes[0].set_ylabel('Terrorist Group')


sns.heatmap(favtargrp, annot=True, fmt='d', cmap='flare', ax=axes[1])
axes[1].set_title('Top Groups vs Target Type')
axes[1].set_xlabel('Target Type')
axes[1].set_xticklabels(axes[1].get_xticklabels(), rotation=45,
ha='right')
axes[1].set_ylabel('Terrorist Group')
plt.tight_layout()

plt.show()

chi2att,pvalueatt,_,_ = chi2_contingency(favattgrp)
chi2tar,pvaluetar,_,_ = chi2_contingency(favtargrp)
vatt = vcramer(favattgrp)
vtar = vcramer(favtargrp)

print("Test statistici sull'eventuale correlazione tra gruppo
terroristico e tipo di attacco")
print(f"Chi-Quadrato: {chi2att:.2f}")
print(f"p-value: {pvalueatt:.4f}")
print(f"V di Cramér: {vatt:.4f}")
print()
print("Test statistici sull'eventuale correlazione tra gruppo
terroristico e tipo di bersaglio")
print(f"Chi-Quadrato: {chi2tar:.2f}")
print(f"p-value: {pvaluetar:.4f}")
print(f"V di Cramér: {vtar:.4f}")
```






```
Test statistici sull'eventuale correlazione tra gruppo terroristico e
tipo di attacco
Chi-Quadrato: 4686.65
p-value: 0.0000
V di Cramér: 0.1664

Test statistici sull'eventuale correlazione tra gruppo terroristico e
tipo di bersaglio
Chi-Quadrato: 9341.54
```

```
p-value: 0.0000
V di Cramér: 0.2215
```

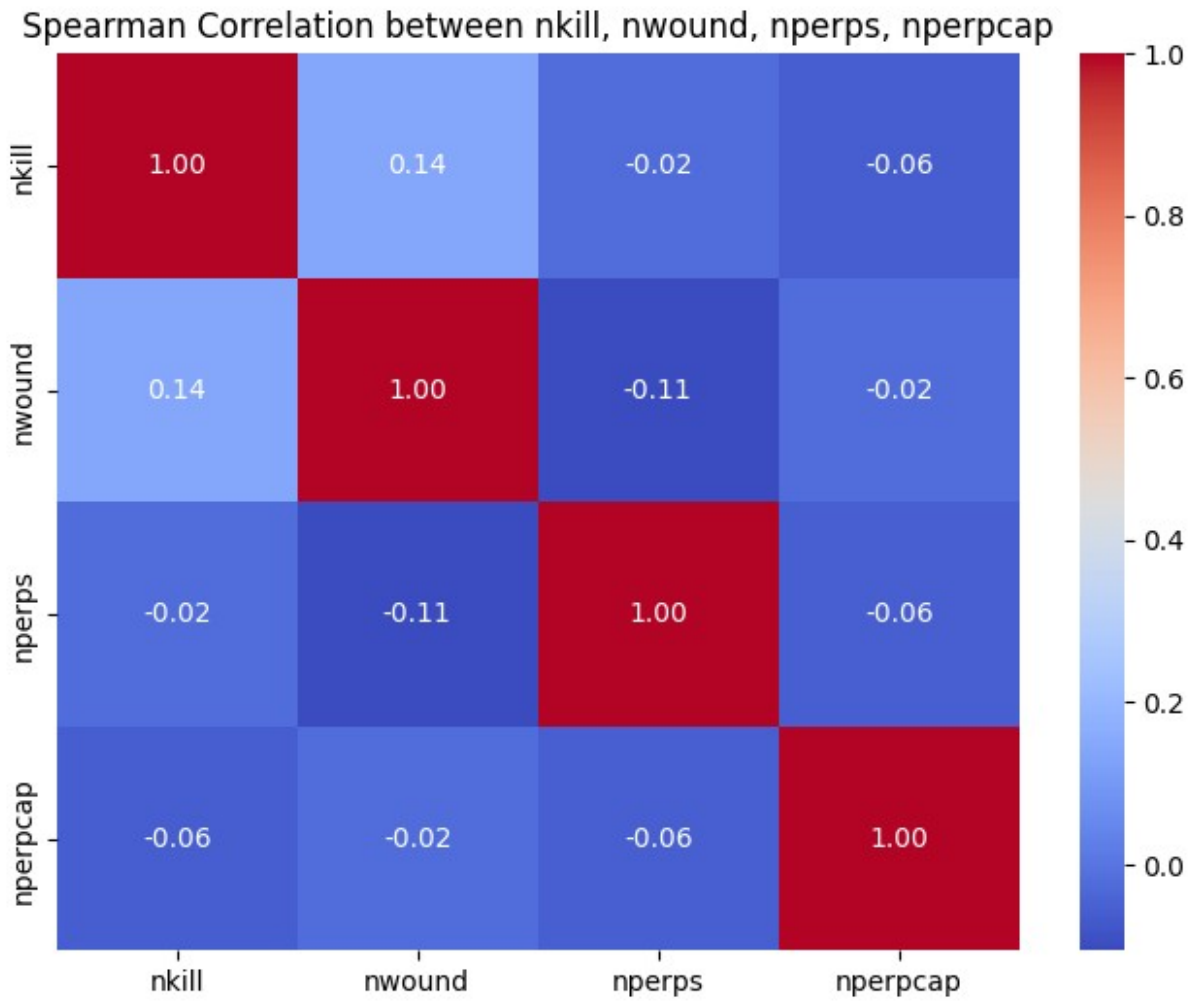
Il test **chi-quadrato** evidenzia che entrambe le associazioni sono statisticamente significative (**p-value** ≈ 0), quindi le differenze osservate non sono casuali. Tuttavia, la forza dell'associazione, misurata tramite la V di Cramér, mostra differenze tra le due relazioni:

- ***gname-attacktype1_txt*** -> v = 0.1664 assoicazione debole, sebbene alcuni dei gruppi possono avere preferenze sul tipo di attacco, la distinzione tra i gruppi rispetto i metodi usati non è marcata;
- ***gname-targettype1_txt*** -> v = 0.2215 associazione debole/moderata, i gruppi terroristici mostrano un pattern più distintivo sulla scelta dei bersagli.

3.2.3: Uccisi, feriti, autori coinvolti e autori catturati (Numerica vs. Numerica)

```
num_cols = ['nkill', 'nwound', 'nperps', 'nperpcap']
num_data = terrorism_cleaned[num_cols]


spearman_corr = num_data.corr(method='spearman')
print(spearman_corr)

plt.figure(figsize=(8,6))
sns.heatmap(spearman_corr, annot=True, fmt=".2f", cmap='coolwarm')
plt.title("Spearman Correlation between nkill, nwound, nperps,
nperpcap")
plt.show()
```

```
              nkill    nwound    nperps  nperpcap
nkill      1.000000  0.141278 -0.021913 -0.060045
nwound     0.141278  1.000000 -0.106031 -0.021621
nperps    -0.021913 -0.106031  1.000000 -0.057684
nperpcap  -0.060045 -0.021621 -0.057684  1.000000
```



Le variabili analizzate, cioè ***nkill***, ***nwound***, ***nperps*** e ***nperpcap***, risultano quasi indipendenti tra loro. Le correlazioni monotone, misurate tramite Spearman, sono tutte molto basse, indicando l'assenza di relazioni significative tra queste feature.

Una debolissima associazione esiste tra nkill e nwound.

Il motivo di queste relazioni debolissime, anche se inspiegabili, assumono un senso se si pensa che il numero di 0 contenuti all'interno di ***nkill*** e ***nwound*** è molto alto. Il valore 0 però non è presente in ***nperps*** dato che un attacco terroristico deve avere come minimo un autore.

Il motivo delle correlazioni così deboli, sebbene possa sembrare inspiegabile, trova una spiegazione nel fatto che le variabili ***nkill*** e ***nwound*** contengono un elevato numero di zeri, ovvero molti attacchi senza vittime o feriti. Al contrario, ***nperps*** non contiene valori zero, perché ogni attacco terroristico deve avere almeno un autore.

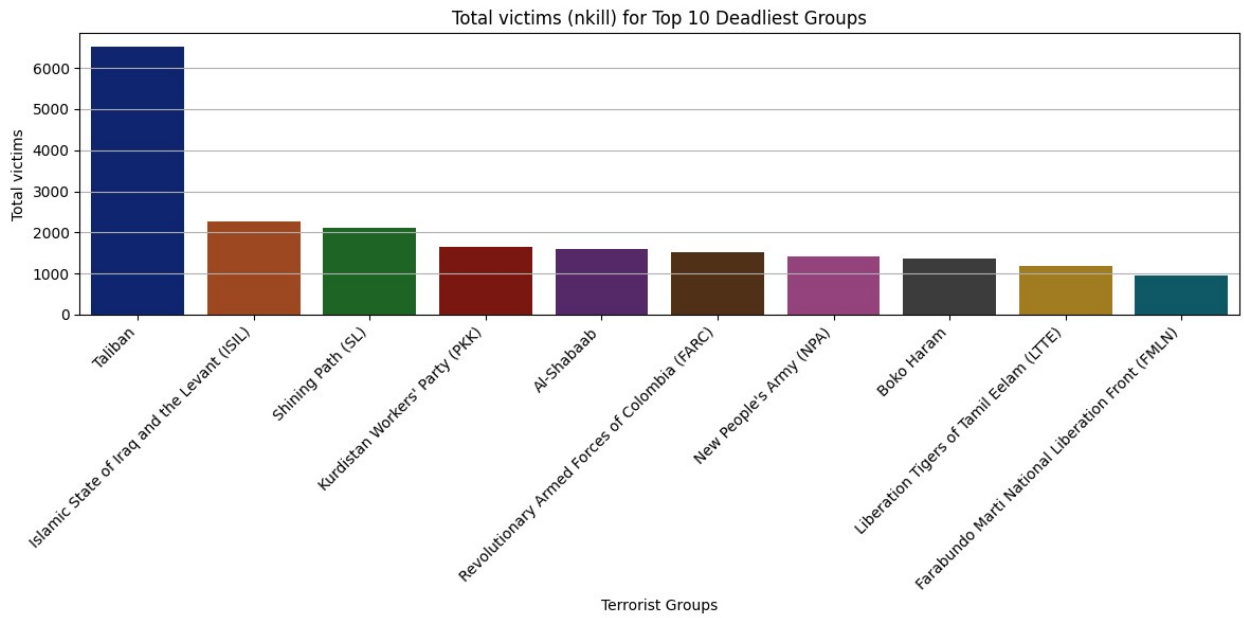
3.2.4: Vittime per gruppo terroristico (Numerica vs. Categorica)

```
# Rimuovere i gruppi 'Unknown' (per migliore leggibilità)
terrorism_filtered2 = terrorism_cleaned[terrorism_cleaned['gname'] !=
'Unknown']
```

```python
# Calcolo totale vittime (somma di nkill) per ciascun gruppo
victims_per_group = (
    terrorism_filtered2
    .groupby('gname')['nkill']
    .sum()
    .reset_index()
    .rename(columns={'nkill':'freq_abs'})
)

# Frequenza relativa
victims_per_group['freq_rel'] = (
    (victims_per_group['freq_abs'] /
victims_per_group['freq_abs'].sum()).round(2)
)

# Ordiniamo per gruppi più letali
victims_sorted = victims_per_group.sort_values(by='freq_abs',
ascending=False)

print("Frequenza assoluta e relativa delle vittime per gruppo
terroristico (prime 10 righe):")
print(victims_sorted.head(10))

# Visualizzazione con barplot (totale vittime per gruppo)
df_top = victims_sorted.head(10).copy()

plt.figure(figsize=(12,6))
ax = sns.barplot(
    x='gname',
    y='freq_abs',
    data=df_top,
    palette='dark'
)

plt.xticks(rotation=45, ha='right')
plt.title("Total victims (nkill) for Top 10 Deadliest Groups")
plt.ylabel("Total victims")
plt.xlabel("Terrorist Groups")
plt.grid(axis='y')

plt.tight_layout()
plt.show()
```

```
Frequenza assoluta e relativa delle vittime per gruppo terroristico
(prime 10 righe):
                                                  gname  freq_abs
freq_rel
2535                                           Taliban    6518.0
0.14
```

```
1232       Islamic State of Iraq and the Levant (ISIL)    2253.0
0.05
2385                                Shining Path (SL)    2101.0
0.05
1453                   Kurdistan Workers' Party (PKK)    1638.0
0.04
150                                       Al-Shabaab    1596.0
0.04
2196     Revolutionary Armed Forces of Colombia (FARC)    1511.0
0.03
1827                          New People's Army (NPA)    1427.0
0.03
574                                       Boko Haram    1375.0
0.03
1503           Liberation Tigers of Tamil Eelam (LTTE)    1185.0
0.03
904   Farabundo Marti National Liberation Front (FMLN)     961.0
0.02
```



Il **Taliban** domina nettamente la classifica con **oltre 5.500 vittime**, rappresentando il **14%** del totale registrato nel dataset. I gruppi successivi (**Shining Path, ISIL, PKK, FARC…**) si attestano tra **1.000** e **2.000 vittime**, con frequenze relative tra il **2%** e il **5%**.

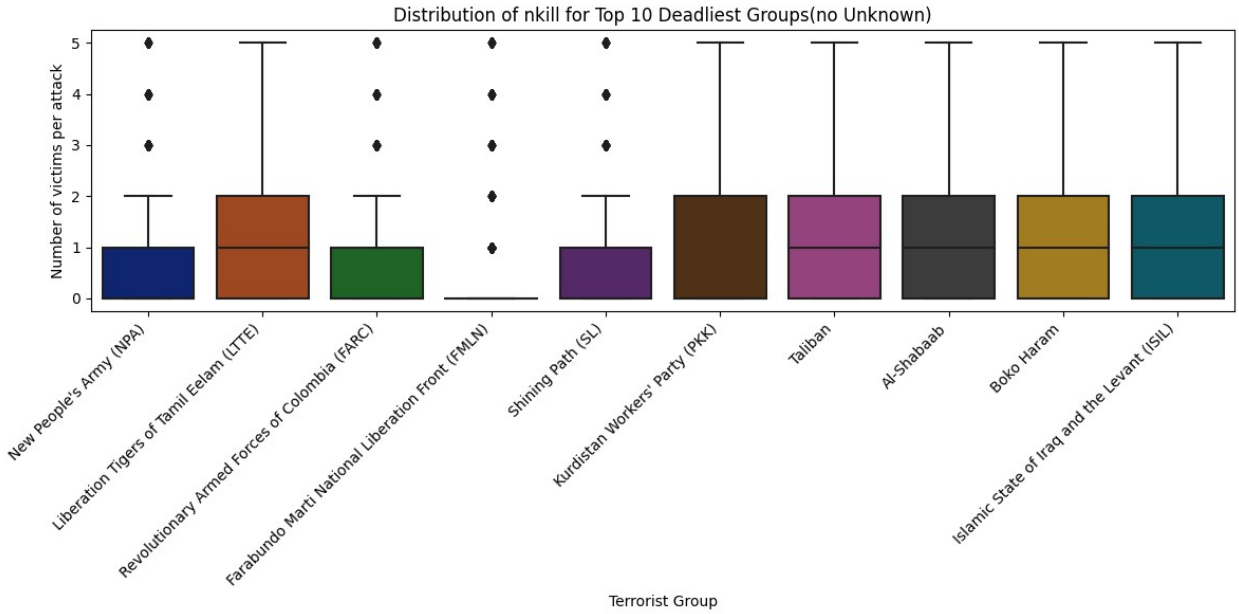
Tuttavia potrebbero esserci attacchi con più vittime o più vittime in un singolo attacco, e il grafico a barre non ci aiuta in questo. per approfondire, possiamo visualizzare i **boxplot** delle vittime per attacco (nkill) per ciascun gruppo, visualizzando mediana, quartili e media:

```
# Calcolo indici statistici per ogni gruppo
stats_per_group = (
    terrorism_filtered2
```

```
    .groupby('gname')['nkill']
    .agg(['count', 'mean', 'median', 'sum'])
    .reset_index()
)

# Ordiniamo per numero totale di vittime
stats_sorted = stats_per_group.sort_values(by='sum', ascending=False)

print("Gruppi terroristici più letali (prime 10 righe):")
print(stats_sorted.head(10))

# Visualizzazione distribuzioni con boxplot (solo top 10 gruppi più
letali)
top_groups = stats_sorted.head(10)['gname']
df_top =
terrorism_filtered2[terrorism_filtered2['gname'].isin(top_groups)]

plt.figure(figsize=(12,6))
ax = sns.boxplot(
    x='gname',
    y='nkill',
    data=df_top,
    palette='dark',
    whis=1.5
)

plt.xticks(rotation=45, ha='right')
plt.title("Distribution of nkill for Top 10 Deadliest Groups(no
Unknown)")
plt.ylabel("Number of victims per attack")
plt.xlabel("Terrorist Group")

# Calcolo e annotazione dei baffi per ciascun gruppo
for i, group in enumerate(top_groups):
    group_data = df_top[df_top['gname'] == group]['nkill'].dropna()
    Q1 = group_data.quantile(0.25)
    Q3 = group_data.quantile(0.75)
    IQR = Q3 - Q1
    lowwhis = Q1 - 1.5*IQR
    upwhis = Q3 + 1.5*IQR

plt.tight_layout()
plt.show()
```

```
Gruppi terroristici più letali (prime 10 righe):
                                                  gname  count
mean  \
2535                                           Taliban   4294
1.517932
1232       Islamic State of Iraq and the Levant (ISIL)   1959
```

```
1.150077
2385                                Shining Path (SL)   3099
0.677961
1453                   Kurdistan Workers' Party (PKK)   1652
0.991525
150                                        Al-Shabaab   1514
1.054161
2196     Revolutionary Armed Forces of Colombia (FARC)   1744
0.866399
1827                           New People's Army (NPA)   1667
0.856029
574                                        Boko Haram    914
1.504376
1503           Liberation Tigers of Tamil Eelam (LTTE)    911
1.300768
904   Farabundo Marti National Liberation Front (FMLN)   1980
0.485354

      median     sum
2535     1.0  6518.0
1232     1.0  2253.0
2385     0.0  2101.0
1453     0.0  1638.0
150      1.0  1596.0
2196     0.0  1511.0
1827     0.0  1427.0
574      1.0  1375.0
1503     1.0  1185.0
904      0.0   961.0
```



Dalla tabella della **media** e della **mediana** notiamo che:

- Il **Taliban** presenta la media più alta: **1.51 vittime per attacco**, su **oltre 3.500 attacchi**. Questo lo rende sia il più attivo che il più letale;
- **Boko Haram** e **LTTE** mostrano medie elevate (**~1.5 e ~1.3**), pur avendo un numero di attacchi inferiore. Questo suggerisce una **strategia più distruttiva per singolo evento**.

Dalla visualizzazione dei **boxplot** notiamo come la **distribuzione delle mediane** sia **schiacciata verso il basso**, tornando infatti alla mediana, è spesso inferiore alla media, indicando una distribuzione con **molti attacchi con poche vittime** e **pochi attacchi molto letali**.

Questi dati si prestano perfettamente a un'**analisi inferenziale** per stimare la **media di vittime per attacco**, tenendo conto della variabilità interna. In particolare: Si potrebbe calcolare l'**intervallo di confidenza**, utile per confrontare la letalità attesa. Questo approccio permetterebbe di distinguere tra **differenze statisticamente significative** e **semplici fluttuazioni casuali**.

Fase 4: Inferenza Statistica

Procediamo con il campionamento di sottoinsiemi popolazione a noi utili per dedurre proprietà interessanti sul nostro dataset GTD.

In particolare tale fase, detta inferenza statistica, ci permette di andare oltre dalla semplice descrizione dei dati e di rispondere a domande più generali sulla **letalità degli attacchi** riportate nel paragrafo successivo.

In questa fase abbiamo utilizzato tecniche inferenziali quali:

- **campionamento**;
- **media, deviazione standard e standard error**;
- **intervalli di confidenza**;
- **test d'ipotesi e p-value**.

4.1 Domande inferenziali considerate

1. **Media uccisioni in Middle East & North Africa**

   "La media di 'nkill' nella regione più colpita è uguale a quella della popolazione?"

2. **Domanda 2 (Numerica)**

   "Qual è il numero medio di vittime per attacco $ \text{nkill} $ e qual è l'intervallo di confidenza al 95% per questa media?"

3. **Domanda 3 (confronto tra due campioni)**

   "Il numero medio di vittime per attacco è diverso tra il periodo prima del 2010 e il periodo 2010–2016?"

4.2 Domanda 1: Media uccisioni in Middle East & North Africa

4.2.1: Definizione della domanda e del campione

Durante l'analisi, abbiamo notato che la maggior parte degli attacchi terroristici degli ultimi anni si concentra principalmente in Middle East & North Africa. Sarebbe utile verificare se la media dei morti corrisponde alla popolazione, o se risulta inferiore o superiore.

- **Campione usato**:
  - Campione casuale di attacchi registrati nella regione ***Middle East & North Africa***. La variabile di interesse è nkill.
- **Obiettivo**:
  - Verificare, tramite un test di ipotesi, se la media dei morti per attacco in MENA è coerente con il valore atteso (ad esempio proporzionale alla popolazione o rispetto alla media globale).

```
interested = 'Middle East & North Africa'
middleeast_northafrica =
terrorism_cleaned[terrorism_cleaned['region_txt'] == interested]


nsample = 5000
sample = middleeast_northafrica.sample(n=nsample, random_state = 42)

mean_sample = sample['nkill'].mean()
std_sample = sample['nkill'].std()

print(f"Campione di {nsample} elementi con region_txt =
'{interested}'")
print(f"Media morti per attacco del campione = {mean_sample:.2f}")
print(f"Deviazione standard morti per attacco del campione =
{std_sample:.2f}")

Campione di 5000 elementi con region_txt = 'Middle East & North
Africa'
Media morti per attacco del campione = 0.82
Deviazione standard morti per attacco del campione = 1.15
```

4.2.2: Test di ipotesi

- **Ipotesi nulla (H0)**: la media dei morti per attacco in 'Middle East & North Africa' è uguale al valore atteso µ0.

- **Ipotesi alternativa (Ha)**: la media dei morti per attacco in 'Middle East & North Africa' è diversa dal valore atteso µ0.

- **Livello di significatività (α)**: 0.05

```
null_hypothesis = terrorism_cleaned['nkill'].mean()


t, pvalue =
stats.ttest_1samp(middleeast_northafrica['nkill'],null_hypothesis)

print(f"T-statistica: {t}")
print(f"p-value: {pvalue}")
```

```
T-statistica: 17.287046004763585
p-value: 1.1796978664301564e-66
```

Diamo un'interpretazione ai valori che abbiamo trovato, infatti posto un livello di significatività (α), si ha che:

```
a = 0.05

if pvalue <= 0.05:
    print("Rifiutiamo l'ipotesi nulla: La media dei morti per attacco
in 'Middle East & North Africa' è diversa dal valore atteso.")
else:
    print("Accettiamo l'ipotesi nulla: La media dei morti per attacco
in 'Middle East & North Africa' è compatibile con il valore atteso.")
```

```
Rifiutiamo l'ipotesi nulla: La media dei morti per attacco in 'Middle
East & North Africa' è diversa dal valore atteso.
```

```
print(f"La media campionaria calcolata dove region_txt = {interested}
è : {mean_sample:.2f}")
print(f"La media della popolazione è uguale a: {null_hypothesis:.2f}")
```

```
La media campionaria calcolata dove region_txt = Middle East & North
Africa è : 0.82
La media della popolazione è uguale a: 0.69
```

4.2.3: Interpretazione

In termini pratici, ciò implica che Middle East & North Africa è l'area con il maggior numero di attacchi e anche una delle aree in cui ***ogni singolo evento terroristico risulta mediamente più letale***. Frequenza e gravità elevata, contribuisce a caratterizzare la regione come ***epicentro principale dell'attività terroristica globale***.

4.3 Domanda 2: Intervallo di confidenza per la media di vittime per attacco

Nel punto 3.2.4 abbiamo accennato quanto **la media di vittime per attacco** si presti bene all'analisi inferenziale, facendo sorgere la questione del suo intervallo di confidenza.

Adesso procediamo step by step per notare differenze significative o fluttuazioni casuali.

4.3.1 Definizione della domanda e del campione

Per iniziare consideriamo:

- **Campione**: Estraiamo un campione casuale di $n = 10.000$ osservazioni dal dataset
- **Obiettivo**: Calcolare la media campionaria e determinare se l'intervallo di confidenza al 95% contiene la "vera" media della popolazione.

```
# Estrazione del campione dalla popolazione (nkill pulito)
nsample = 10000
nkill_sample = terrorism_cleaned['nkill'].sample(n=nsample,
random_state=42)

# Calcolo statistiche campionarie
mean_sample = nkill_sample.mean()
std_sample = nkill_sample.std()

print(f"Campione casuale di {nsample} elementi estratto dalla
popolazione.")
print(f"Media vittime del campione (stimatore): {mean_sample:.4f}")
print(f"Deviazione standard del campione: {std_sample:.4f}")
```

```
Campione casuale di 10000 elementi estratto dalla popolazione.
Media vittime del campione (stimatore): 0.6991
Deviazione standard del campione: 1.1199
```

4.3.2: Test di ipotesi e Intervallo di Confidenza

Per convalidare la nostra stima, confrontiamo la media del campione con la media reale della popolazione (che nel nostro caso è nota e ci aiuta simulare un processo inferenziale).

- **Ipotesi Nulla (H_0):** La media del campione ($\acute{x}$) è uguale alla media della popolazione (μ). $H_0 : \acute{x} = \mu$.
- **Ipotesi Alternativa (H_a):** La media del campione è significativamente diversa dalla media della popolazione. $H_a : \acute{x} \neq \mu$.
- **Livello di significatività (α):** 0.05.

Anche se nel nostro caso si tratta di un campione di 10000 osservazioni, utilizzeremo la **statistica t-student**, adatta anche per **grandi dimenisoni** poiché tende ad una **distribuzione normale**.

```
# Calcolo dell'Intervallo di Confidenza al 95%
confidence_level = 0.95
degrees_freedom = nsample
std = np.std(nkill_sample)
print(f"Deviazione Standard del campione: {std: .4f}")
standard_error = std/np.sqrt(len(nkill_sample))
print(f"Standard Error: {standard_error: .4f}")
print()
# Calcolo estremi dell'intervallo
confidence_interval = stats.t.interval(confidence_level,
degrees_freedom, loc=mean_sample, scale=standard_error)

# Test d'ipotesi (One-sample T-test)
null_hypothesis=terrorism_cleaned['nkill'].mean()
t_stat, pvalue = stats.ttest_1samp(nkill_sample, null_hypothesis)

print(f"Intervallo di Confidenza (95%): [{confidence_interval[0]:.4f},
{confidence_interval[1]:.4f}]")
print(f"Media reale della popolazione: {null_hypothesis:.4f}")
print(f"P-value del test: {pvalue:.4f}")
```

```
Deviazione Standard del campione:  1.1199
Standard Error:  0.0112

Intervallo di Confidenza (95%): [0.6771, 0.7211]
Media reale della popolazione: 0.6878
P-value del test: 0.3111
```

Diamo un'interpretazione ai valori trovati. Infatti, posto un livello di significatività (α), si ha che:

```
alpha = 0.05

if pvalue <= alpha:
    conclusion_h0 = "Rifiutiamo l'ipotesi nulla: la media del campione
differisce significativamente dalla popolazione."
else:
    conclusion_h0 = "Accettiamo l'ipotesi nulla: la media del campione
è statisticamente coerente con la media della popolazione."

#Verifica pratica tramite Intervallo di Confidenza
is_inside = confidence_interval[0] <= null_hypothesis <=
confidence_interval[1]
inside_msg = "SÌ" if is_inside else "NO"

# Output dei risultati
print(f"--- Risultati Inferenziali ---")
```

```
print(f"Media Campionaria: {mean_sample:.4f}")
print(f"Media Popolazione: {null_hypothesis:.4f}")
print(f"Intervallo di Confidenza (95%): [{confidence_interval[0]:.4f},
{confidence_interval[1]:.4f}]")
print(f"La media della popolazione cade nell'intervallo?
{inside_msg}")
print(f"P-value: {pvalue:.4f}")
print(f"Esito: {conclusion_h0}")
```

```
--- Risultati Inferenziali ---
Media Campionaria: 0.6991
Media Popolazione: 0.6878
Intervallo di Confidenza (95%): [0.6771, 0.7211]
La media della popolazione cade nell'intervallo? SÌ
P-value: 0.3111
Esito: Accettiamo l'ipotesi nulla: la media del campione è
statisticamente coerente con la media della popolazione.
```

4.3.3 Interpretazione

Il campione di 10000 eventi è un **ottimo stimatore della realtà globale**, infatti **non abbiamo prove sufficienti** per **rifiutare l'ipotesi nulla**. La **media reale (0.69)** cade esattamente **nell'intervallo di confidenza [0.68, 0.72]**, il quale è anche molto stretto, riducendo al minimo l'incertezza.

Bisogna notare che la **deviazione standard campionaria** è **leggermente superiore** alla **media stessa**. Uno scostamento che rivela una **distribuzione asimmetrica**: molti eventi registrano 0 o 1 vittima e pochi eventi registrano più di una vittima, spostando la media verso l'alto.

4.4 Domanda 3: Confronto tra due campioni

Per mettere in gioco esplicitamente **una divisione in campioni**, si può aggiungere come confronto d'esempio il numero medio di vittime per attacco tra il periodo **prima del 2010** e il periodo **2010–2016**, per verificare la differenza tra loro.

4.4.1 Definizione dei due campioni

In questa analisi consideriamo la variabile numerica $X = \text{nkill}$, che rappresenta il **numero di vittime per attacco**.

Definiamo due **campioni indipendenti** sulla base del periodo temporale:

- **Campione 1 (pre-2010)**: tutti gli attacchi con `year < 2010` per cui `nkill` è presente (non NaN).
- **Campione 2 (2010–2016)**: tutti gli attacchi con `year >= 2010` per cui `nkill` è presente (non NaN).

L'obiettivo sarà confrontare le **medie** di X nei due periodi.

```
sample1 = terrorism_cleaned[(terrorism_cleaned['year'] < 2010) &
(~terrorism_cleaned['nkill'].isna())]['nkill']


sample2 = terrorism_cleaned[(terrorism_cleaned['year'] >= 2010) &
(~terrorism_cleaned['nkill'].isna())]['nkill']


print("Campione 1 (pre-2010):", sample1.shape[0], "osservazioni")
print("Campione 2 (>= 2010):", sample2.shape[0], "osservazioni")
```

```
Campione 1 (pre-2010): 66501 osservazioni
Campione 2 (>= 2010): 55858 osservazioni
```

4.4.2 Ipotesi

Indichiamo con:

- $\mu_1 = E[X \mid \text{year} < 2010]$: media vera del numero di vittime per attacco nel periodo **pre-2010**.
- $\mu_2 = E[X \mid \text{year} \geq 2010]$: media vera del numero di vittime per attacco nel periodo **2010–2016**.

Vogliamo verificare se le due medie di popolazione sono uguali oppure no.

Formuliamo quindi le seguenti ipotesi:

- **Ipotesi nulla** H_0: $\mu_1 = \mu_2$
  (il numero medio di vittime per attacco è lo stesso nei due periodi).

- **Ipotesi alternativa** H_1: $\mu_1 \neq \mu_2$
  (il numero medio di vittime per attacco è diverso nei due periodi).

```
n1, mean1, std1 = len(sample1), sample1.mean(), sample1.std(ddof=1)
n2, mean2, std2 = len(sample2), sample2.mean(), sample2.std(ddof=1)

print("Campione 1 (pre-2010): n1 =", n1, ", mean1 =", mean1, ", std1
=", std1)
print("Campione 2 (>= 2010):  n2 =", n2, ", mean2 =", mean2, ", std2
=", std2)


t_stat, p_value = stats.ttest_ind(sample1, sample2, equal_var=False)

print("Statistica t:", t_stat)
print("p-value:", p_value)
```

```
Campione 1 (pre-2010): n1 = 66501 , mean1 = 0.639163320852318 , std1 =
1.1008770464438982
```

```
Campione 2 (>= 2010):  n2 = 55858 , mean2 = 0.7456049267786172 , std2
= 1.1070900865017037
Statistica t: -16.794985711542306
p-value: 3.1439177399598476e-63
```

4.4.3 Risultati

Dalle statistiche descrittive otteniamo (valori da sostituire con quelli effettivamente calcolati):

- **Campione 1 (pre-2010)**:
  - $n_1 = 66501$ (osservazioni)
  - $\acute{x}_1 = 0.639163320852318$ (media di nkill)
  - $s_1 = 1.1008770464438982$ (deviazione standard)
- **Campione 2 (2010–2016)**:
  - $n_2 = 55858$
  - $\acute{x}_2 = 0.7456049267786172$
  - $s_2 = 1.1070900865017037$

Il t-test a due campioni indipendenti (Welch) fornisce:

- Statistica t: $T \approx -16.794985711542306$
- p-value:
  $p \approx 3.1439\ldots \times 10^{-63} \approx 0,0000000000000000000000000000000000000000000000000000000000000314$

Assumendo un livello di significatività $\alpha = 0.05$, confronteremo il p-value con α nel punto successivo.

4.4.4 Conclusione

Assumendo un livello di significatività $\alpha = 0.05$:

- se $p \leq 0.05$, rifiutiamo l'ipotesi nulla $H_0 : \mu_1 = \mu_2$;
- se $p > 0.05$, non abbiamo evidenza sufficiente per rifiutare H_0.

Nel nostro caso, il p-value calcolato è $p \approx 3.1439\ldots \times 10^{-63}$.

Di conseguenza $p \leq 0.05$, quindi rifiutiamo H_0 e concludiamo che esiste una differenza statisticamente significativa nel numero medio di vittime per attacco tra il periodo pre-2010 e il periodo 2010–2016.

Osservando le medie campionarie, si può notare che $\acute{x}_2 > \acute{x}_1$: quindi il numero medio di vittime per attacco risulta **più alto** nel periodo 2010–2016 rispetto al periodo precedente, suggerendo che, nel periodo recente, gli attacchi tendevano ad essere più letali (nel campione GTD considerato).

4.5 Riepilogo finale della Fase 4

Chiudiamo con il riepilogo delle informazioni ottenute, che riassume:

- **Il Middle East & North Africa** come l'area con il maggior numero di attacchi, ognuno dei quali ***risulta mediamente più letale***, potendo affermare che tali aree sono l'***epicentro principale dell'attività terroristica globale***;
- Il **numero medio di vittime per attacco di 0.69** in un intervallo di confidenza al 95% di **[0.68, 0.72]**, notando **molti attacchi con poche vittime** e **pochi attacchi con più di una vittima**;
- Nella **differenza tra il pre-2010 e il 2010-2016** sul **numero medio di vittime per attacco**, come ***più alto*** nel periodo **2010–2016** rispetto al periodo precedente, suggerendo che, nel periodo ***recente***, gli ***attacchi*** tendevano ad essere ***più letali***.

FASE 5: Analisi Statistica(Inference)

L'*analisi statistica* condotta in questa sezione si basa sull'utilizzo di *strumenti di regressione*, al fine di individuare le **relazioni significative** tra le **caratteristiche** di attacchi terroristici(numero morti, ostaggi e tempi di attacco estesi) e un set di **variabili che descrivono il fenomeno**.

L'obiettivo non è quindi la previsione di eventi futuri, ma l'analisi delle probabilità condizionate e dei pattern storici che emergono dai dati studiati.

In particolare, l'attenzione è rivolta all'analisi degli effetti associati a specifiche modalità di attacco, al fine di comprendere quali **tipologie** risultino potenzialmente **più letali** o richiedano un livello di **preparazione maggiore**.

Ad esempio, se determinate condizioni di un attacco sono associate a una maggiore probabilità di presa di ostaggi, potrebbe rendersi necessario l'intervento di negoziatori specializzati; analogamente, se alcune caratteristiche indicano una maggiore probabilità che l'attacco si protragga per diverse ore o giorni, ciò implicherebbe la necessità di una pianificazione logistica migliore da parte delle autorità.

5.1 Analisi statistica -'nkill'

Nelle fasi precedenti abbiamo verificato se c'era presenza di correlazione tra ***'nkill'*** e variabili discrete quali:

- **nwound**,
- **nkill**,
- **nperpcap**.

I risultati hanno dimostrato che c'è una piccola correlazione solo con **nwound**, mentre con le altre features non c'è alcuna correlazione. Questo è coerente con la distribuzione dei dati, in cui la maggior parte dei valori assume 0 o 1.

Possiamo però trovare un **modello di regressione** lineare che spieghi almeno in parte la variabilità di **nkill**, introducendo anche altre variabili a disposizione?

Dato che la variabile è **discreta** e presenta valori molto piccoli, applicare direttamente una regressione lineare potrebbe essere ininfluente o erroneo, perché:

- la **regressione lineare** produce valori continui, quindi, confrontare direttamente la discreta nkill con la stima continua può portare a interpretazioni poco precise;
- si devono gestire valori come 0 e outlier(anche se rimossi in precedenza). Per questo motivo si utilizza il log1p che gestisce anche valori di nkill = 0(presente in maggioranza).

Prendiamo variabili che possono riguardare il numero di morti, come:

- ***weaptype1_txt***;
- ***attacktype1_txt***;
- ***targtype1_txt***;
- ***nwound***;
- ***suicide***;
- ***success***;

Si potevano utilizzare gli indici numerici per le variabili categoriali, ma si è preferito il testo per una veloce intepretazione durante la fase di analisi.

```
colols =
['weaptype1_txt','attacktype1_txt','region_txt','targtype1_txt','nkill
','nwound','success','suicide']
dataforols = terrorism_cleaned[colols].copy()

dataforols = dataforols[(dataforols['attacktype1_txt'] != "Unknown") &
(dataforols['targtype1_txt'] != "Unknown") &
(dataforols['weaptype1_txt'] != "Unknown")]

dataforols = tocategory(dataforols,
['weaptype1_txt','attacktype1_txt','region_txt','targtype1_txt'])
dataforols = toint(dataforols,['success','suicide'])



dataforols['lognkill'] = np.log1p(dataforols['nkill'])
model = smf.ols('lognkill ~ C(targtype1_txt) + C(attacktype1_txt) +
C(region_txt) + C(weaptype1_txt) + success + suicide +
nwound',data=dataforols).fit()

model.summary()

<class 'statsmodels.iolib.summary.Summary'>
"""
                            OLS Regression Results

==============================================================================
Dep. Variable:               lognkill   R-squared:
0.329
```

```
Model:                            OLS   Adj. R-squared:
0.329
Method:                 Least Squares   F-statistic:
1050.
Date:                Thu, 22 Jan 2026   Prob (F-statistic):
0.00
Time:                        09:47:32   Log-Likelihood:
-59396.
No. Observations:              109311   AIC:
1.189e+05
Df Residuals:                  109259   BIC:
1.194e+05
Df Model:                          51

Covariance Type:            nonrobust

==============================================================================
========================================================================
=======================

coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------------------------------------
-----------------------
Intercept
0.3794      0.099      3.844      0.000       0.186       0.573
C(targtype1_txt)[T.Airports & Aircraft]
-0.0684      0.039     -1.766      0.077      -0.144       0.008
C(targtype1_txt)[T.Business]
-0.0302      0.036     -0.837      0.403      -0.101       0.041
C(targtype1_txt)[T.Educational Institution]
-0.0813      0.037     -2.214      0.027      -0.153      -0.009
C(targtype1_txt)[T.Food or Water Supply]
-0.0752      0.046     -1.636      0.102      -0.165       0.015
C(targtype1_txt)[T.Government (Diplomatic)]
-0.0941      0.037     -2.558      0.011      -0.166      -0.022
C(targtype1_txt)[T.Government (General)]
-0.0269      0.036     -0.745      0.456      -0.098       0.044
C(targtype1_txt)[T.Journalists & Media]
-0.1213      0.037     -3.269      0.001      -0.194      -0.049
C(targtype1_txt)[T.Maritime]
-0.1117      0.049     -2.303      0.021      -0.207      -0.017
C(targtype1_txt)[T.Military]
0.1237      0.036      3.417      0.001       0.053       0.195
C(targtype1_txt)[T.NGO]
-0.0650      0.040     -1.629      0.103      -0.143       0.013
C(targtype1_txt)[T.Other]
0.0160      0.042      0.381      0.703      -0.066       0.099
C(targtype1_txt)[T.Police]
```

```
0.0978      0.036      2.703      0.007       0.027       0.169
C(targtype1_txt)[T.Private Citizens & Property]
0.1012      0.036      2.802      0.005       0.030       0.172
C(targtype1_txt)[T.Religious Figures/Institutions]
-0.0015      0.037     -0.042      0.967      -0.074       0.070
C(targtype1_txt)[T.Telecommunication]
-0.0990      0.039     -2.535      0.011      -0.175      -0.022
C(targtype1_txt)[T.Terrorists/Non-State Militia]
0.2155      0.037      5.775      0.000       0.142       0.289
C(targtype1_txt)[T.Tourists]
-0.0457      0.045     -1.025      0.305      -0.133       0.042
C(targtype1_txt)[T.Transportation]
-0.0419      0.037     -1.146      0.252      -0.114       0.030
C(targtype1_txt)[T.Utilities]
-0.1428      0.037     -3.908      0.000      -0.214      -0.071
C(targtype1_txt)[T.Violent Political Party]
-0.0233      0.038     -0.615      0.538      -0.098       0.051
C(attacktype1_txt)[T.Assassination]
0.1186      0.005     25.762      0.000       0.110       0.128
C(attacktype1_txt)[T.Bombing/Explosion]
-0.1370      0.007    -18.860      0.000      -0.151      -0.123
C(attacktype1_txt)[T.Facility/Infrastructure Attack]
-0.1756      0.009    -20.096      0.000      -0.193      -0.159
C(attacktype1_txt)[T.Hijacking]
-0.2979      0.024    -12.348      0.000      -0.345      -0.251
C(attacktype1_txt)[T.Hostage Taking (Barricade Incident)]
-0.3019      0.020    -15.273      0.000      -0.341      -0.263
C(attacktype1_txt)[T.Hostage Taking (Kidnapping)]
-0.2455      0.008    -32.629      0.000      -0.260      -0.231
C(attacktype1_txt)[T.Unarmed Assault]
-0.3975      0.021    -19.282      0.000      -0.438      -0.357
C(region_txt)[T.Central America & Caribbean]
0.0383      0.030      1.273      0.203      -0.021       0.097
C(region_txt)[T.Central Asia]
0.0870      0.036      2.416      0.016       0.016       0.158
C(region_txt)[T.East Asia]
0.0243      0.034      0.712      0.477      -0.043       0.091
C(region_txt)[T.Eastern Europe]
0.0095      0.030      0.313      0.754      -0.050       0.069
C(region_txt)[T.Middle East & North Africa]
0.1236      0.030      4.173      0.000       0.066       0.182
C(region_txt)[T.North America]
0.0274      0.031      0.882      0.378      -0.034       0.088
C(region_txt)[T.South America]
0.0536      0.030      1.804      0.071      -0.005       0.112
C(region_txt)[T.South Asia]
0.1566      0.030      5.289      0.000       0.099       0.215
C(region_txt)[T.Southeast Asia]
0.0510      0.030      1.710      0.087      -0.007       0.110
```

```
C(region_txt)[T.Sub-Saharan Africa]
0.1655      0.030      5.545      0.000       0.107       0.224
C(region_txt)[T.Western Europe]
-0.0292      0.030     -0.984      0.325      -0.087       0.029
C(weaptype1_txt)[T.Chemical]
-0.2500      0.090     -2.770      0.006      -0.427      -0.073
C(weaptype1_txt)[T.Explosives/Bombs/Dynamite]
-0.4464      0.086     -5.176      0.000      -0.615      -0.277
C(weaptype1_txt)[T.Fake Weapons]
-0.2592      0.115     -2.245      0.025      -0.485      -0.033
C(weaptype1_txt)[T.Firearms]
-0.1904      0.086     -2.213      0.027      -0.359      -0.022
C(weaptype1_txt)[T.Incendiary]
-0.5460      0.086     -6.327      0.000      -0.715      -0.377
C(weaptype1_txt)[T.Melee]
-0.2306      0.086     -2.690      0.007      -0.399      -0.063
C(weaptype1_txt)[T.Other]
-0.1698      0.098     -1.726      0.084      -0.363       0.023
C(weaptype1_txt)[T.Radiological]
0.0178      0.143      0.124      0.901      -0.263       0.298
C(weaptype1_txt)[T.Sabotage Equipment]
-0.4777      0.096     -4.979      0.000      -0.666      -0.290
C(weaptype1_txt)[T.Vehicle (not to include vehicle-borne explosives,
i.e., car or truck bombs)]    -0.2075      0.095     -2.182      0.029
-0.394      -0.021
success
0.2995      0.004     71.117      0.000       0.291       0.308
suicide
0.6877      0.010     66.565      0.000       0.667       0.708
nwound
0.0349      0.001     32.855      0.000       0.033       0.037
==============================================================================
Omnibus:                    13417.491   Durbin-Watson:
1.817
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
20089.995
Skew:                           0.906   Prob(JB):
0.00
Kurtosis:                       4.061   Cond. No.
444.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
"""
```

Possiamo vedere come la variabile **region_txt** ha valori alti del p-value per quasi tutti i valori. Procediamo a rimuoverla secondo il criterio di **backward elimination**.

```
model = smf.ols('lognkill ~ C(targtype1_txt) + C(attacktype1_txt) +
C(weaptype1_txt) + success + suicide + nwound ',data=dataforols).fit()
model.summary()

<class 'statsmodels.iolib.summary.Summary'>
"""
                            OLS Regression Results

==========================================================================
========
Dep. Variable:               lognkill   R-squared:
0.315
Model:                            OLS   Adj. R-squared:
0.315
Method:                 Least Squares   F-statistic:
1256.
Date:                Thu, 22 Jan 2026   Prob (F-statistic):
0.00
Time:                        09:47:33   Log-Likelihood:
-60522.
No. Observations:              109311   AIC:
1.211e+05
Df Residuals:                  109270   BIC:
1.215e+05
Df Model:                          40

Covariance Type:            nonrobust

==========================================================================
==========================================================================
========================

coef    std err          t      P>|t|      [0.025      0.975]
--------------------------------------------------------------------------
--------------------------------------------------------------------------
------------------------
Intercept
0.3804      0.093      4.071      0.000       0.197       0.564
C(targtype1_txt)[T.Airports & Aircraft]
-0.0452      0.038     -1.192      0.233      -0.120       0.029
C(targtype1_txt)[T.Business]
-0.0114      0.035     -0.324      0.746      -0.080       0.058
C(targtype1_txt)[T.Educational Institution]
-0.0111      0.036     -0.310      0.757      -0.081       0.059
C(targtype1_txt)[T.Food or Water Supply]
-0.0208      0.045     -0.458      0.647      -0.110       0.068
C(targtype1_txt)[T.Government (Diplomatic)]
```

```
-0.0659      0.036     -1.834      0.067      -0.136       0.005
C(targtype1_txt)[T.Government (General)]
0.0175      0.035      0.499      0.618      -0.051       0.086
C(targtype1_txt)[T.Journalists & Media]
-0.0867      0.036     -2.393      0.017      -0.158      -0.016
C(targtype1_txt)[T.Maritime]
-0.0790      0.048     -1.644      0.100      -0.173       0.015
C(targtype1_txt)[T.Military]
0.1693      0.035      4.810      0.000       0.100       0.238
C(targtype1_txt)[T.NGO]
-0.0002      0.039     -0.004      0.997      -0.077       0.076
C(targtype1_txt)[T.Other]
0.0531      0.041      1.284      0.199      -0.028       0.134
C(targtype1_txt)[T.Police]
0.1464      0.035      4.161      0.000       0.077       0.215
C(targtype1_txt)[T.Private Citizens & Property]
0.1559      0.035      4.440      0.000       0.087       0.225
C(targtype1_txt)[T.Religious Figures/Institutions]
0.0511      0.036      1.426      0.154      -0.019       0.121
C(targtype1_txt)[T.Telecommunication]
-0.0512      0.038     -1.341      0.180      -0.126       0.024
C(targtype1_txt)[T.Terrorists/Non-State Militia]
0.2799      0.036      7.697      0.000       0.209       0.351
C(targtype1_txt)[T.Tourists]
-0.0364      0.044     -0.828      0.408      -0.123       0.050
C(targtype1_txt)[T.Transportation]
0.0092      0.036      0.259      0.796      -0.060       0.079
C(targtype1_txt)[T.Utilities]
-0.1032      0.036     -2.901      0.004      -0.173      -0.033
C(targtype1_txt)[T.Violent Political Party]
0.0671      0.037      1.814      0.070      -0.005       0.140
C(attacktype1_txt)[T.Assassination]
0.1032      0.005     22.347      0.000       0.094       0.112
C(attacktype1_txt)[T.Bombing/Explosion]
-0.1537      0.007    -21.016      0.000      -0.168      -0.139
C(attacktype1_txt)[T.Facility/Infrastructure Attack]
-0.1818      0.009    -20.624      0.000      -0.199      -0.165
C(attacktype1_txt)[T.Hijacking]
-0.2949      0.024    -12.100      0.000      -0.343      -0.247
C(attacktype1_txt)[T.Hostage Taking (Barricade Incident)]
-0.3354      0.020    -16.831      0.000      -0.374      -0.296
C(attacktype1_txt)[T.Hostage Taking (Kidnapping)]
-0.2294      0.008    -30.237      0.000      -0.244      -0.215
C(attacktype1_txt)[T.Unarmed Assault]
-0.4142      0.021    -19.907      0.000      -0.455      -0.373
C(weaptype1_txt)[T.Chemical]
-0.2367      0.091     -2.605      0.009      -0.415      -0.059
C(weaptype1_txt)[T.Explosives/Bombs/Dynamite]
-0.3943      0.087     -4.543      0.000      -0.564      -0.224
```

```
C(weaptype1_txt)[T.Fake Weapons]
-0.2402      0.116     -2.063      0.039      -0.468      -0.012
C(weaptype1_txt)[T.Firearms]
-0.1431      0.087     -1.653      0.098      -0.313       0.027
C(weaptype1_txt)[T.Incendiary]
-0.5294      0.087     -6.095      0.000      -0.700      -0.359
C(weaptype1_txt)[T.Melee]
-0.1760      0.086     -2.039      0.041      -0.345      -0.007
C(weaptype1_txt)[T.Other]
-0.1381      0.099     -1.393      0.164      -0.332       0.056
C(weaptype1_txt)[T.Radiological]
0.0044      0.144      0.031      0.976      -0.278       0.287
C(weaptype1_txt)[T.Sabotage Equipment]
-0.4323      0.097     -4.472      0.000      -0.622      -0.243
C(weaptype1_txt)[T.Vehicle (not to include vehicle-borne explosives,
i.e., car or truck bombs)]    -0.1566      0.096     -1.634      0.102
-0.344       0.031
success
0.3052      0.004     71.988      0.000       0.297       0.313
suicide
0.7251      0.010     69.681      0.000       0.705       0.745
nwound
0.0418      0.001     39.653      0.000       0.040       0.044
==============================================================================
========
Omnibus:                    14234.358   Durbin-Watson:
1.807
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
21882.634
Skew:                           0.937   Prob(JB):
0.00
Kurtosis:                       4.136   Cond. No.
433.
==============================================================================
========

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
"""
```

Dato che alcuni **p-value** mostrano correlazione tra logaritmo delle vittime e le categorie, utilizziamo alcuni valori delle categorie come **baseline**. In questo modo anche le categorie non significativamente correlate avranno p-value basso rispetto alla baseline.

Attraverso il metodo **Treatment** abbiamo posto ***"Military"*** come baseline per **targtype1_txt** e ***"Explosives/Bombs/Dynamite"*** come baseline per **weaptype1_txt**.

```
model = smf.ols('lognkill ~ C(targtype1_txt, Treatment(reference
="Military")) + C(attacktype1_txt)+ C(weaptype1_txt,
Treatment(reference="Explosives/Bombs/Dynamite"))+ success + suicide +
nwound ',data=dataforols).fit()
model.summary()
```

```
<class 'statsmodels.iolib.summary.Summary'>
"""
                            OLS Regression Results

==============================================================================
========
Dep. Variable:               lognkill   R-squared:
0.315
Model:                            OLS   Adj. R-squared:
0.315
Method:                 Least Squares   F-statistic:
1256.
Date:                Thu, 22 Jan 2026   Prob (F-statistic):
0.00
Time:                        09:47:34   Log-Likelihood:
-60522.
No. Observations:              109311   AIC:
1.211e+05
Df Residuals:                  109270   BIC:
1.215e+05
Df Model:                          40

Covariance Type:            nonrobust

==============================================================================
==============================================================================
==============================================================================
===

coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
------------------------------------------------------------------------------
------------------------------------------------------------------------------
---
Intercept
0.1555      0.008     18.945      0.000       0.139       0.172
C(targtype1_txt, Treatment(reference="Military"))[T.Abortion Related]
-0.1693      0.035     -4.810      0.000      -0.238      -0.100
C(targtype1_txt, Treatment(reference="Military"))[T.Airports &
Aircraft]
-0.2145      0.015    -14.182      0.000      -0.244      -0.185
C(targtype1_txt, Treatment(reference="Military"))[T.Business]
-0.1807      0.005    -34.722      0.000      -0.191      -0.171
C(targtype1_txt, Treatment(reference="Military"))[T.Educational
```

```
Institution]
-0.1804      0.008    -21.703      0.000      -0.197      -0.164
C(targtype1_txt, Treatment(reference="Military"))[T.Food or Water
Supply]
-0.1901      0.029     -6.524      0.000      -0.247      -0.133
C(targtype1_txt, Treatment(reference="Military"))[T.Government
(Diplomatic)]
-0.2353      0.009    -26.091      0.000      -0.253      -0.218
C(targtype1_txt, Treatment(reference="Military"))[T.Government
(General)]
-0.1518      0.005    -30.513      0.000      -0.162      -0.142
C(targtype1_txt, Treatment(reference="Military"))[T.Journalists &
Media]
-0.2561      0.010    -25.405      0.000      -0.276      -0.236
C(targtype1_txt, Treatment(reference="Military"))[T.Maritime]
-0.2484      0.033     -7.498      0.000      -0.313      -0.183
C(targtype1_txt, Treatment(reference="Military"))[T.NGO]
-0.1695      0.018     -9.554      0.000      -0.204      -0.135
C(targtype1_txt, Treatment(reference="Military"))[T.Other]
-0.1163      0.022     -5.187      0.000      -0.160      -0.072
C(targtype1_txt, Treatment(reference="Military"))[T.Police]
-0.0229      0.005     -4.779      0.000      -0.032      -0.014
C(targtype1_txt, Treatment(reference="Military"))[T.Private Citizens &
Property]
-0.0134      0.004     -2.987      0.003      -0.022      -0.005
C(targtype1_txt, Treatment(reference="Military"))[T.Religious
Figures/Institutions]
-0.1183      0.009    -13.613      0.000      -0.135      -0.101
C(targtype1_txt, Treatment(reference="Military"))[T.Telecommunication]
-0.2205      0.016    -13.885      0.000      -0.252      -0.189
C(targtype1_txt, Treatment(reference="Military"))[T.Terrorists/Non-
State Militia]
0.1105      0.010     10.702      0.000       0.090       0.131
C(targtype1_txt, Treatment(reference="Military"))[T.Tourists]
-0.2058      0.027     -7.654      0.000      -0.258      -0.153
C(targtype1_txt, Treatment(reference="Military"))[T.Transportation]
-0.1601      0.007    -21.654      0.000      -0.175      -0.146
C(targtype1_txt, Treatment(reference="Military"))[T.Utilities]
-0.2725      0.007    -37.371      0.000      -0.287      -0.258
C(targtype1_txt, Treatment(reference="Military"))[T.Violent Political
Party]
-0.1023      0.012     -8.217      0.000      -0.127      -0.078
C(attacktype1_txt)[T.Assassination]
0.1032      0.005     22.347      0.000       0.094       0.112
C(attacktype1_txt)[T.Bombing/Explosion]
-0.1537      0.007    -21.016      0.000      -0.168      -0.139
C(attacktype1_txt)[T.Facility/Infrastructure Attack]
-0.1818      0.009    -20.624      0.000      -0.199      -0.165
C(attacktype1_txt)[T.Hijacking]
```

```
-0.2949      0.024    -12.100      0.000      -0.343      -0.247
C(attacktype1_txt)[T.Hostage Taking (Barricade Incident)]
-0.3354      0.020    -16.831      0.000      -0.374      -0.296
C(attacktype1_txt)[T.Hostage Taking (Kidnapping)]
-0.2294      0.008    -30.237      0.000      -0.244      -0.215
C(attacktype1_txt)[T.Unarmed Assault]
-0.4142      0.021    -19.907      0.000      -0.455      -0.373
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Biological]
0.3943      0.087      4.543      0.000       0.224       0.564
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Chemical]
0.1576      0.036      4.362      0.000       0.087       0.228
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Fake Weapons]
0.1540      0.079      1.957      0.050      -0.000       0.308
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Firearms]
0.2512      0.007     35.942      0.000       0.237       0.265
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Incendiary]
-0.1351      0.010    -13.085      0.000      -0.155      -0.115
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Melee]
0.2183      0.011     19.391      0.000       0.196       0.240
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Other]
0.2561      0.052      4.955      0.000       0.155       0.357
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Radiological]
0.3987      0.119      3.355      0.001       0.166       0.632
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Sabotage Equipment]
-0.0380      0.044     -0.870      0.384      -0.124       0.048
C(weaptype1_txt, Treatment(reference="Explosives/Bombs/Dynamite"))
[T.Vehicle (not to include vehicle-borne explosives, i.e., car or
truck bombs)]     0.2377      0.047      5.004      0.000       0.145
0.331
success
0.3052      0.004     71.988      0.000       0.297       0.313
suicide
0.7251      0.010     69.681      0.000       0.705       0.745
nwound
0.0418      0.001     39.653      0.000       0.040       0.044
==========================================================================
========
Omnibus:                    14234.358   Durbin-Watson:
1.807
Prob(Omnibus):                  0.000   Jarque-Bera (JB):
```

```
21882.634
Skew:                           0.937   Prob(JB):
0.00
Kurtosis:                       4.136   Cond. No.
171.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is
correctly specified.
"""
```

Come spiegato sopra, stiamo calcolando un modello di regressione lineare che spiega il logaritmo di nkill. Il modello spiega circa il **32%** della variabilità nei dati: un valore non altissimo, ma in dataset così grandi e complessi descrive comunque parte del fenomeno. Le variabili **success**, **suicide** e **nwound** risultano tutte correlate statisticamente. Ad esempio, per suicide, un aumento di un'unità è associato a un incremento medio del numero di vittime di circa il **101%** rispetto la baseline, calcolato come **(e^0.7)-1**.

Analizzando vari casi, abbiamo che:

- **β0** ossia l'**intercetta** quando tutti i valori non vengono considerati è pari a **0.155**, si ha un valore di **(e^0.155)**-1 ossia un valore atteso di **nkill = 0,16**;
- quando **βi = -0,2145** (ad esempio per targettype = Airports & Aircraft) indica che rispetto alla baseline, il logaritmo del numero di vittime diminuisce di 0.2145. In termini percentuali si traduce in **−19.2%**, quindi ci si aspetta vittime in meno rispetto a un attacco contro la baseline.
- in attacktype1_txt, tutti gli attacchi tranne ***Assassination*** hanno coefficienti negativi, perchè tendono a causare meno vittime rispetto alla baseline.

5.2 Analisi statistica -'ishostkid'

L'obiettivo di questa analisi è capire quali delle variabili o condizioni sono associate alla presenza di ostaggi durante un attacco terroristico, ossia **'ishostkid = True'**. Per far ciò, utilizziamo una regressine logistica, che è adatta quando una variabile dipendente assume due valori ***vero/falso***, permettendo inoltre di stimare la probabilità di un attacco. Comprendere questi fattori permette di delineare pattern ricorrenti negli attacchi che coinvolgono ostaggi, e può aiutare nell'analisi dei dati. Per far ciò si prendono in considerazione le seguenti variabili:

- **'targtype1_txt'** -> il tipo del bersaglio può condizionare l'attacco e la probabilità che siano presi ostaggi;
- **'weaptype1_txt'** -> l'utilizzo di un'arma cambia a seconda dello scopo dell'attacco;
- **'nkill'**;
- **'nperps'**;
- **'nwound'**;
- **'extended'** -> indica se l'attacco dura più di 24 ore; attacchi lunghi potrebbero coinvolgere ostaggi.

- **'multiple'**-> incidenti multipli si verificano insieme;
- **'region_txt'**-> regione geografica, per catturare eventuali differenze nella tattica degli attacchi.

Inizialmente era stata considerata anche ***'suicide'***, dato che poteva risultare anticorrelato(se c'è un suicidio da parte del terrorista, probabilmente non era previsto il rapimento di ostaggi) a **'ishostkid'**.

Tuttavia, si è scelto di toglierla perchè non era statistcamente significativa(P-value molto alto) ed era causa di errore del modello.

Inoltre, si è deciso di rimuovere le categorie con pochissimi dati, dato che possono distorcere il modello. Si è posta una soglia **threshold = 1000** che toglie tutte le categorie irrilevanti al fine dell'analisi.

```
collogit =
['attacktype1_txt','targtype1_txt','weaptype1_txt','ishostkid','nkill'
,'success','nperps','extended','multiple','nwound','region_txt']

dataforlogit = terrorism_cleaned[collogit].copy()


threshold = 1000
categoricalcol = ['attacktype1_txt', 'targtype1_txt', 'weaptype1_txt']

for col in categoricalcol:
    counts = dataforlogit[col].value_counts()
    toremove = counts[counts < threshold].index
    dataforlogit = dataforlogit[~dataforlogit[col].isin(toremove)]


dataforlogit = tocategory(dataforlogit,
['attacktype1_txt','targtype1_txt','weaptype1_txt','region_txt'])
dataforlogit = toint(dataforlogit,
['success','ishostkid','multiple','extended'])


model = logit('ishostkid ~ C(targtype1_txt) + C(weaptype1_txt) +
C(region_txt) + multiple +extended + nkill + success + nperps +
nwound', data=dataforlogit).fit()

model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.107481
         Iterations 10

<class 'statsmodels.iolib.summary.Summary'>
"""
                           Logit Regression Results
```

```
==============================================================================
========
Dep. Variable:              ishostkid   No. Observations:
14976
Model:                          Logit   Df Residuals:
14941
Method:                           MLE   Df Model:
34
Date:                Thu, 22 Jan 2026   Pseudo R-squ.:
0.5894
Time:                        09:47:36   Log-Likelihood:
-1609.6
converged:                       True   LL-Null:
-3920.7
Covariance Type:            nonrobust   LLR p-value:
0.000
==============================================================================
================================================
                                                                  coef    std
err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
-----------------------------------------------
Intercept                                                      -8.4385
2.014     -4.190      0.000     -12.386      -4.491
C(targtype1_txt)[T.Educational Institution]                    -0.0669
0.245     -0.273      0.785      -0.546       0.413
C(targtype1_txt)[T.Government (Diplomatic)]                    -0.1736
0.230     -0.756      0.450      -0.624       0.277
C(targtype1_txt)[T.Government (General)]                       -0.7458
0.167     -4.472      0.000      -1.073      -0.419
C(targtype1_txt)[T.Journalists & Media]                        -0.4976
0.248     -2.009      0.045      -0.983      -0.012
C(targtype1_txt)[T.Military]                                   -1.5059
0.255     -5.914      0.000      -2.005      -1.007
C(targtype1_txt)[T.Police]                                     -1.5929
0.222     -7.171      0.000      -2.028      -1.158
C(targtype1_txt)[T.Private Citizens & Property]                 0.0030
0.151      0.020      0.984      -0.293       0.299
C(targtype1_txt)[T.Religious Figures/Institutions]             -1.0624
0.355     -2.992      0.003      -1.758      -0.366
C(targtype1_txt)[T.Terrorists/Non-State Militia]               -0.1202
0.368     -0.327      0.744      -0.842       0.601
C(targtype1_txt)[T.Transportation]                             -1.1849
0.441     -2.688      0.007      -2.049      -0.321
C(targtype1_txt)[T.Unknown]                                    -0.2493
0.551     -0.452      0.651      -1.329       0.831
C(targtype1_txt)[T.Utilities]                                  -1.9059
0.835     -2.284      0.022      -3.542      -0.270
C(targtype1_txt)[T.Violent Political Party]                    -1.5105
```

```
0.536     -2.816      0.005      -2.562      -0.459
C(weaptype1_txt)[T.Firearms]                           2.3453
0.253      9.265      0.000       1.849       2.841
C(weaptype1_txt)[T.Incendiary]                        -0.9772
0.557     -1.753      0.080      -2.070       0.115
C(weaptype1_txt)[T.Melee]                              2.5581
0.326      7.836      0.000       1.918       3.198
C(weaptype1_txt)[T.Unknown]                            4.1063
0.265     15.496      0.000       3.587       4.626
C(region_txt)[T.Central America & Caribbean]           3.0155
1.998      1.509      0.131      -0.900       6.931
C(region_txt)[T.Central Asia]                          3.1017
2.096      1.480      0.139      -1.007       7.210
C(region_txt)[T.East Asia]                             2.1039
2.079      1.012      0.312      -1.971       6.179
C(region_txt)[T.Eastern Europe]                        3.4058
2.006      1.698      0.090      -0.526       7.337
C(region_txt)[T.Middle East & North Africa]            2.8229
1.991      1.418      0.156      -1.079       6.724
C(region_txt)[T.North America]                         3.2160
2.001      1.607      0.108      -0.706       7.138
C(region_txt)[T.South America]                         3.1513
1.990      1.584      0.113      -0.749       7.051
C(region_txt)[T.South Asia]                            2.8003
1.990      1.407      0.159      -1.099       6.700
C(region_txt)[T.Southeast Asia]                        2.4285
1.992      1.219      0.223      -1.476       6.333
C(region_txt)[T.Sub-Saharan Africa]                    2.5777
1.997      1.291      0.197      -1.337       6.492
C(region_txt)[T.Western Europe]                        2.6003
1.991      1.306      0.192      -1.302       6.503
multiple                                              -0.6583
0.292     -2.257      0.024      -1.230      -0.087
extended                                               6.4357
0.344     18.686      0.000       5.761       7.111
nkill                                                 -0.9758
0.082    -11.902      0.000      -1.137      -0.815
success                                                0.9990
0.168      5.940      0.000       0.669       1.329
nperps                                                 0.2212
0.017     12.856      0.000       0.187       0.255
nwound                                                -0.6397
0.086     -7.455      0.000      -0.808      -0.472
==============================================================================
==================================================
"""
```

Come mostrato attraverso la funzione ***.summary()*** del modello di regressione logistica, possiamo osservare che i p-value di ***multiple*** e di tutte le categorie di ***region_txt*** sono molto alti,

quindi queste variabili non sono statisticamente significative nel predire la presenza di ostaggi. Inoltre, alcune categorie specifiche di ***targtype1_txt*** e ***weaptype1_txt*** presentano p-value elevati, ma sono mantenute nel modello per consentire una distinzione chiara tra categorie significative e non.

```
modelcleaned = logit('ishostkid ~ C(targtype1_txt) + C(weaptype1_txt)
+ nkill + success + nperps + extended +
nwound',data=dataforlogit).fit()
modelcleaned.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.108765
         Iterations 10

<class 'statsmodels.iolib.summary.Summary'>
"""
                           Logit Regression Results

==============================================================================
========
Dep. Variable:              ishostkid   No. Observations:
14976
Model:                          Logit   Df Residuals:
14953
Method:                           MLE   Df Model:
22
Date:                Thu, 22 Jan 2026   Pseudo R-squ.:
0.5845
Time:                        09:47:37   Log-Likelihood:
-1628.9
converged:                       True   LL-Null:
-3920.7
Covariance Type:            nonrobust   LLR p-value:
0.000
==============================================================================
==================================================
                                                          coef    std
err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
-------------------------------------------------
Intercept                                                -5.6849
0.302    -18.852      0.000      -6.276      -5.094
C(targtype1_txt)[T.Educational Institution]              -0.0583
0.239     -0.244      0.808      -0.527       0.411
C(targtype1_txt)[T.Government (Diplomatic)]              -0.1835
0.228     -0.806      0.420      -0.630       0.263
C(targtype1_txt)[T.Government (General)]                 -0.7514
0.164     -4.575      0.000      -1.073      -0.429
C(targtype1_txt)[T.Journalists & Media]                  -0.4361
0.244     -1.785      0.074      -0.915       0.043
```

```
C(targtype1_txt)[T.Military]                                -1.5589
0.251     -6.222      0.000      -2.050      -1.068
C(targtype1_txt)[T.Police]                                  -1.5974
0.221     -7.239      0.000      -2.030      -1.165
C(targtype1_txt)[T.Private Citizens & Property]             -0.0620
0.146     -0.425      0.671      -0.348       0.224
C(targtype1_txt)[T.Religious Figures/Institutions]          -1.0646
0.354     -3.004      0.003      -1.759      -0.370
C(targtype1_txt)[T.Terrorists/Non-State Militia]            -0.1604
0.366     -0.438      0.661      -0.878       0.557
C(targtype1_txt)[T.Transportation]                          -1.1900
0.441     -2.701      0.007      -2.054      -0.326
C(targtype1_txt)[T.Unknown]                                 -0.2557
0.552     -0.463      0.643      -1.338       0.826
C(targtype1_txt)[T.Utilities]                               -1.8770
0.853     -2.200      0.028      -3.549      -0.205
C(targtype1_txt)[T.Violent Political Party]                 -1.5609
0.532     -2.934      0.003      -2.604      -0.518
C(weaptype1_txt)[T.Firearms]                                 2.4184
0.252      9.590      0.000       1.924       2.913
C(weaptype1_txt)[T.Incendiary]                              -0.9043
0.543     -1.665      0.096      -1.969       0.160
C(weaptype1_txt)[T.Melee]                                    2.6277
0.323      8.136      0.000       1.995       3.261
C(weaptype1_txt)[T.Unknown]                                  4.2544
0.262     16.229      0.000       3.741       4.768
nkill                                                       -0.9775
0.081    -12.051      0.000      -1.136      -0.819
success                                                      0.9945
0.167      5.943      0.000       0.667       1.323
nperps                                                       0.2168
0.017     13.090      0.000       0.184       0.249
extended                                                     6.4106
0.342     18.731      0.000       5.740       7.081
nwound                                                      -0.6570
0.086     -7.668      0.000      -0.825      -0.489
==============================================================================
==================================================
"""
```

Il valore di ****Pseudo R-squ. = 0.5845**** e ciò indica che il modello spiega bene la variazione nella probabilità di avere ostaggi. Dato che per la regressione logistica valori tra 0.2–0.4 sono già considerati buoni, il modello ha una buona capacità predittiva.

Come nel modello precedente analizziamo il valore dei p-value per dire con certezza che una variabile è statisticamente correlata ad un'altra. Si ricorda che:

- **p-value >= 0.05 ->** non si rifiuta l'ipotesi nulla, quindi la variabile non è statisticamente significativa."

- **p-value < 0.05 ->** si può rifiutare l'ipotesi nulla, la variabile è quindi statisticamente significativa;

Verifichiamo quindi il p-value delle variabili utilizzate:

- ****targtype1_txt**** presenta le categorie **'Government (General)'**, **'Military'**, **'Police'**, **'Religious Figures/Institutions'**, **'Transportation'**, **'Utilities**, **'Violent Political Party'** significative perchè p < 0.05 -> *il tipo di bersaglio influisce sulla probabilità che ci siano ostaggi*. Il resto delle categorie ossia **'Educational Institution'**, **'Government (Diplomatic)'**, **'Private Citizens & Property'**, **'Terrorists/Non-State Militia'**, **'Unknown'** non mostrano un effetto statisticamente valido.

- ****weaptype1_txt**** presenta le categorie **'Firearms'**, **'Melee'**, **'Unknown'** significative perchè p < 0.05 -> *l'arma utilizzata influisce sulla probabilità che ci siano ostaggi*. Solo la categorie **'Incendiary'** non mostra un effetto statisticamente valido.

- Le variabili **'nkill'**,**'nwound'**, **'nperps'**,**'success'** e **'extended'** sono tutte statisticamente significative, dato che hanno p-value = 0.

Passiamo ad analizzare gli **intervalli di confidenza**, entro cui ci si aspetta che il valore del coefficiente cada con una probabilità alta (solitamente 95%).

Analizziamo gli intervalli di confidenza per le variabili categoriche **targtype1_txt** e **weaptype1_txt**:

- **targtype1_txt ->** la maggior parte delle categorie ha intervalli di confidenza negativi, ciò indica un effetto anticorrelato con ishostkid(più è negativo e meno probabile è che ci siano ostaggi). Le categorie che non sono statisticamente significative, gli intervalli di valori sono vicini a zero e includono lo 0 stesso. Esempio: ***'Educational Institution'*** ha intervallo di confidenza = ***[-0.527 0.411]*** e coefficiente = **-0.0583**;
- **weaptype1_txt ->** le categorie hanno intervalli positivi e lontani dallo zero. La categoria ***'Incendiary'*** che è non significativa ha intervallo di confidenza = ***[-1.969 0.160]***.

Per rendere più interpretabili i coefficienti della regressione logistica, possiamo trasformarli in ***odds ratio(probabilità che accada una condizione/ probabilità che non accada)*** usando l'esponenziale dei coefficienti. Questo ci permette di capire di quanto gli odds di avere ostaggi cambiano quando una variabile aumenta di 1 unità, mantenendo costanti le altre.

Sottraendo 1 e moltiplicando per 100 otteniamo l'effetto percentuale sugli odds, che è più intuitivo da leggere rispetto ai log-odds.

```
oddsratio = np.exp(modelcleaned.params)
print(oddsratio.round(4))
print()
print()

print()
```

```python
print("Aumento degli odds in percentuale:")

for ind,perc in np.exp(modelcleaned.params).items():
    perc = (perc -1 )
    print(f"{ind:<50} : {perc:>9.2%}")
```

```
Intercept                                                0.0034
C(targtype1_txt)[T.Educational Institution]              0.9434
C(targtype1_txt)[T.Government (Diplomatic)]              0.8323
C(targtype1_txt)[T.Government (General)]                 0.4717
C(targtype1_txt)[T.Journalists & Media]                  0.6466
C(targtype1_txt)[T.Military]                             0.2104
C(targtype1_txt)[T.Police]                               0.2024
C(targtype1_txt)[T.Private Citizens & Property]          0.9399
C(targtype1_txt)[T.Religious Figures/Institutions]       0.3449
C(targtype1_txt)[T.Terrorists/Non-State Militia]         0.8518
C(targtype1_txt)[T.Transportation]                       0.3042
C(targtype1_txt)[T.Unknown]                              0.7744
C(targtype1_txt)[T.Utilities]                            0.1531
C(targtype1_txt)[T.Violent Political Party]              0.2099
C(weaptype1_txt)[T.Firearms]                            11.2278
C(weaptype1_txt)[T.Incendiary]                           0.4048
C(weaptype1_txt)[T.Melee]                               13.8417
C(weaptype1_txt)[T.Unknown]                             70.4174
nkill                                                    0.3762
success                                                  2.7034
nperps                                                   1.2420
extended                                               608.2503
nwound                                                   0.5184
dtype: float64



Aumento degli odds in percentuale:
Intercept                                          :   -99.66%
C(targtype1_txt)[T.Educational Institution]        :    -5.66%
C(targtype1_txt)[T.Government (Diplomatic)]        :   -16.77%
C(targtype1_txt)[T.Government (General)]           :   -52.83%
C(targtype1_txt)[T.Journalists & Media]            :   -35.34%
C(targtype1_txt)[T.Military]                       :   -78.96%
C(targtype1_txt)[T.Police]                         :   -79.76%
C(targtype1_txt)[T.Private Citizens & Property]    :    -6.01%
C(targtype1_txt)[T.Religious Figures/Institutions] :   -65.51%
C(targtype1_txt)[T.Terrorists/Non-State Militia]   :   -14.82%
C(targtype1_txt)[T.Transportation]                 :   -69.58%
C(targtype1_txt)[T.Unknown]                        :   -22.56%
C(targtype1_txt)[T.Utilities]                      :   -84.69%
C(targtype1_txt)[T.Violent Political Party]        :   -79.01%
C(weaptype1_txt)[T.Firearms]                       :  1022.78%
C(weaptype1_txt)[T.Incendiary]                     :   -59.52%
```

```
C(weaptype1_txt)[T.Melee]                                  :  1284.17%
C(weaptype1_txt)[T.Unknown]                                :  6941.74%
nkill                                                      :   -62.38%
success                                                    :   170.34%
nperps                                                     :    24.20%
extended                                                   : 60725.03%
nwound                                                     :   -48.16%
```

Infine, possiamo dire che:

- **targtype1_txt ->** la maggior parte dei bersagli diminuisce la probabilità di ostaggi, soprattutto le categorie ***"istituzionali"*** e ***militari***;
- **weaptype1_txt ->** Le ***armi da fuoco*** o ***corpo a corpo*** tendono a favorire ostaggi; armi incendiarie no;
- **nkill (-62%) e nwound (-48%) ->** più vittime o feriti riducono probabilità di ostaggi;
- **nperps(+24%) e success (+170%) ->** più autori e attacco riuscito aumantano la probabilità di ostaggi;
- **extended (>24h: +60725%) ->** un attacco aumenta moltissimo la probabilità di ostaggi.

5.3 Analisi statistica -'extended'

Una variabile che suscita curiosità sull'aspetto della letalità è **extended**. Nel Codebook GTD, extended è un indicatore binario che segnala se un attacco terroristico è prolungato nel tempo, cioè se dura più di 24 ore.

```
print(terrorism_cleaned['extended'].unique())
```

```
[0 1]
```

Poiché extended è una variabile binaria (0/1), la scelta metodologica più appropriata è la **regressione logistica**.

Osservazione Chiave: Questo approccio è coerente con l'obiettivo di comprendere quali fattori aumentano la complessità e la **letalità** di un attacco per poter intervenire tempestivamente.

Predittori scelti:

- **'attacktype1_txt'** -> alcune tattiche richiedono più tempo per essere portate a termine, altre invece sono istantanee;
- **'targtype1_txt'** -> il bersaglio può condizionare la dinamica temporale dell'attacco;
- **'weaptype1_txt'** -> il tipo d'arma può influenzare la durata dell'evento, implicando eventi brevi o il controllo prolungato;
- **'region_txt'**-> Alcune aree possono presentare conflitti prolungati, assedi o operazioni coordinate su più giorni;

- **'nkill'** -> Un numero elevato di vittime può essere indicativo di attacchi più complessi o prolungati;
- **'nwound'**-> Analogamente a nkill, un alto numero di feriti può riflettere attacchi più intensi o protratti nel tempo;
- **'ishostkid'** -> La presa di ostaggi potrebbe essere tipica degli attacchi prolungati, poiché implica negoziazioni, barricamenti o situazioni di stallo;
- **'multiple'**-> attacchi multipli possono richiedere inevitabilmente più tempo per essere completati o gestiti;
- **'success'** -> un attacco riuscito potrebbe richiedere sia più tempo che completarsi immediatamente;

Procediamo introducendo nel nostro modello il predittore *ishostkid*:

```
categorical_vars = ['attacktype1_txt', 'targtype1_txt',
'weaptype1_txt', 'region_txt']
numeric_vars = ['nkill', 'nwound', 'ishostkid','multiple','success']


dataforlogit2 = terrorism_cleaned[categorical_vars + numeric_vars +
['extended']].copy()


threshold = 1000

for var in categorical_vars:
    counts = dataforlogit2[var].value_counts()
    valid_levels = counts[counts >= threshold].index
    dataforlogit2 =
dataforlogit2[dataforlogit2[var].isin(valid_levels)]


dataforlogit2 = tocategory(dataforlogit2, categorical_vars)
dataforlogit2 = toint(dataforlogit2, numeric_vars)


model = logit('extended ~ ishostkid', data=dataforlogit2).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.044510
         Iterations 11

<class 'statsmodels.iolib.summary.Summary'>
"""
                           Logit Regression Results

==============================================================================
========
Dep. Variable:               extended   No. Observations:
115892
```

```
Model:                           Logit   Df Residuals:
115890
Method:                            MLE   Df Model:
1
Date:                Thu, 22 Jan 2026   Pseudo R-squ.:
0.6902
Time:                        09:47:38   Log-Likelihood:
-5158.3
converged:                       True   LL-Null:
-16653.
Covariance Type:            nonrobust   LLR p-value:
0.000
==================================================================
========
                 coef    std err          z      P>|z|      [0.025
0.975]
------------------------------------------------------------------
--------
Intercept     -7.0590      0.103    -68.411      0.000      -7.261
-6.857
ishostkid      7.3481      0.106     69.183      0.000       7.140
7.556
==================================================================
========
"""
```

Il valore di Pseudo R-squared indica una **buona capacità esplicativa del modello (69%)**; il predittore **ishostkid è statisticamente significativo** (p-value < 0.05).

Procederemo ora inserendo tutti gli altri predittori nel modello per valutarne l'effetto.

```
model = logit('extended ~ attacktype1_txt + targtype1_txt +
weaptype1_txt + region_txt + nkill + nwound + multiple + ishostkid +
success', data=dataforlogit2).fit()
model.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.042651
         Iterations 11

<class 'statsmodels.iolib.summary.Summary'>
"""
                           Logit Regression Results

==================================================================
========
Dep. Variable:               extended   No. Observations:
115892
Model:                           Logit   Df Residuals:
115856
```

```
Method:                           MLE   Df Model:
35
Date:                Thu, 22 Jan 2026   Pseudo R-squ.:
0.7032
Time:                        09:47:40   Log-Likelihood:
-4942.9
converged:                       True   LL-Null:
-16653.
Covariance Type:            nonrobust   LLR p-value:
0.000
==============================================================================
=================================================
                                                        coef    std
err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------------------
-------------------------------------------------
Intercept                                              -9.1543
0.365    -25.063      0.000      -9.870      -8.438
attacktype1_txt[T.Assassination]                       -0.2913
0.177     -1.644      0.100      -0.639       0.056
attacktype1_txt[T.Bombing/Explosion]                   -0.4439
0.279     -1.590      0.112      -0.991       0.103
attacktype1_txt[T.Facility/Infrastructure Attack]       0.3315
0.298      1.114      0.265      -0.252       0.915
attacktype1_txt[T.Hostage Taking (Kidnapping)]          0.6134
0.115      5.323      0.000       0.388       0.839
attacktype1_txt[T.Unknown]                             -0.5037
0.341     -1.479      0.139      -1.171       0.164
targtype1_txt[T.Educational Institution]               -0.1950
0.148     -1.316      0.188      -0.486       0.096
targtype1_txt[T.Government (Diplomatic)]                0.0037
0.148      0.025      0.980      -0.287       0.294
targtype1_txt[T.Government (General)]                   0.0134
0.090      0.150      0.881      -0.162       0.189
targtype1_txt[T.Journalists & Media]                    0.4869
0.135      3.597      0.000       0.222       0.752
targtype1_txt[T.Military]                              -0.1421
0.125     -1.132      0.257      -0.388       0.104
targtype1_txt[T.Police]                                -0.1483
0.118     -1.255      0.210      -0.380       0.083
targtype1_txt[T.Private Citizens & Property]           -0.2522
0.080     -3.151      0.002      -0.409      -0.095
targtype1_txt[T.Religious Figures/Institutions]        -0.0067
0.170     -0.039      0.969      -0.339       0.326
targtype1_txt[T.Terrorists/Non-State Militia]          -0.2835
0.208     -1.364      0.173      -0.691       0.124
targtype1_txt[T.Transportation]                        -1.0192
0.271     -3.756      0.000      -1.551      -0.487
targtype1_txt[T.Unknown]                               -0.3776
```

```
0.445     -0.849      0.396      -1.249       0.494
targtype1_txt[T.Utilities]                          -0.0053
0.368     -0.015      0.988      -0.726       0.716
targtype1_txt[T.Violent Political Party]            -0.0927
0.247     -0.375      0.708      -0.577       0.392
weaptype1_txt[T.Firearms]                            0.2281
0.224      1.016      0.309      -0.212       0.668
weaptype1_txt[T.Incendiary]                         -0.0450
0.335     -0.134      0.893      -0.702       0.612
weaptype1_txt[T.Melee]                              -0.3001
0.245     -1.223      0.221      -0.781       0.181
weaptype1_txt[T.Unknown]                             0.4901
0.227      2.157      0.031       0.045       0.936
region_txt[T.Eastern Europe]                         1.0960
0.205      5.348      0.000       0.694       1.498
region_txt[T.Middle East & North Africa]             0.9683
0.136      7.139      0.000       0.702       1.234
region_txt[T.North America]                          0.1035
0.246      0.421      0.674      -0.379       0.586
region_txt[T.South America]                          0.5249
0.137      3.838      0.000       0.257       0.793
region_txt[T.South Asia]                             1.1205
0.132      8.502      0.000       0.862       1.379
region_txt[T.Southeast Asia]                         1.0194
0.155      6.596      0.000       0.717       1.322
region_txt[T.Sub-Saharan Africa]                     1.1286
0.144      7.824      0.000       0.846       1.411
region_txt[T.Western Europe]                         0.6269
0.182      3.438      0.001       0.269       0.984
nkill                                               -0.0422
0.026     -1.615      0.106      -0.093       0.009
nwound                                               0.0071
0.037      0.192      0.848      -0.065       0.079
multiple                                            -0.1136
0.111     -1.020      0.308      -0.332       0.105
ishostkid                                            6.2761
0.149     42.138      0.000       5.984       6.568
success                                              1.5854
0.232      6.831      0.000       1.131       2.040
==============================================================================
==============================================
"""
```

Il modello logit ha un **Pseudo R^2 = 0.7032**, quindi spiega molto bene la probabilità che un attacco sia extended (>24 ore).

Molte variabili hanno un effetto chiaro e interpretabile sulla probabilità che l'attacco duri più di 24 ore. Ma per adesso continuiamo la modellazione.

Variabili **NON significative** ($p > 0.05$):

- Molte categorie di attacktype1_txt
- Molte categorie di targtype1_txt
- Quasi tutte le categorie di weaptype1_txt
- region_txt[T.North America]
- nkill
- nwound
- multiple

Alcune categorie delle variabili fittizie (dummy variable) non risultano statisticamente significative. Tuttavia, poiché fanno parte di variabili categoriali con più livelli, non le elimineremo singolarmente. Applicheremo la **backward elimination** solo alle variabili numeriche/intere, non ai singoli livelli delle dummy.

```
modelcleaned = logit('extended ~ attacktype1_txt + targtype1_txt +
weaptype1_txt + region_txt + ishostkid + success',
data=dataforlogit2).fit()
modelcleaned.summary()
```

```
Optimization terminated successfully.
         Current function value: 0.042666
         Iterations 11

<class 'statsmodels.iolib.summary.Summary'>
"""
                           Logit Regression Results

==================================================================
========
Dep. Variable:               extended   No. Observations:
115892
Model:                          Logit   Df Residuals:
115859
Method:                           MLE   Df Model:
32
Date:                Thu, 22 Jan 2026   Pseudo R-squ.:
0.7031
Time:                        09:47:41   Log-Likelihood:
-4944.7
converged:                       True   LL-Null:
-16653.
Covariance Type:            nonrobust   LLR p-value:
0.000
==================================================================
==================================================
                                                        coef    std
err          z      P>|z|      [0.025      0.975]
------------------------------------------------------------------
------------------------------------------------
Intercept                                            -9.2050
```

```
0.360    -25.560      0.000      -9.911      -8.499
attacktype1_txt[T.Assassination]                      -0.2885
0.176     -1.635      0.102      -0.634       0.057
attacktype1_txt[T.Bombing/Explosion]                  -0.3992
0.278     -1.438      0.150      -0.943       0.145
attacktype1_txt[T.Facility/Infrastructure Attack]      0.3678
0.296      1.244      0.213      -0.212       0.947
attacktype1_txt[T.Hostage Taking (Kidnapping)]         0.6418
0.113      5.683      0.000       0.420       0.863
attacktype1_txt[T.Unknown]                            -0.5186
0.340     -1.526      0.127      -1.185       0.147
targtype1_txt[T.Educational Institution]              -0.1958
0.148     -1.323      0.186      -0.486       0.094
targtype1_txt[T.Government (Diplomatic)]               0.0130
0.148      0.088      0.930      -0.277       0.303
targtype1_txt[T.Government (General)]                  0.0124
0.090      0.138      0.890      -0.163       0.188
targtype1_txt[T.Journalists & Media]                   0.4934
0.135      3.647      0.000       0.228       0.759
targtype1_txt[T.Military]                             -0.1687
0.124     -1.355      0.175      -0.413       0.075
targtype1_txt[T.Police]                               -0.1770
0.117     -1.511      0.131      -0.407       0.053
targtype1_txt[T.Private Citizens & Property]          -0.2666
0.080     -3.351      0.001      -0.423      -0.111
targtype1_txt[T.Religious Figures/Institutions]       -0.0041
0.170     -0.024      0.981      -0.336       0.328
targtype1_txt[T.Terrorists/Non-State Militia]         -0.3104
0.207     -1.499      0.134      -0.716       0.095
targtype1_txt[T.Transportation]                       -1.0141
0.271     -3.738      0.000      -1.546      -0.482
targtype1_txt[T.Unknown]                              -0.4028
0.444     -0.908      0.364      -1.272       0.467
targtype1_txt[T.Utilities]                            -0.0097
0.365     -0.027      0.979      -0.726       0.706
targtype1_txt[T.Violent Political Party]              -0.1008
0.247     -0.407      0.684      -0.586       0.384
weaptype1_txt[T.Firearms]                              0.2491
0.223      1.119      0.263      -0.187       0.685
weaptype1_txt[T.Incendiary]                           -0.0403
0.334     -0.121      0.904      -0.695       0.614
weaptype1_txt[T.Melee]                                -0.2999
0.244     -1.231      0.218      -0.777       0.178
weaptype1_txt[T.Unknown]                               0.5198
0.225      2.313      0.021       0.079       0.960
region_txt[T.Eastern Europe]                           1.1102
0.205      5.427      0.000       0.709       1.511
region_txt[T.Middle East & North Africa]               0.9615
0.136      7.094      0.000       0.696       1.227
```

```
region_txt[T.North America]                                0.1084
0.246      0.441      0.659      -0.374       0.590
region_txt[T.South America]                                0.5284
0.137      3.867      0.000       0.261       0.796
region_txt[T.South Asia]                                   1.1140
0.132      8.458      0.000       0.856       1.372
region_txt[T.Southeast Asia]                               1.0230
0.154      6.632      0.000       0.721       1.325
region_txt[T.Sub-Saharan Africa]                           1.1205
0.143      7.819      0.000       0.840       1.401
region_txt[T.Western Europe]                               0.6399
0.182      3.512      0.000       0.283       0.997
ishostkid                                                  6.2696
0.149     42.172      0.000       5.978       6.561
success                                                    1.5736
0.232      6.783      0.000       1.119       2.028
==============================================================================
"""
```

Bontà del modello

Il modello di regressione logistica presenta un **Pseudo R^2 = 0.7031**, un valore molto elevato per un modello logit, indicando che **i predittori** inclusi **spiegano una quota sostanziale** della variabilità nella probabilità che un **attacco terroristico** sia **extended** (durata > 24 ore). Il **test di verosimiglianza** (LLR p-value < 0.001) conferma la **significatività complessiva del modello**.

Predittori significativi (p < 0.05):

- attacktype1_txt[T.Hostage Taking (Kidnapping)]
- targtype1_txt[T.Journalists & Media]
- targtype1_txt[T.Private Citizens & Property]
- targtype1_txt[T.Transportation]
- weaptype1_txt[T.Unknown]
- region_txt (tutte le regioni tranne North America)
- ishostkid
- success

Gli **intervalli di confidenza al 95%** confermano la **significatività dei predittori principali**:

- Per i ***coefficienti significativi***, l'intervallo **non include lo zero** (come **ishostkid** avente l'intervallo *[5.978 6.561]* oppure **success** con l'intervallo *[1.119 2.028]*);
- Per i **coefficienti non significativi**, l'intervallo **include lo zero**, indicando che l'effetto non è distinguibile dalla baseline (come per **targtype1_txt[T.Government (Diplomatic)]** avente l'intervallo *[-0.277 0.303]*).

Interpretiamo adesso i coefficienti tramite la funzione *exp* che calcola le **Odds Ratio**.

```python
oddsratio = np.exp(modelcleaned.params)
print(oddsratio.round(4))
print()
print()

print("Aumento degli odds in percentuale:")
for ind,perc in np.exp(modelcleaned.params).items():
    perc = (perc -1 )
    print(f"{ind:<50} : {perc:>9.2%}")
```

```
Intercept                                                0.0001
attacktype1_txt[T.Assassination]                         0.7493
attacktype1_txt[T.Bombing/Explosion]                     0.6708
attacktype1_txt[T.Facility/Infrastructure Attack]        1.4445
attacktype1_txt[T.Hostage Taking (Kidnapping)]           1.8999
attacktype1_txt[T.Unknown]                               0.5953
targtype1_txt[T.Educational Institution]                 0.8222
targtype1_txt[T.Government (Diplomatic)]                 1.0131
targtype1_txt[T.Government (General)]                    1.0124
targtype1_txt[T.Journalists & Media]                     1.6379
targtype1_txt[T.Military]                                0.8447
targtype1_txt[T.Police]                                  0.8378
targtype1_txt[T.Private Citizens & Property]             0.7660
targtype1_txt[T.Religious Figures/Institutions]          0.9959
targtype1_txt[T.Terrorists/Non-State Militia]            0.7331
targtype1_txt[T.Transportation]                          0.3627
targtype1_txt[T.Unknown]                                 0.6684
targtype1_txt[T.Utilities]                               0.9904
targtype1_txt[T.Violent Political Party]                 0.9041
weaptype1_txt[T.Firearms]                                1.2829
weaptype1_txt[T.Incendiary]                              0.9605
weaptype1_txt[T.Melee]                                   0.7409
weaptype1_txt[T.Unknown]                                 1.6817
region_txt[T.Eastern Europe]                             3.0348
region_txt[T.Middle East & North Africa]                 2.6157
region_txt[T.North America]                              1.1145
region_txt[T.South America]                              1.6962
region_txt[T.South Asia]                                 3.0465
region_txt[T.Southeast Asia]                             2.7816
region_txt[T.Sub-Saharan Africa]                         3.0664
region_txt[T.Western Europe]                             1.8963
ishostkid                                              528.2609
success                                                  4.8238
dtype: float64


Aumento degli odds in percentuale:
Intercept                                          :   -99.99%
attacktype1_txt[T.Assassination]                   :   -25.07%
attacktype1_txt[T.Bombing/Explosion]               :   -32.92%
```

```
attacktype1_txt[T.Facility/Infrastructure Attack]  :    44.45%
attacktype1_txt[T.Hostage Taking (Kidnapping)]     :    89.99%
attacktype1_txt[T.Unknown]                         :   -40.47%
targtype1_txt[T.Educational Institution]           :   -17.78%
targtype1_txt[T.Government (Diplomatic)]           :     1.31%
targtype1_txt[T.Government (General)]              :     1.24%
targtype1_txt[T.Journalists & Media]               :    63.79%
targtype1_txt[T.Military]                          :   -15.53%
targtype1_txt[T.Police]                            :   -16.22%
targtype1_txt[T.Private Citizens & Property]       :   -23.40%
targtype1_txt[T.Religious Figures/Institutions]    :    -0.41%
targtype1_txt[T.Terrorists/Non-State Militia]      :   -26.69%
targtype1_txt[T.Transportation]                    :   -63.73%
targtype1_txt[T.Unknown]                           :   -33.16%
targtype1_txt[T.Utilities]                         :    -0.96%
targtype1_txt[T.Violent Political Party]           :    -9.59%
weaptype1_txt[T.Firearms]                          :    28.29%
weaptype1_txt[T.Incendiary]                        :    -3.95%
weaptype1_txt[T.Melee]                             :   -25.91%
weaptype1_txt[T.Unknown]                           :    68.17%
region_txt[T.Eastern Europe]                       :   203.48%
region_txt[T.Middle East & North Africa]           :   161.57%
region_txt[T.North America]                        :    11.45%
region_txt[T.South America]                        :    69.62%
region_txt[T.South Asia]                           :   204.65%
region_txt[T.Southeast Asia]                       :   178.16%
region_txt[T.Sub-Saharan Africa]                   :   206.64%
region_txt[T.Western Europe]                       :    89.63%
ishostkid                                          : 52726.09%
success                                            :   382.38%
```

Interpretazione delle Odds Ratio del modello Logit su extended

Procediamo prima con le **odds ratio > 1**:

- ***intercetta = 0.000101 ->*** Quando tutte le variabili indipendenti sono pari a zero, le odds - probabilità che accada sulle probabilità che non accada che l'attacco sia extended - sono **quasi nulle**. Questo rappresenta la **baseline** di un attacco prolungato in assenza di fattori esplicativi;

- Un aumento di un'unità nel valore di ***ishostkid*** corrisponde a un aumento di circa $e^{6.2696} \approx 528.26$ nelle odds (il 52.700%), mantenendo costanti le altre variabili;

- Un aumento di un'unità nel valore di ***success*** corrisponde a un aumento di circa $e^{1.5736} \approx 4.82$ nelle odds (il 382%), mantenendo costanti le altre variabili;

- ***Tipo di attacco: rapimento di ostaggi ->*** Un aumento di un'unità nella categoria **Hostage Taking (Kidnapping)** corrisponde a un aumento di circa $e^{0.6418} \approx 1.90$ nelle

odds **(il 90%)**, mantenendo costanti le altre variabili. I rapimenti di ostaggi hanno odds quasi doppie di essere prolungati;

- ***Tipo di attacco: Attacchi contro infrastrutture ->*** Un aumento di un'unità nella categoria **Facility/Infrastructure Attack** corrisponde a un aumento di circa $e^{0.6418} \approx 1.44$ nelle odds **(il 44%)**, mantenendo costanti le altre variabili. Attacchi contro infrastrutture tendono ad avere una durata maggiore;

- ***Tipo di bersaglio: media e giornalisti ->*** Un aumento di un'unità nella categoria **Journalists & Media** corrisponde a un aumento di circa $e^{0.4934} \approx 1.64$ nelle odds **(il 64%)**, mantenendo costanti le altre variabili. Attacchi contro media e giornalisti sono associati a eventi più lunghi;

- ***Tipo di arma: armi da fuoco ->*** le odds di **Firearms** sono di **1.28 (28%)**. L'uso di armi da fuoco è associato a una maggiore probabilità di attacco prolungato;

- ***Tipo di arma: sconosciute ->*** le odds sulle **armi Unknown** sono di circa **1.68 (il 68%)**. Quando il tipo di arma è sconosciuto, le odds aumentano significativamente;

- ***Regione geografica***
  - **South Asia** = 3.05 → +205%
  - **Sub-Saharan Africa** = 3.07 → +207%
  - **Eastern Europe** = 3.03 → +203%
  - **Southeast Asia** = 2.78 → +178%
  - **Middle East & North Africa** = 2.61 → +161%
  - **South America** = 1.70 → +70%
  - **Western Europe** = 1.89 → +89%
  - **North America** = 1.11 → +11%

Concludendo con le **odds ratio < 0**:

- ***Tipo di attacco***
  - **Assassination** = 0.75 → −25%
  - **Bombing/Explosion** = 0.67 → −33%
  - **Unknown** = 0.59 → −41%
- ***Tipo di bersaglio***
  - **Educational Institution** = 0.82 → −18%
  - **Military** = 0.84 → −16%
  - **Police** = 0.84 → −16%
  - **Private Citizens & Property** = 0.76 → −24%
  - **Religious Figures/Institutions** = 0.99 → ≈0%
  - **Terrorists/Non-State Militia** = 0.73 → −27%
  - **Transportation** = 0.36 → −64%
  - **Unknown** = 0.67 → −33%
  - **Utilities** = 0.99 → ≈0%
  - **Violent Political Party** = 0.90 → −10%

- ***Tipo di arma***
  - **Incendiary** = 0.96 → −4%
  - **Melee** = 0.74 → −26%

Sintesi generale

- ***Gli attacchi extended ->*** Gli attacchi extended sono influenzati dal **rapimento di ostaggi**, dal **successo operativo**, dagli **attacchi contro i media** e dal **contesto geografico**. Questi fattori sono fortemente associati a operazioni complesse, negoziazioni o contesti geopolitici che favoriscono eventi prolungati;

- ***Gli attacchi non extended ->*** Attacchi rapidi come **esplosivi**, **assassinii**, **corpo a corpo** e bersagli come **trasporti**, **militari/polizia** riducono drasticamente la probabilità di durata oltre le 24 ore.

FASE 6: Analisi Predittiva e Machine Learning

L'*analisi predittiva* condotta in questa sezione si basa sull'utilizzo di *strumenti di classificazione*, che permettono di rispondere a domande come chi/come.

Una volta che un attacco è in atto, questi modelli consentono di classificare la **letalità dalle vittime**, il **tipo di attacco** e/o il **gruppo terroristico più compatibile**, fornendo indicazioni utili per comprendere le modalità operative adottate e per stimare come l'evento potrebbe evolversi e quali siano le possibili richieste o obiettivi degli attaccanti.

Non si cerca di prevedere in senso stretto eventuali attacchi futuri(dato che sono influenzati da dinamiche, politiche sociali e/o religiose), ma invece è quello di analizzare l'evoluzione operativa dell'evento terroristico, una volta iniziato e stimarne i possibili esiti.

Inoltre, i modelli sviluppati possono essere utilizzati per una **rianalisi del dataset** nei casi in cui il gruppo terroristico sia classificato come **"Unknown"**.

In tali situazioni, i classificatori forniscono una stima probabilistica del gruppo più compatibile con le caratteristiche dell'attacco, permettendo:

- arricchire le informazioni disponibili;
- approfondire la comprensione delle dinamiche operative.

Tutto ciò deve essere condotto con la coscienza che tali risultati non implicano l'assegnazione di responsabilità certe, poiché l'analisi è di natura esclusivamente statistica e non può sostituire un'analisi basata su fatti reali.

6.1 Classificazione Predittiva -'nkill'

Un **modello predittivo di letalità** in uno **scenario applicativo** può essere utilizzato per:

- supportare organi decisionali nella valutazione del rischio;
- identificare pattern ricorrenti associati ad attacchi più pericolosi (un attacco con almeno una vittima richiede un intervento immediato);

- proporre di allocare risorse in modo più efficiente;
- migliorare la capacità di risposta preventiva.

Quello che stiamo cercando è assicurarci di **classificare bene quale attacco abbia letalità** (**alta recall**), Un'approccio che **appiattisce i dati**, ma consapevole di **ottenere gran parte dell'informazione**.

L'obiettivo di questa fase è costruire un modello di **classificazione binaria** in grado di predire se un attacco terroristico sarà **letale** oppure **non letale**, utilizzando il precedente *problema di regressione lineare nkill*, modellato e analizzato statisticamente nel paragrafo 5.1.

Definiamo la *variabile target*:

- $\mathbf{y = 1} \rightarrow$ attacco letale (nkill > 0)
- $\mathbf{y = 0} \rightarrow$ attacco non letale (nkill = 0)

Questa formulazione ci consente di trasformare il *problema di regressione nkill* in un **problema di classificazione** più *stabile* e più *utile* dal punto di vista operativo.

Le *variabili predittive* considerate sono le stesse utilizzate nel modello di regressione lineare (nkill):

- **'weaptype1_txt'**;
- **'attacktype1_txt'**;
- **'region_txt'**;
- **'targtype1_txt'**;
- **'nwound'**;
- **'success'**;
- **'suicide'**.

Iniziamo costruendo un modello discriminativo, la **regressione logistica**. Le funzioni che utilizzeremo sono le seguenti: StandardScaler per variabili numeriche, OneHotEncoder per variabili categoriche, ColumnTransformer per unire tutto ed Train/Test split (70/30).

```
#predittori numerici e categorali
colbincl =
['weaptype1_txt','attacktype1_txt','region_txt','targtype1_txt','nwoun
d','success','suicide']
numeric_col = ['nwound','success','suicide']
categorical_col =
['weaptype1_txt','attacktype1_txt','region_txt','targtype1_txt']

# Creazione della variabile binaria: 1 = letale, 0 = non letale
terrorism_cleaned['letalità'] = (terrorism_cleaned['nkill'] >
0).astype(int)


#1. carichiamo i dati
y = terrorism_cleaned['letalità']
```

```
X = terrorism_cleaned[colbincl].copy()

print(f"Features shape: {X.shape}")
print(f"Target shape: {y.shape}")
print(f"Classes: 0 = {'non letale'}, 1 = {'letale'}")

#setup sperimentale
preproc = ColumnTransformer([
    ("num", StandardScaler(), numeric_col),
    ("cat", OneHotEncoder(handle_unknown='ignore'), categorical_col)
])

# teniamo da parte il 30% per il test finale
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42, stratify=y
)

print()
print(f"Grandezza dati di train {len(X_train)}")
print(f"Grandezza dati di test {len(X_test)}")
```

```
Features shape: (122359, 7)
Target shape: (122359,)
Classes: 0 = non letale, 1 = letale

Grandezza dati di train 85651
Grandezza dati di test 36708
```

Creiamo una pipeline che scala i dati adattando il modello di regressione logistica. Non specificando la *penalty* avremo una regressione logistica automaticamente **regolarizzata con Ridge L2**.

Ricordiamo che l'iperparametro C è l'inverso della *forza di regolarizzazione* λ, quindi un **C basso** indicherà una **regolarizzazione forte (alto bias, modello semplice)** e un **C alto** indicherà una **regolarizzazione debole (alta varianza, modello complesso)**.

Ci serviremo della **Ricerca a Griglia** per trovare il migliore valore per C.

```
# creiamo la pipeline per la regressione logistica con
regolarizzazione Ridge
pipelog = Pipeline([
    ("preprocess", preproc),
    ("clf", LogisticRegression(
        solver='lbfgs',
        max_iter=1000,
    ))
])

# definiamo una griglia di ricerca del parametro C ottimale
param_gridlog = {"clf__C": np.logspace(-1, 1, 5) # [0.001, 0.01, 0.1,
```

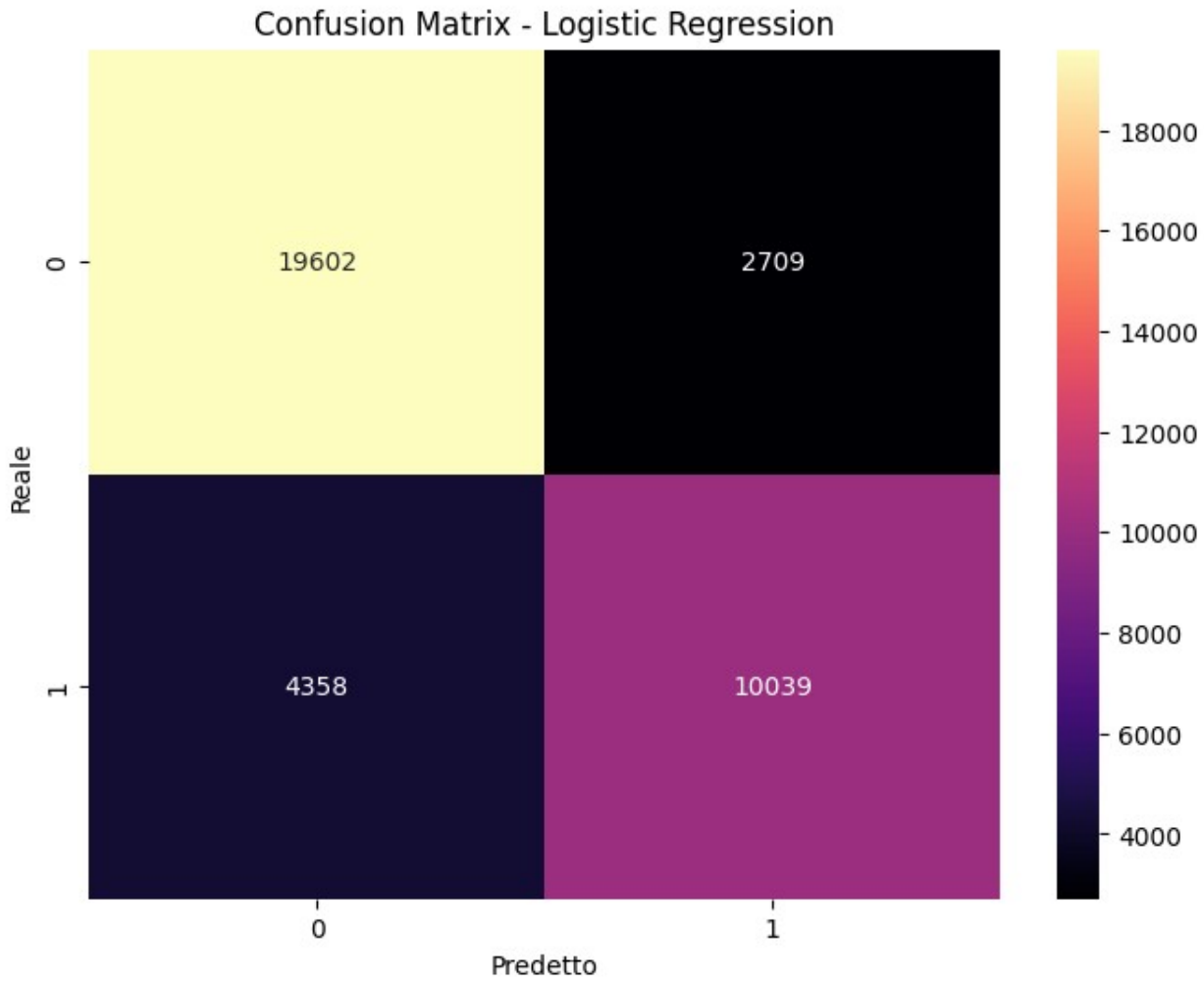
```
1, 10, 100, 1000]
}


# eseguiamo la ricerca del miglior parametro C con CV
grid_lr = GridSearchCV(
    pipelog, param_gridlog,
    cv=3, scoring="accuracy", n_jobs=-1
)
grid_lr.fit(X_train, y_train)

print(f"Miglior parametro C trovato in CV:
{grid_lr.best_params_['clf__C']}")
print(f"Miglior score di accuratezza in CV:
{grid_lr.best_score_:.4f}")
```

```
Miglior parametro C trovato in CV: 0.31622776601683794
Miglior score di accuratezza in CV: 0.8082
```

Una volta trovato il miglior C e riaddestrato il training set, il modello va valutato sul test set per ottenere la prestazione finale.

```
# prediciamo sul test set del miglior modello trovato
y_pred_lr = grid_lr.predict(X_test)

print(f"--- Valutazione Finale del Modello (Test Set) ---")
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print(classification_report(y_test, y_pred_lr, target_names=['non
letale', 'letale']))

cm = confusion_matrix(y_test, y_pred_lr)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="magma")
plt.xlabel("Predetto")
plt.ylabel("Reale")
plt.title("Confusion Matrix - Logistic Regression")
plt.show()
```

```
--- Valutazione Finale del Modello (Test Set) ---
Accuracy: 0.807480658167157
              precision    recall  f1-score   support

  non letale       0.82      0.88      0.85     22311
      letale       0.79      0.70      0.74     14397

    accuracy                           0.81     36708
   macro avg       0.80      0.79      0.79     36708
weighted avg       0.81      0.81      0.81     36708
```



Il modello discrimina bene tra attacchi letali e non letali, con un **buon equilibrio tra recall (0.70) e precision (0.79)**. In oltre è molto *affidabile*, con una **accuratezza pari circa 81%**!

Tuttavia vogliamo trovare un modello che minimizzi al meglio la mancanza di attacchi letali, ovvero alto recall (falsi negativi). A Tal proposito, passando ad un modello generativo, optiamo per il **Gaussian Naive Bayes**.

```
# preprocessing per Gaussian Naive Bayes con matrice densa e non
sparsa
GNBpreproc = ColumnTransformer([
    ("num", StandardScaler(), numeric_col),
    ("cat", OneHotEncoder(handle_unknown='ignore',
sparse_output=False), categorical_col)
])

# Pipeline con preprocessing + Gaussian Naive Bayes
pipe_gnb = Pipeline([
    ("preprocess", GNBpreproc),
    ("clf", GaussianNB())
```

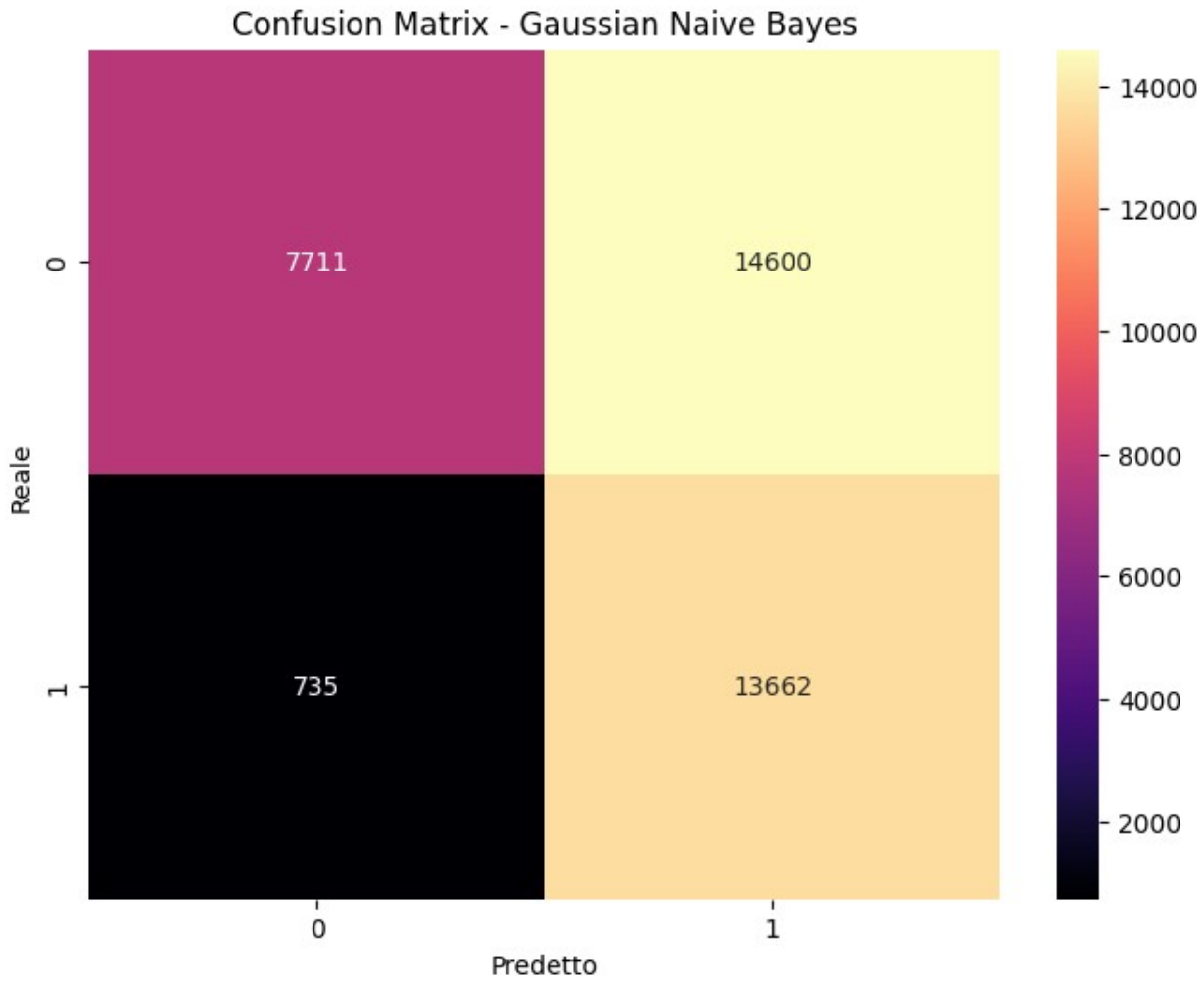
```
])

# GridSearchCV senza parametri da ottimizzare
param_grid_gnb = {}

grid_gnb = GridSearchCV(
    pipe_gnb,
    param_grid_gnb,
    cv=3,
    scoring="accuracy",
    n_jobs=-1
)

# Fit
grid_gnb.fit(X_train, y_train)

print("Miglior score in CV:", grid_gnb.best_score_)
```

```
Miglior score in CV: 0.5729995796381453
```

Verifichiamo il comportamento dei nuovi dati sul test set del miglior modello GNB sulla letalità:

```
# Predizione sul test set
y_pred_gnb = grid_gnb.predict(X_test)

print("\nAccuracy:", accuracy_score(y_test, y_pred_gnb))
print(classification_report(y_test, y_pred_gnb, target_names=['non
letale', 'letale']))

# Confusion matrix
cm = confusion_matrix(y_test, y_pred_gnb)
plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="magma")
plt.xlabel("Predetto")
plt.ylabel("Reale")
plt.title("Confusion Matrix - Gaussian Naive Bayes")
plt.show()
```

```
Accuracy: 0.5822436526097853
              precision    recall  f1-score   support

  non letale       0.91      0.35      0.50     22311
      letale       0.48      0.95      0.64     14397

    accuracy                           0.58     36708
   macro avg       0.70      0.65      0.57     36708
weighted avg       0.74      0.58      0.56     36708
```



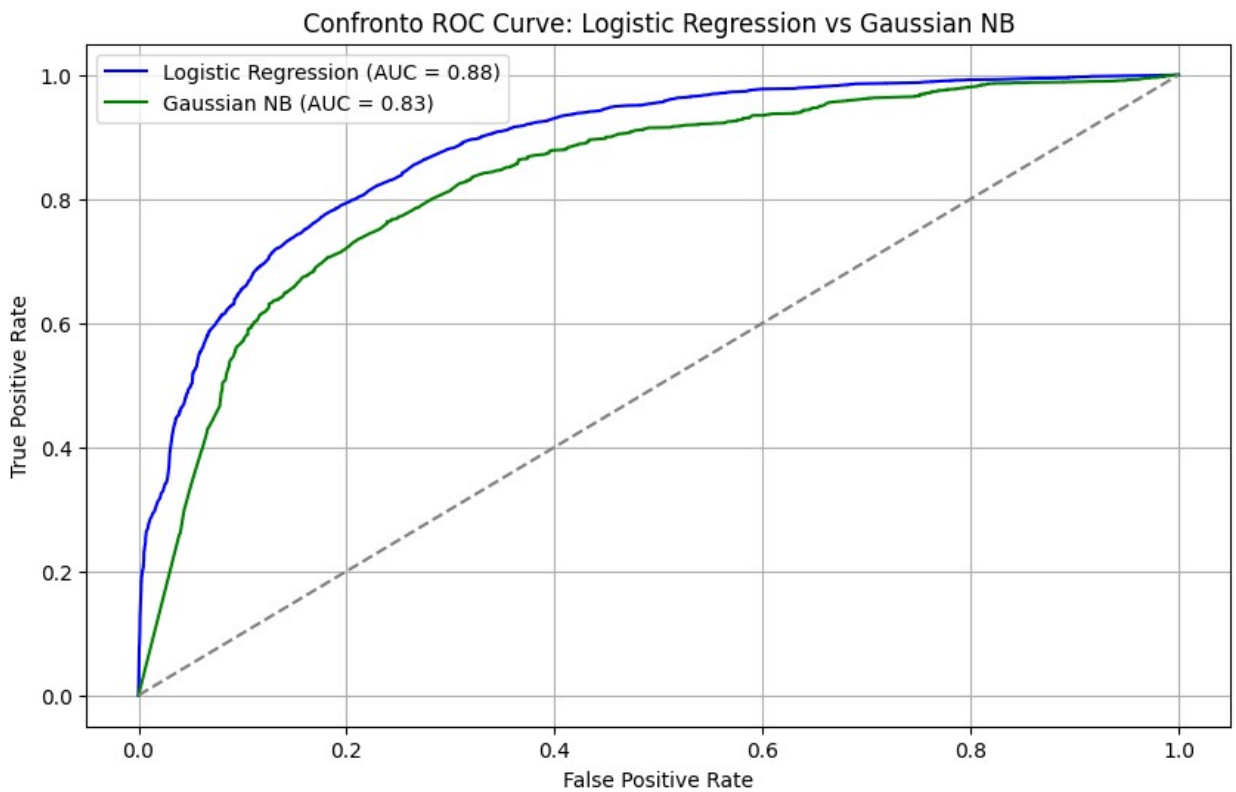
Il modello GNB mostra un comportamento fortemente orientato alla cattura della classe letale (quello che cercavamo), con un **recall del 95%** ma una **precision del 48%**. Questo **tradeoff** lo rende utile in scenari dove è prioritario non perdere attacchi con vittime, anche a costo di generare allarmi non necessari.

Tuttavia, la **bassa accuratezza (del 58%)** e la scarsa capacità di discriminare la classe non letale ne **limitano l'applicabilità operativa**, facendo intuire che il **modello di regressione logistica** sia **più stabile** e **utile** nello scenario applicativo.

Confrontiamo le curve Roc tra quest'ultimo e il miglior modello di regressione logistica per confermare quale sia il modello migliore.

```
# Probabilità predette (classe 1) dai due modelli
probs_lr = grid_lr.predict_proba(X_test)[:, 1]
probs_gnb = grid_gnb.predict_proba(X_test)[:, 1]

# ROC curve
fpr_lr, tpr_lr, _ = roc_curve(y_test, probs_lr)
fpr_gnb, tpr_gnb, _ = roc_curve(y_test, probs_gnb)
```

```
# AUC
auc_lr = roc_auc_score(y_test, probs_lr)
auc_gnb = roc_auc_score(y_test, probs_gnb)

# Plot
plt.figure(figsize=(10, 6))
plt.plot(fpr_lr, tpr_lr, label=f"Logistic Regression (AUC =
{auc_lr:.2f})", color="blue")
plt.plot(fpr_gnb, tpr_gnb, label=f"Gaussian NB (AUC = {auc_gnb:.2f})",
color="green")
plt.plot([0, 1], [0, 1], linestyle="--", color="gray")
plt.xlabel("False Positive Rate")
plt.ylabel("True Positive Rate")
plt.title("Confronto ROC Curve: Logistic Regression vs Gaussian NB")
plt.legend()
plt.grid(True)
plt.show()
```



Nel contesto applicativo in cui è **prioritario intercettare attacchi con vittime**, il modello **Gaussian Naive Bayes** si distingue per **l'elevatissimo recall** sulla classe letale, pur sacrificando precisione e accuratezza.

Tuttavia, la **curva ROC evidenzia che la Logistic Regression** mantiene un **miglior equilibrio globale tra sensibilità e specificità**, con un **AUC superiore**. In scenari operativi dove è cruciale

ridurre falsi negativi ma anche contenere falsi allarmi, la **Logistic Regression risulta più adatta** a supportare decisioni affidabili e sostenibili.

6.2 Classificazione Predittiva -'attacktype1_txt'

In questo paragrafo, proviamo a classificare la variabile **attacktype1_txt** in funzione delle variabili predittive:

- ***targtype1_txt***
- ***gname***
- ***weaptype1_txt***
- ***success***
- ***nkill***
- ***year***

Abbiamo scelto queste variabili perché, nelle precedenti analisi predittive, avevamo considerato *attacktype1_txt* come variabile predittiva e quindi queste feature risultano rilevanti per spiegare e prevedere il tipo di attacco.

```
colsoftmax = [ 'targtype1_txt','weaptype1_txt', 'success', 'nkill',
'year']
numeric_col = ['success','nkill','year']
categorical_col = ['targtype1_txt', 'weaptype1_txt']

X = terrorism_cleaned[colsoftmax].copy()


y = terrorism_cleaned['attacktype1_txt']
X_train, X_test, y_train, y_test = train_test_split(X,y,test_size =
0.3,random_state = 42, stratify = y)

print(f"Grandezza dati di train {len(X_train)}")
print(f"Grandezza dati di test {len(X_test)}")


preproc = ColumnTransformer(
    transformers=[
        ("num", StandardScaler(), numeric_col),
        ("cat", OneHotEncoder(handle_unknown="ignore"),
categorical_col)
    ]
)

Grandezza dati di train 85651
Grandezza dati di test 36708
```

I dati vengono suddivisi in **dati di train (70%)**, utilizzati per addestrare il modello, e **dati di test (30%)**, utilizzati per valutare le predizioni su campioni non visti durante l'addestramento.

Per riportare sulla stessa scala le feature numeriche, utilizziamo **StandardScaler**, che le trasforma centrando la media a 0 e scalando la deviazione standard a 1.

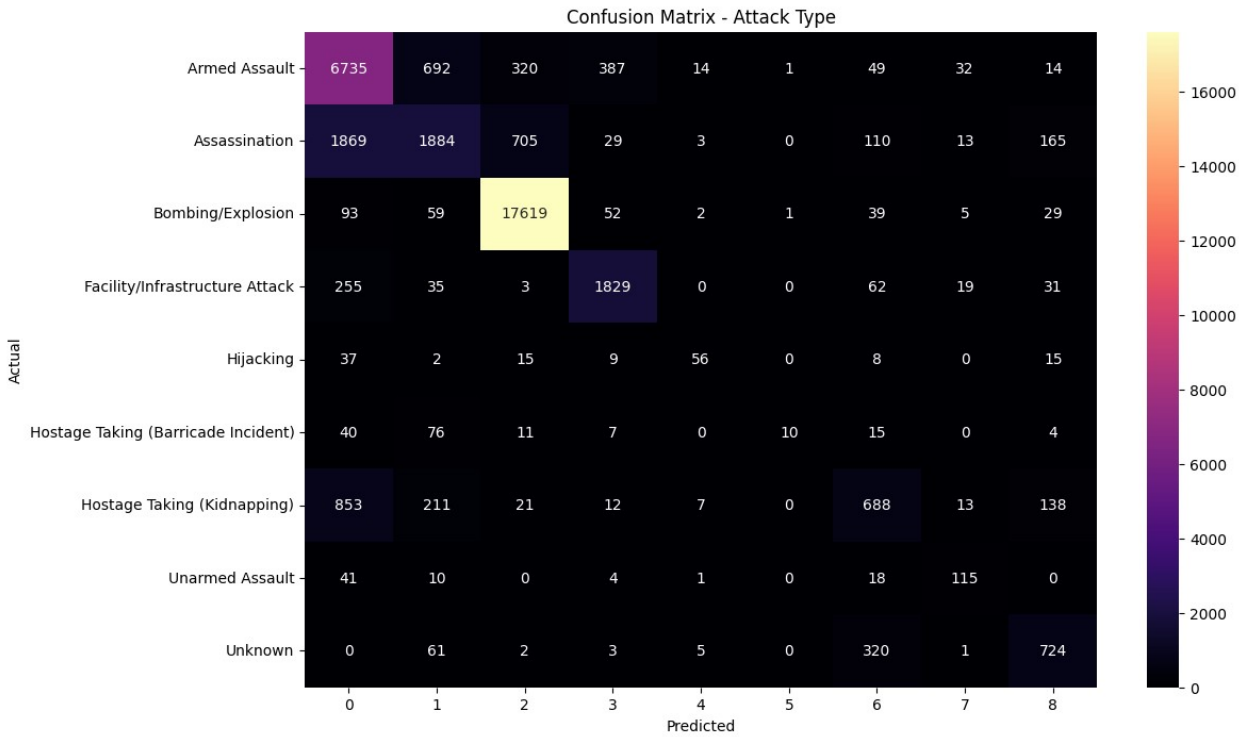
Le feature categoriche vengono trasformate in variabili binarie e dove ogni categoria rappresenta una variabile stessa. Utilizziamo **OneHotEncoder** per la conversione.

Questa preparazione è essenziale per applicare correttamente la **Regressione logistica multiclasse**.

```
pipelog = Pipeline([
    ("preprocess", preproc),
    ("clf", LogisticRegression(
        solver='lbfgs',
        max_iter=500
    ))
])

param_grid = {"clf__C": [0.01, 1]}
grid = GridSearchCV(pipelog, param_grid, cv=5,
scoring="accuracy",n_jobs=-1)
grid.fit(X_train, y_train)


y_pred = grid.predict(X_test)


print(f"Accuracy: {accuracy_score(y_test, y_pred):.2f}")
print(classification_report(y_test, y_pred,zero_division=0))

labels = sorted(y_test.unique())


cm = confusion_matrix(y_test, y_pred)
plt.figure(figsize=(12,8))
sns.heatmap(cm, annot=True, fmt="d", cmap="magma", cbar=True,
yticklabels=labels)
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix - Attack Type")
plt.show()
```

```
Accuracy: 0.81
                                      precision    recall  f1-score
support

                       Armed Assault       0.68      0.82      0.74
8244
                       Assassination       0.62      0.39      0.48
4778
                   Bombing/Explosion       0.94      0.98      0.96
17899
```

```
     Facility/Infrastructure Attack       0.78      0.82      0.80
2234
                           Hijacking       0.64      0.39      0.49
142
Hostage Taking (Barricade Incident)       0.83      0.06      0.11
163
        Hostage Taking (Kidnapping)       0.53      0.35      0.42
1943
                     Unarmed Assault       0.58      0.61      0.59
189
                            Unknown       0.65      0.65      0.65
1116

                           accuracy                           0.81
36708
                          macro avg       0.69      0.56      0.58
36708
                       weighted avg       0.80      0.81      0.80
36708
```



Abbiamo creato una pipeline che unisce il preprocessing dei dati numerici e categorici con una Logistic Regression multiclasse. Poi tramite una **Grid Search**, abbiamo selezionato il miglior parametro di regolarizzazione C. Il modello risultante ha raggiunto un'accuratezza dell'**80%** sul test set.

Analizzando i vari valori, abbiamo che:

- classi con più frequenza, ad esempio **'Armed Assault'** o **'Bombing/esplosion'**, vengono predette molto bene. Possiamo osservare un F1-score alto, indicatore di una correttezza predittiva e capacità di trovare tutti i valori veri;
- classi con meno frequenza, ad esempio **'Hijacking'** o **'Hostage Taking(Barricade Incident)'**, mostrano un recall più basso, indicando che il modello fa fatica a trovare tutte le row. Nonostante la precision sia alta, il valore dell'F1-score si aggira intorno al 0,5.

In generale il modello funziona molto bene su classi frequenti e fa fatica a classificare casi rari presenti nel dataset.

6.3 Classificazione Predittiva -'gname(primi 10 gruppi)'

Precedentemente nell'analisi multivariata, si è visto come i primi 10 gruppi(per numero di attacchi) il bersaglio o il tipo di attacco siano parzialmente caratterizzanti. Per questo motivo si è scelto di creare un modello, contenente qualche informazione in più, che date delle informazioni in input prova a predire il gruppo più indicato per un determinato attacco. La variabile che verrà predetta è **gname**, mentre le variabili sulle quali verranno costruiti i modelli sono:

- ***year***;
- ***region_txt***;
- ***attacktype1_txt***;
- ***targtype1_txt***;
- ***weaptype1_txt***.

Le righe selezionate per costruire i modelli dovranno soddisfare queste due proprietà:

- rientra nella ***top 10 gruppi*** per numero di attacchi;
- il valore di **gname** ≠ *'Unknown'*.

Come visto precedentemente verrà suddivisa la porzione di dataframe in:

- valori di training;
- valori di test.

```
groups = terrorism_cleaned[terrorism_cleaned['gname'] != 'Unknown']
['gname'].value_counts(normalize=True).sort_values(ascending=False).he
ad(10).index

numerical = ['year']
categorical =
['region_txt','attacktype1_txt','targtype1_txt','weaptype1_txt']


dftop =
terrorism_cleaned[terrorism_cleaned['gname'].isin(groups)].copy()

X = dftop[numerical+categorical]
y = dftop['gname']


X_train, X_test, y_train, y_test = train_test_split(X,y,test_size =
```

```
0.2, random_state = 42, stratify = y)



densetrans = FunctionTransformer(lambda X: X.toarray() if hasattr(X,
"toarray") else X, validate=False)

preproc = ColumnTransformer(
    transformers = [
        ('num', StandardScaler(), numerical),
        ("cat", OneHotEncoder(handle_unknown="ignore"), categorical)
    ]
)


models = {
    'LogisticRegression': LogisticRegression(solver ='lbfgs',max_iter=
500),
    'NaiveBayes': GaussianNB(),
    'LDA': LinearDiscriminantAnalysis()
}



pipelines = {}

for name, model in models.items():
        if name in ['NaiveBayes','LDA']:
            pipelines[name] = Pipeline([
                ('preprocessor', preproc),
                ('dense', densetrans),
                ('clf', model)])
        else:
            pipelines[name] = Pipeline([
                ('preprocessor', preproc),
                ('clf', model)])

results = {}

names = []
times = []
accurancies = []

for name, pipe in pipelines.items():
    try:

        start = time.time()

        pipe.fit(X_train, y_train)
        y_pred = pipe.predict(X_test)
```

```
        stop = time.time()- start

        accurancies.append(accuracy_score(y_test, y_pred))
        times.append(stop)
        names.append(name)

        print(f"\n{name} - Accuracy: {accurancies[-1]:.3f}")
        print(classification_report(y_test, y_pred))

        results[name] = y_pred
    except Exception as e:
        print(f"{name} failed: {e}")
```

```
LogisticRegression - Accuracy: 0.889
                                                      precision    recall
f1-score   support

                                        Al-Shabaab       1.00      1.00
1.00       303
              Basque Fatherland and Freedom (ETA)       0.65      0.78
0.71       348
Farabundo Marti National Liberation Front (FMLN)       1.00      1.00
1.00       396
                       Irish Republican Army (IRA)       0.62      0.52
0.57       300
     Islamic State of Iraq and the Levant (ISIL)       0.82      0.94
0.88       392
                    Kurdistan Workers' Party (PKK)       0.88      0.68
0.77       330
                           New People's Army (NPA)       1.00      1.00
1.00       333
   Revolutionary Armed Forces of Colombia (FARC)       0.91      0.72
0.81       349
                                 Shining Path (SL)       0.86      0.96
0.91       620
                                          Taliban       1.00      1.00
1.00       859

                                         accuracy
0.89      4230
                                        macro avg       0.88      0.86
0.86      4230
                                     weighted avg       0.89      0.89
0.89      4230


NaiveBayes - Accuracy: 0.826
```

```
                                                        precision    recall
f1-score   support

                                        Al-Shabaab       1.00      1.00
1.00       303
             Basque Fatherland and Freedom (ETA)       0.58      0.90
0.71       348
Farabundo Marti National Liberation Front (FMLN)       1.00      1.00
1.00       396
                      Irish Republican Army (IRA)       0.57      0.26
0.35       300
     Islamic State of Iraq and the Levant (ISIL)       0.91      0.55
0.69       392
                   Kurdistan Workers' Party (PKK)       0.61      0.85
0.71       330
                           New People's Army (NPA)       0.99      1.00
1.00       333
   Revolutionary Armed Forces of Colombia (FARC)       0.84      0.34
0.49       349
                                Shining Path (SL)       0.72      0.96
0.83       620
                                          Taliban       1.00      1.00
1.00       859

                                         accuracy
0.83      4230
                                        macro avg       0.82      0.79
0.78      4230
                                     weighted avg       0.84      0.83
0.81      4230


LDA - Accuracy: 0.881
                                                        precision    recall
f1-score   support

                                        Al-Shabaab       1.00      1.00
1.00       303
             Basque Fatherland and Freedom (ETA)       0.66      0.75
0.70       348
Farabundo Marti National Liberation Front (FMLN)       1.00      1.00
1.00       396
                      Irish Republican Army (IRA)       0.59      0.56
0.58       300
     Islamic State of Iraq and the Levant (ISIL)       0.78      0.95
0.85       392
                   Kurdistan Workers' Party (PKK)       0.91      0.57
0.70       330
                           New People's Army (NPA)       1.00      1.00
1.00       333
```

```
   Revolutionary Armed Forces of Colombia (FARC)       0.94      0.68
0.79       349
                                Shining Path (SL)       0.85      0.98
0.91       620
                                          Taliban       1.00      1.00
1.00       859

                                         accuracy
0.88      4230
                                        macro avg       0.87      0.85
0.85      4230
                                     weighted avg       0.89      0.88
0.88      4230
```

Applicando i tre modelli si è visto che l'accurancy è alta, questo è dato dalla selezione rigida che si è fatta prima di applicare la pipeline. Questa scelta riduce la complessità del problema e rende le classi più facilmente separabili.

Nonostante ciò, i modelli trovati sono molto utili per capire quali dei gruppi terroristici si distinguono meglio(attraverso le variabili selezionate) rispetto ad altri, ed eventualmente riclassificare gruppi con etichetta **'Unknown'** con le opportune considerazioni:

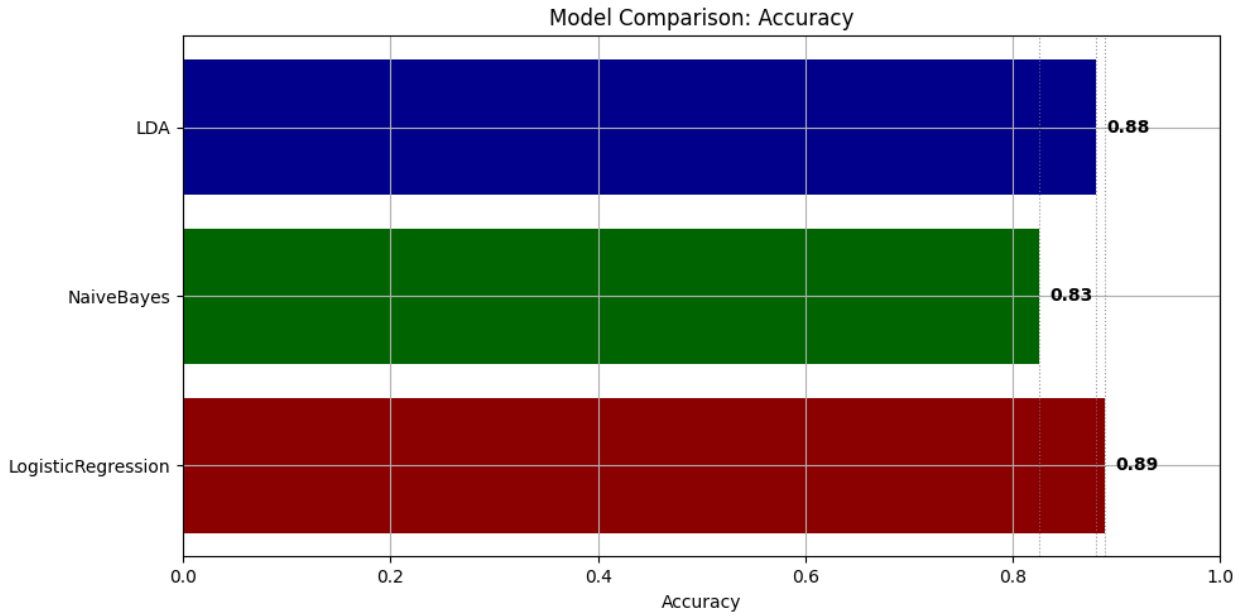
- i modelli dovrebbero essere ricostruiti includendo un numero maggiore di gruppi, escludendo soltanto quelli estremamente rari;
- l'assegnazione del gruppo dovrebbe essere interpretata come una stima probabilistica, non come una verità certa;
- sarebbe opportuno creare una variabile booleana, ad esempio **gname_predicted**, che indichi se l'etichetta è stata assegnata dal modello o è quella originale del dataset.

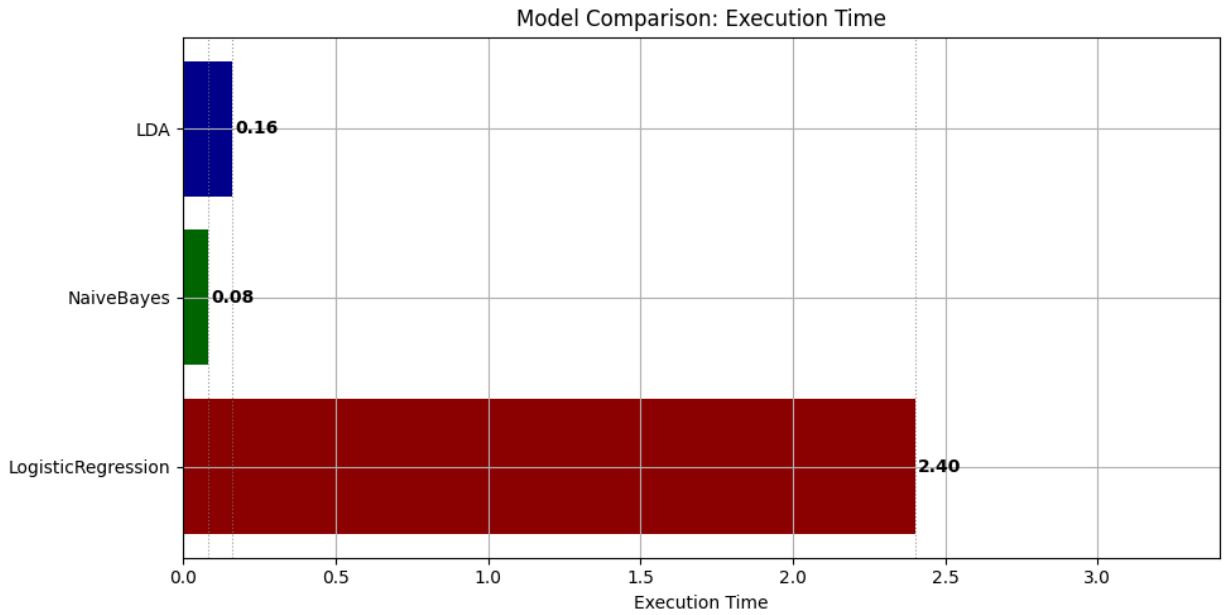
In generale, i risultati ottenuti indicano che:

- gruppi terroristici come **Al-Shabaab**, **Farabundo Martí National Liberation Front**, **Kurdistan Workers' Party** e **Taliban** vengono distinti con maggiore accuratezza. I gruppi in questione presentano pattern ben definiti rispetto al resto;
- gruppi terroristici come **Irish Republican Army** e **Basque Fatherland and Freedom** mostrano performance inferiori in termini di precision, recall e F1-score. Ciò vuol dire che presentano pattern sovrapponibili con altri gruppi o comunque hanno caratteristiche che li rendono meno contrastiguibili ad altri.

```
colors = ['darkred','darkgreen','darkblue']
maxtime = max(times)
fig, axes = plt.subplots(2,1,figsize=(10,10))
axes[0].barh(names,accurancies, color = colors)
axes[0].set_xlim(0,1)
axes[0].set_xlabel("Accuracy")
axes[0].set_title("Model Comparison: Accuracy")
axes[0].grid()
for i, v in enumerate(accurancies):
    axes[0].text(v + 0.01, i, f"{v:.2f}", va='center',
```

```
fontweight='bold')
    axes[0].axvline(x=v, color='gray', linestyle=':', linewidth=0.8,
alpha=0.8)


axes[1].barh(names,times,color=colors)
axes[1].set_xlim(0,maxtime+1)
axes[1].set_xlabel("Execution Time")
axes[1].set_title("Model Comparison: Execution Time")
axes[1].grid()
for i, v in enumerate(times):
    axes[1].text(v + 0.01, i, f"{v:.2f}", va='center',
fontweight='bold')
    axes[1].axvline(x=v, color='grey', linestyle=':', linewidth=0.8,
alpha=0.8)
plt.tight_layout()
plt.show()
```



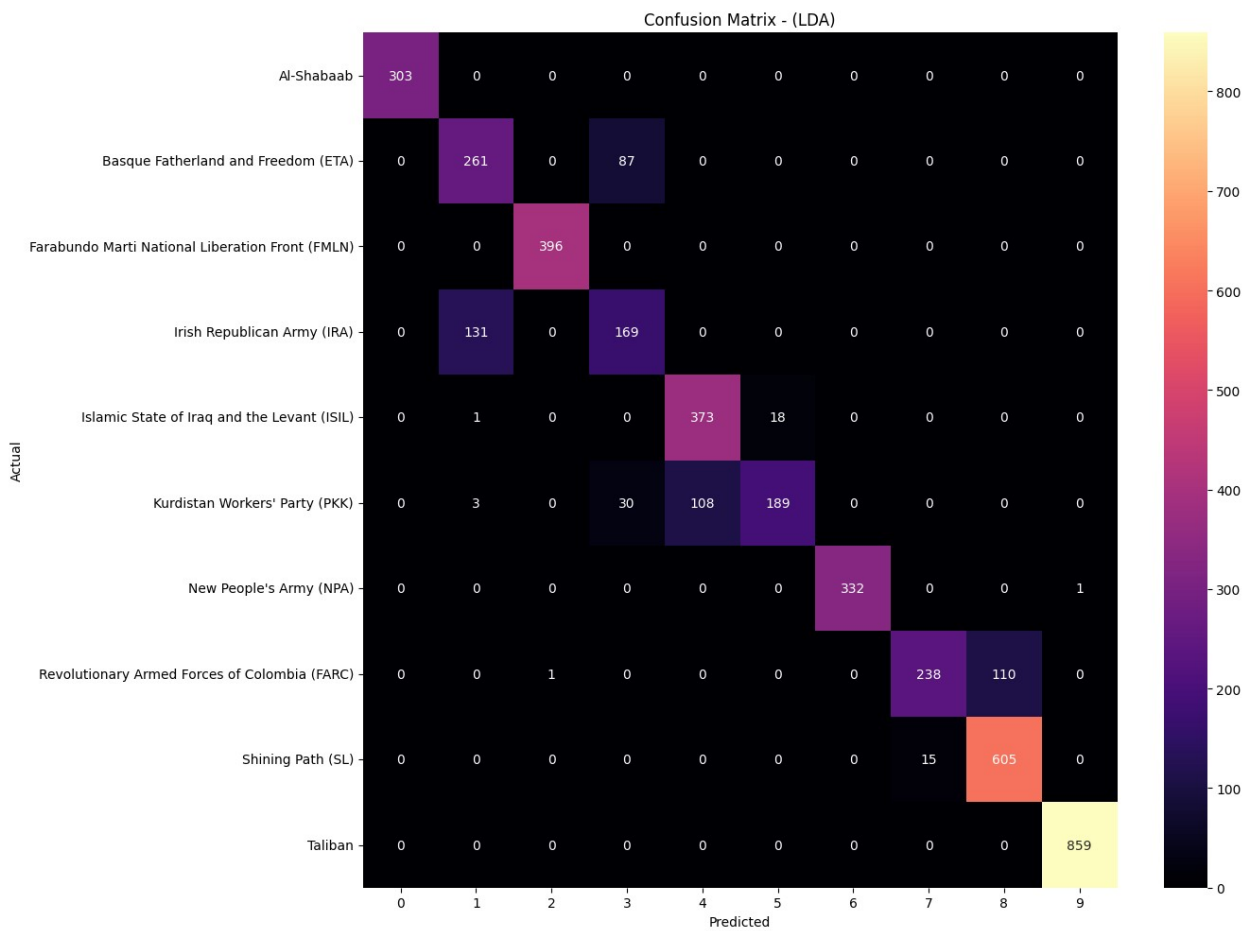


Come accennato prima i tre modelli presentano una **accurancy** elevata, nel dettaglio:

- **LDA ACCURANCY:** ***0.88***;
- **NAIVE BAYES:** ***0.83***;
- **LOGISTIC REGRESSION:** ***0.89***;

La Logistic Regression ottiene l'accuratezza più alta, ma a fronte di un costo computazionale maggiore. La Naive Bayes è il modello più veloce, ma risulta meno affidabile per alcune classi a causa dell'assunzione di indipendenza tra le variabili. La LDA rappresenta il miglior compromesso tra accuratezza e tempo di esecuzione, fornendo risultati molto simili alla Logistic Regression con un costo computazionale nettamente inferiore.

```
bestmodel = "LDA"
bestpipe = pipelines[bestmodel]
bestpipe.fit(X_train,y_train)
y_pred = bestpipe.predict(X_test)

labels = sorted(y.unique())
cm = confusion_matrix(y_test, y_pred, labels=labels)


plt.figure(figsize=(14,10))
sns.heatmap(cm, annot=True, fmt="d", cmap="magma", yticklabels=labels)

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title(f"Confusion Matrix - ({bestmodel})")
plt.tight_layout()
plt.show()
```



FASE 7: Unsupervised Learning

L'approccio utilizzato in questa sezione è l'**unsupervised learning**, ossia la scoperta di strutture nascoste all'interno del nostro dataset attraverso l'apprendimento non supervisionato.

Ciò vuol dire che l'algoritmo cerca di **trovare somiglianze e differenze tra punti dati** del dataset basandosi solo sulle caratteristiche in input. Non si sa in anticipo quale sia il risultato, infatti, le variabili target sono escluse dal modello.

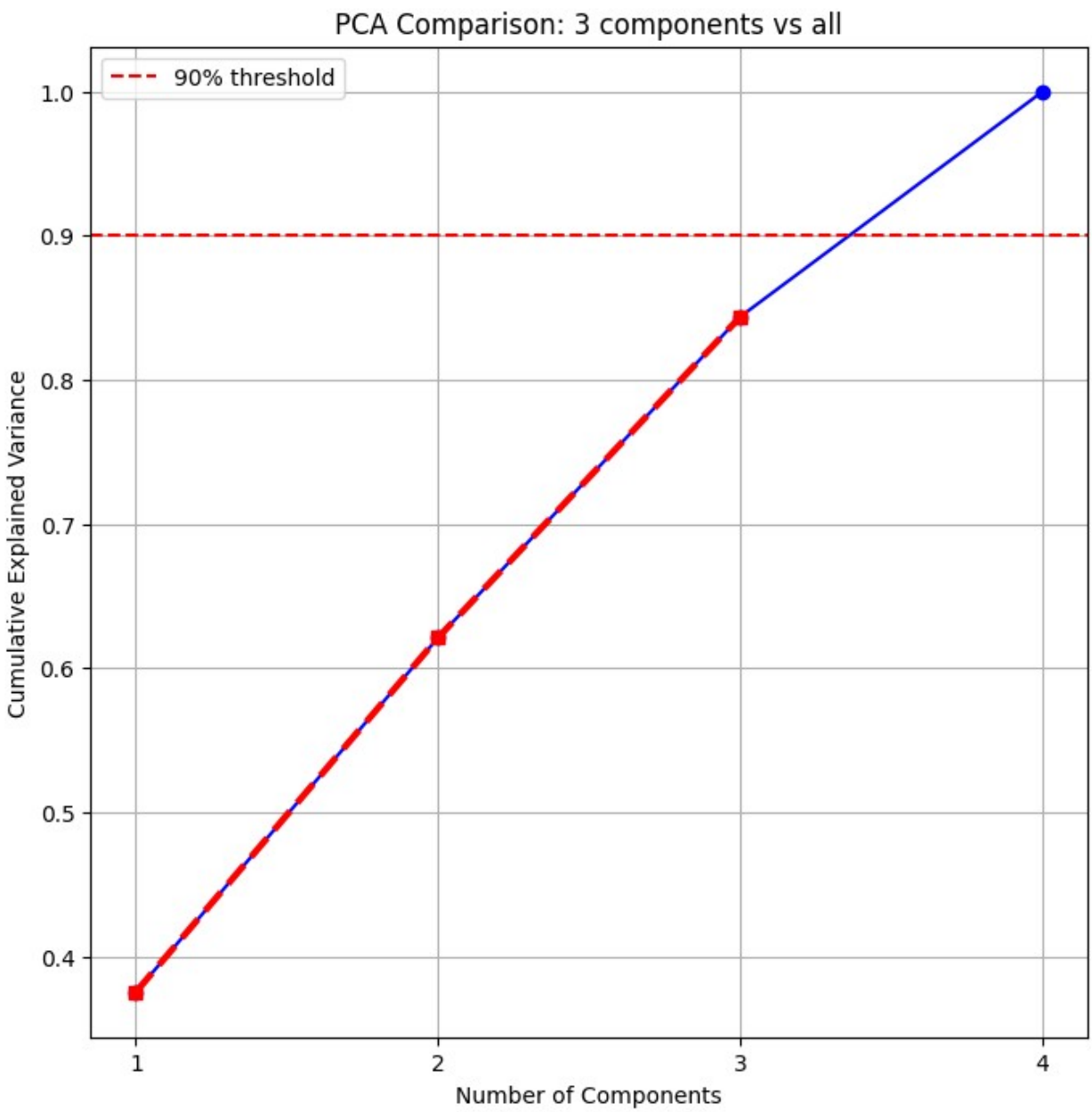
Uno dei possibili obiettivi potrebbe essere la scoperta di strutture nascoste come tipologie di attacchi che sono più o meno pericolosi, a seconda delle caratteristiche intriseche dell'evento(nkill, nwound, nperps, suicide). Inoltre, potremmo applicare un'analisi sui cluster e valutare a posteriori il tasso di successo degli attacchi da parte dei terroristi.

Un'ulteriore domanda di ricerca riguarda la possibilità di individuare pattern temporali e geografici negli attacchi terroristici. Sebbene strumenti precedentemente introdotti riescano a fornire solo indicazioni preliminari, il raggruppamento non supervisionato consente di esplorare in modo più strutturato le concentrazioni spaziali e temporali degli eventi, evidenziando eventuali tendenze nascoste.

7.1 Raggruppamento su dati ridimensionati: Analisi letalita'

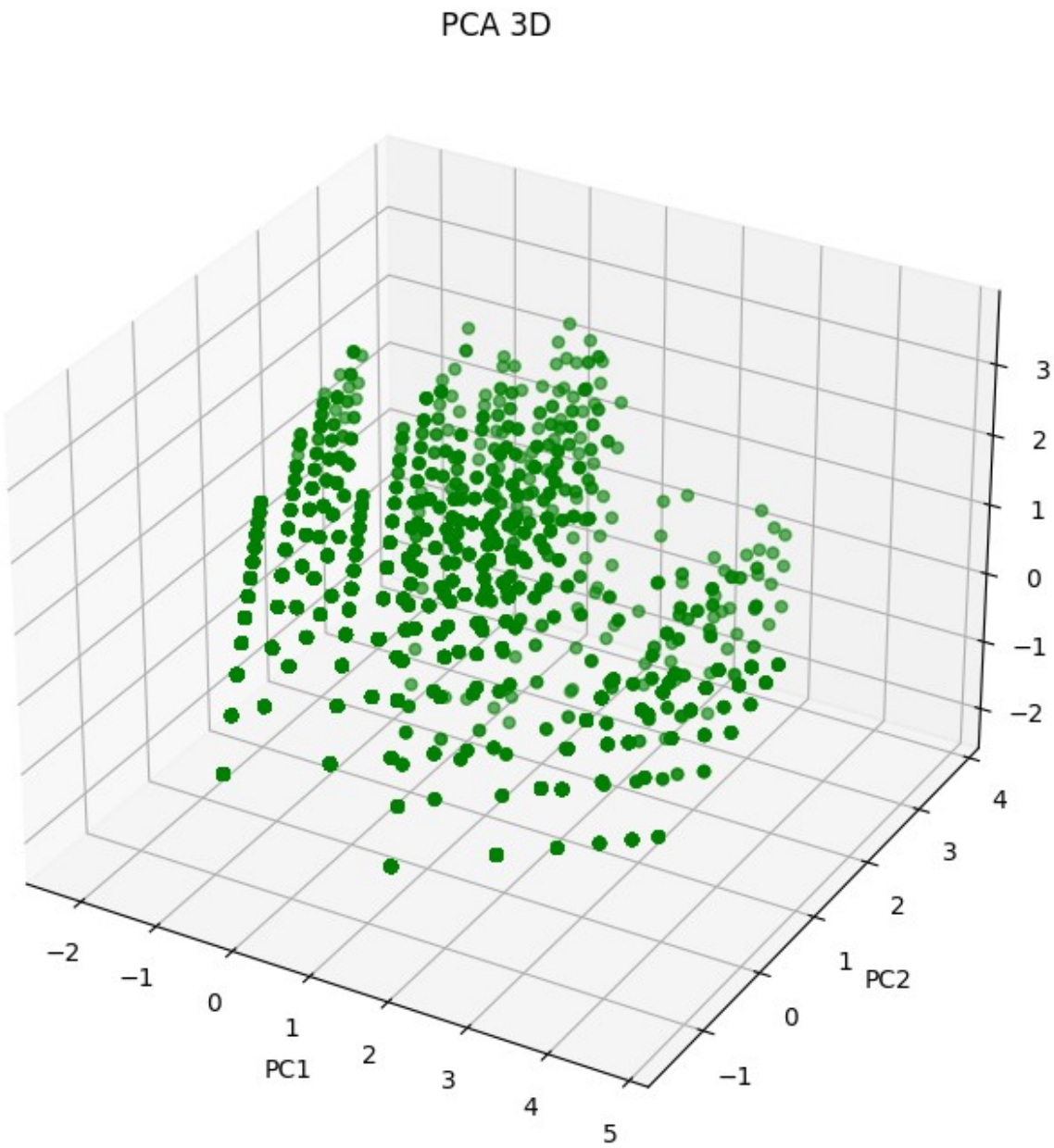
```
col = ['nkill','nwound','nperps','suicide']
df = terrorism_cleaned[col].copy()
df = df.dropna()
for c in ['nkill','nwound','nperps']:
    df[c] = np.log1p(df[c])


scaler = StandardScaler()
X_scaled = scaler.fit_transform(df)

pca = PCA(n_components = 3)
pcafull = PCA()
X_pca = pca.fit_transform(X_scaled)
X_pcafull = pcafull.fit_transform(X_scaled)



varcumulative = np.cumsum(pca.explained_variance_ratio_)
varcumulative_full = np.cumsum(pcafull.explained_variance_ratio_)


plt.figure(figsize=(8,8))
plt.plot(range(1,len(varcumulative_full)+1),varcumulative_full,'-
o',color = 'blue')
plt.plot(range(1,4),varcumulative,'s--', color = 'red', linewidth=3)
plt.axhline(0.9, color='red', linestyle='--', label='90% threshold')
plt.xlabel('Number of Components')
```

```
plt.ylabel('Cumulative Explained Variance')
plt.title('PCA Comparison: 3 components vs all')
plt.xticks(range(1,len(varcumulative_full)+1))
plt.grid()
plt.legend()
plt.show()

print(f"Varianza cumulativa con 3 componenti:
{varcumulative.round(2)}")
```




```
Varianza cumulativa con 3 componenti: [0.38 0.62 0.84]
```

```
fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')

ax.scatter(X_pca[:,0], X_pca[:,1], X_pca[:,2], alpha=0.6, c='green')
ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
ax.set_title('PCA 3D')
ax.grid(True)

plt.show()
```



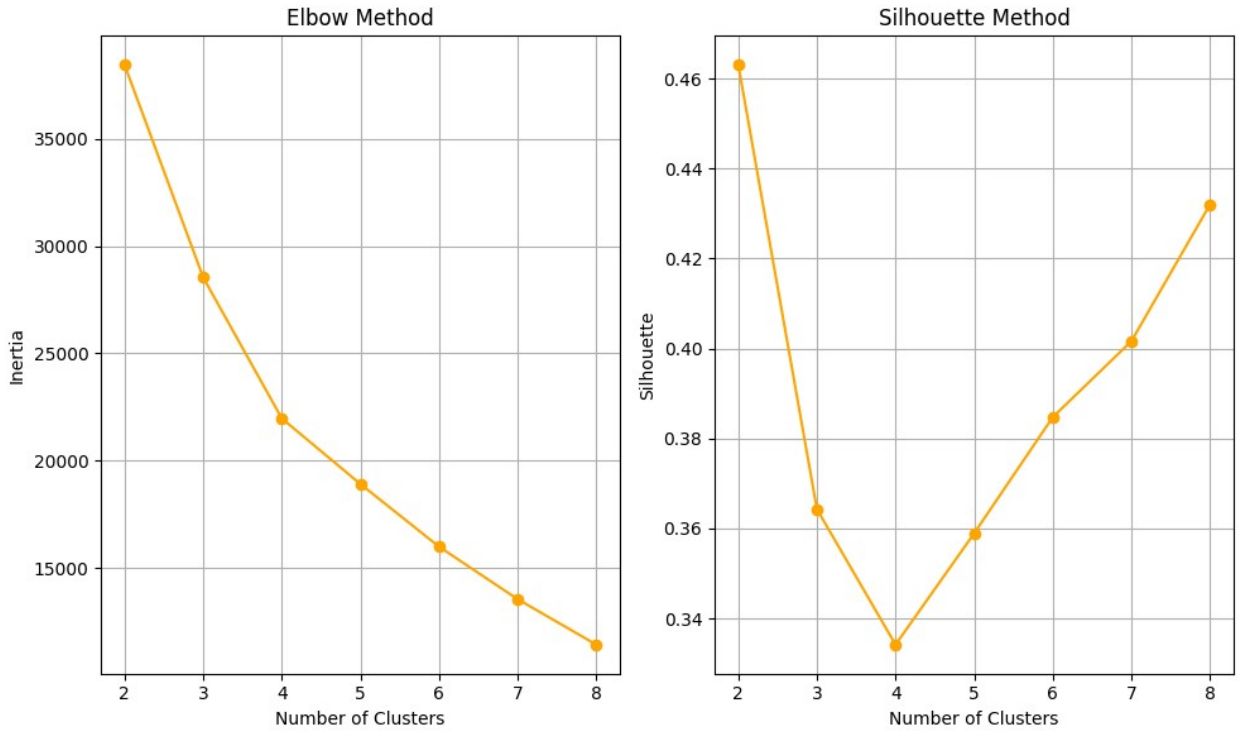
Per diminuire la dimensione del dataset si utilizza una tecnica di **riduzione dimensionale**, in particolare la **PCA(principal component analisys)**. Quindi, da uno spazio ad alta dimensionalità si passa ad uno spazio più piccolo, preservando la **varianza** dei dati.

La scelta del numero di **componenti principali** da mantenere può basarsi sulla quantità di varianza spiegata. Nel nostro caso, sono state **selezionate 3 componenti principali**, che conservano circa l'**84% della varianza** complessiva.

Prese solo le 3 componenti principali, possiamo visualizzare i dati e vedere che il grafico prodotto rappresenta una funzione a forma di sella. I punti dati non sono separati in gruppi netti, ma distribuiti lungo una struttura continua che va da attacchi piccoli a eventi più gravi, senza interruzioni brusche.

Questa rappresentazione permette di osservare la distribuzione e la gradazione della pericolosità degli attacchi. Per approfondire la struttura sottostante, possiamo applicare il K-Means clustering e verificare se emergono gruppi distinti che ci aiutano a identificare tipologie di eventi pericolosi.

```
inertia = []
silhouette = []
for k in range(2,9):
    kmeans = KMeans(n_clusters = k, n_init=10, random_state =42)
    labels = kmeans.fit_predict(X_pca)

    inertia.append(kmeans.inertia_)
    silhouette.append(silhouette_score(X_pca, labels))



plt.figure(figsize=(10,6))
plt.subplot(1,2,1)
plt.plot(range(2,9),inertia,'-o', c='orange')
plt.xlabel("Number of Clusters")
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid()

plt.subplot(1,2,2)
plt.plot(range(2,9),silhouette, '-o', c='orange')
plt.xlabel("Number of Clusters")
plt.ylabel("Silhouette")
plt.title("Silhouette Method")
plt.grid()
plt.tight_layout()
plt.show()
```



Attraverso queste metriche possiamo determinare qual è il numero di cluster ottimale che il modello K-Means deve identificare. Le due metriche principali considerate sono:

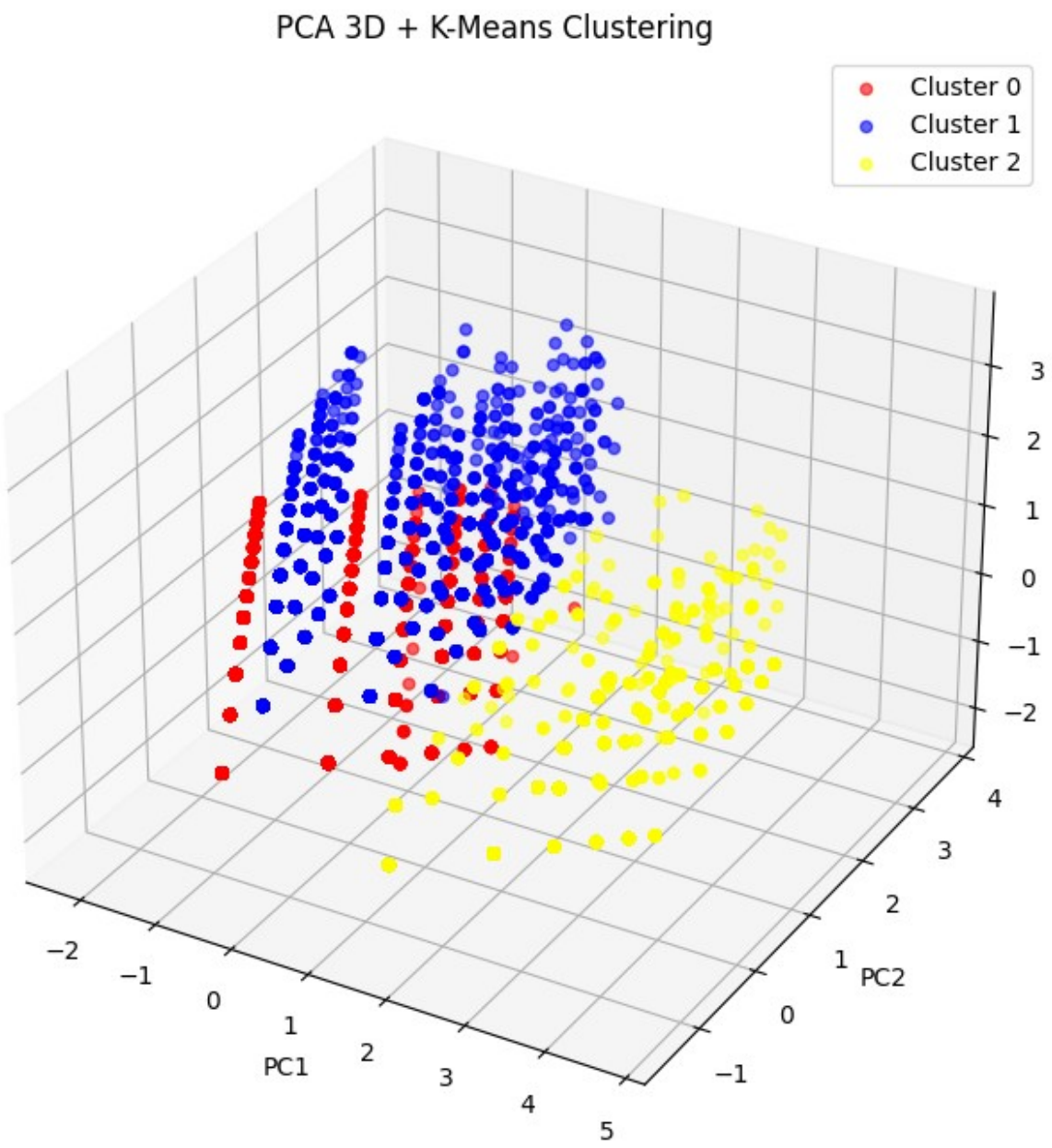
- **Metodo Elbow**: prevede il calcolo dell'***inertia*** ossia una misura di quanto un punto è vicino al proprio centroide. Si fa la somma per ogni punto e infine dopo averlo fatto con vari k(da 2 a 9) si sceglie il valore di k in presenza del gomito. Questo valore è quello in cui riduzioni successive di Inertia diventano marginali.

- **Silhouette Score**: misura quanto un punto è simile ai punti del proprio cluster rispetto a quelli degli altri cluster. Valori più alti indicano una separazione più netta tra cluster. Si scegli il valore di K che produce un silhouette score molto alto.

Nel nostro caso, il massimo Silhouette si ottiene per k=2, ma abbiamo scelto ***k=3*** per motivi interpretativi e strategici: distinguere gli attacchi suicidi dai medi e dai minori fornisce maggiore valore analitico, anche se la separazione matematica è meno netta

```
k = 3
kmeans = KMeans(n_clusters=k, random_state=42, n_init=10)
labels = kmeans.fit_predict(X_pca)

df['cluster'] = labels

fig = plt.figure(figsize=(10,8))
ax = fig.add_subplot(111, projection='3d')

colors = ['red','blue','yellow']
```

```
for i in range(k):
    ax.scatter(X_pca[labels==i,0], X_pca[labels==i,1],
X_pca[labels==i,2],
               color=colors[i], alpha=0.6, label=f'Cluster {i}')

ax.set_xlabel('PC1')
ax.set_ylabel('PC2')
ax.set_zlabel('PC3')
ax.set_title('PCA 3D + K-Means Clustering')
ax.legend()
plt.show()


summary = df.groupby('cluster')
[['nkill','nwound','nperps','suicide']].mean()
summary[['nkill','nwound','nperps']] =
np.expm1(summary[['nkill','nwound','nperps']])
print(summary)
```



cluster	nkill	nwound	nperps	suicide
0	0.613217	0.000000	2.634180	0.002397
1	0.518725	1.592231	2.453328	0.000225
2	1.576173	0.951406	1.308175	0.966589

Dopo aver applicato K-Means con k=3, i cluster individuati rappresentano tre tipologie di attacchi terroristici, distinguibili sia per gravità che per tattica. Questi sono:

- **Cluster 0 -Attacchi di gruppo a bassa intensità**: Questo cluster comprende attacchi con molti perpetratori ma poche vittime. Si tratta di operazioni di gruppo mirate, generalmente poco letali, come assalti minori o operazioni dimostrative.

- **Cluster 1 – Attacchi intermedi / assalti armati**: Cluster di attacchi medi, con un numero di feriti più elevato rispetto al cluster 0, ma senza raggiungere valori estremi di letalità. Rappresenta operazioni locali o assalti armati non suicidi.
- **Cluster 2 – Attacchi suicidi ad alta letalità**: Qui rientrano gli attacchi suicidi ad alta letalità, con pochi perpetratori ma un numero elevato di morti. Questo cluster rappresenta gli eventi più pericolosi e letali del database

```
dfcompleted = terrorism_cleaned[col + ['success']].dropna()
dfcompleted['cluster'] = labels
succrate = dfcompleted.groupby('cluster')['success'].mean().round(2)
print(succrate)

cluster
0    0.84
1    0.82
2    0.70
Name: success, dtype: float64
```

Dopo aver calcolato il successo medio di ogni cluster possiamo vedere che:

- I *cluster meno letali(0 e 1)* hanno un **successo maggiore**.
- Il *cluster suicida(2)*, pur essendo il più letale, ha un tasso di **successo più basso** (70%), probabilmente perché gli attacchi suicidi sono più rischiosi e difficili da portare a termine.

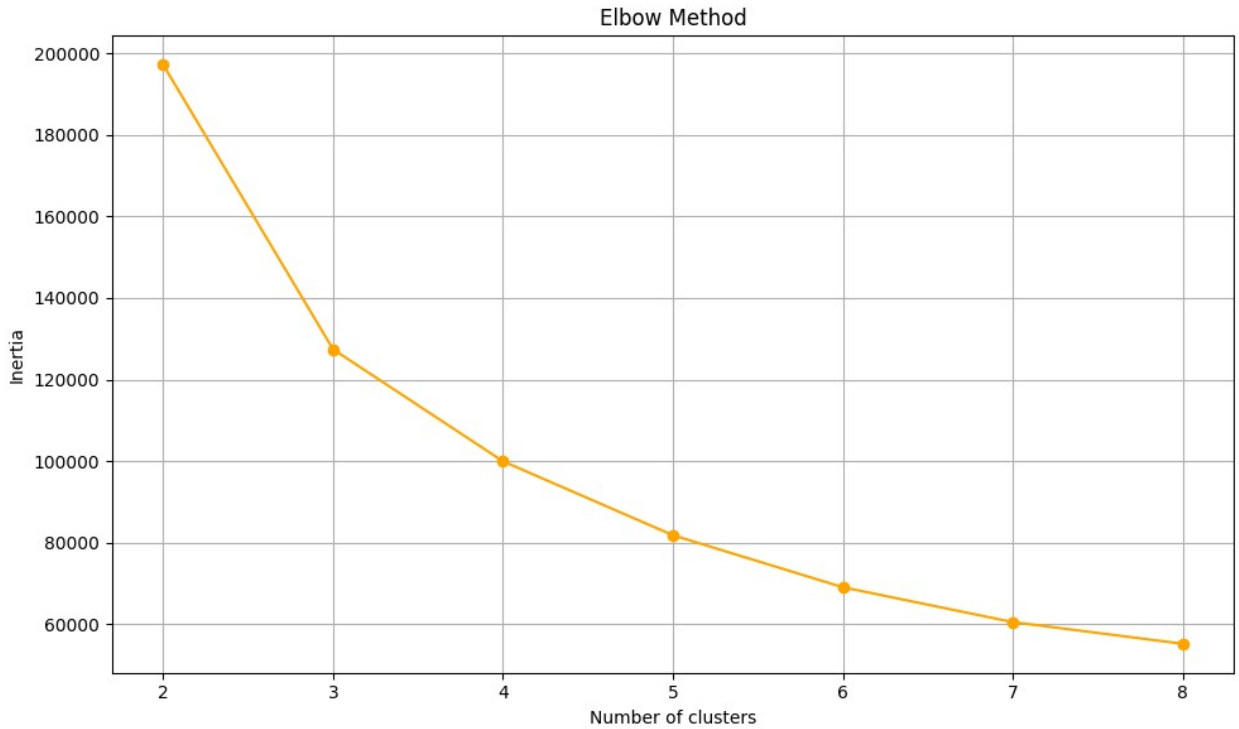
Questo mostra che **pericolosità ≠ probabilità** di successo e quindi che gli attacchi più devastanti non sono sempre quelli più efficaci.

7.2 Raggruppamento su dati non ridimensionati: Analisi temporale e Geografica

In questa paragrafo proveremo a trovare dei gruppi separabili in modo temporale e geografico. Utilizzeremo come nel caso precedente il Kmeans e k ossia il numero dei cluster verrà trovato attraverso il metodo **Elbow**.

```
cols = ['latitude','longitude','year']
df = terrorism_cleaned[cols].dropna().copy()

scaler = StandardScaler()
Xscaled = scaler.fit_transform(df)

inertia = []
for k in range(2,9):
    kmeans = KMeans(n_clusters = k, n_init=10, random_state =42)
    labels = kmeans.fit_predict(Xscaled)
    inertia.append(kmeans.inertia_)

plt.figure(figsize=(10,6))
plt.plot(range(2,9),inertia,'-o', c='orange')
plt.xlabel("Number of clusters")
```

```
plt.ylabel("Inertia")
plt.title("Elbow Method")
plt.grid()
plt.tight_layout()
plt.show()
```



Dal grafico del **Metodo Elbow** emerge che la somma delle distanze diminuisce rapidamente fino a **3-4 cluster**, per poi stabilizzarsi. Questo indica che 3 o 4 cluster rappresentano un buon compromesso tra compattezza dei gruppi e semplicità del modello, mentre aumentare ulteriormente il numero di cluster fornirebbe poca informazione aggiuntiva.

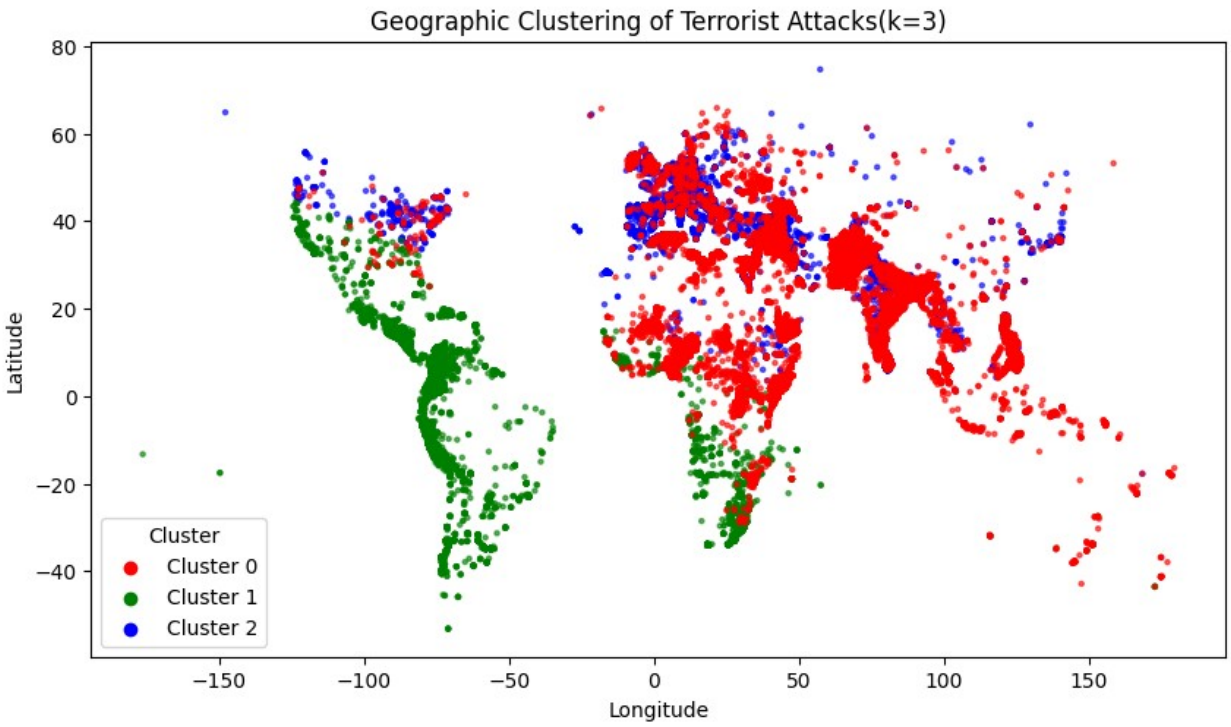
Come sappiamo questo è un problema di **trade-off bias varianza**, per questo motivo:

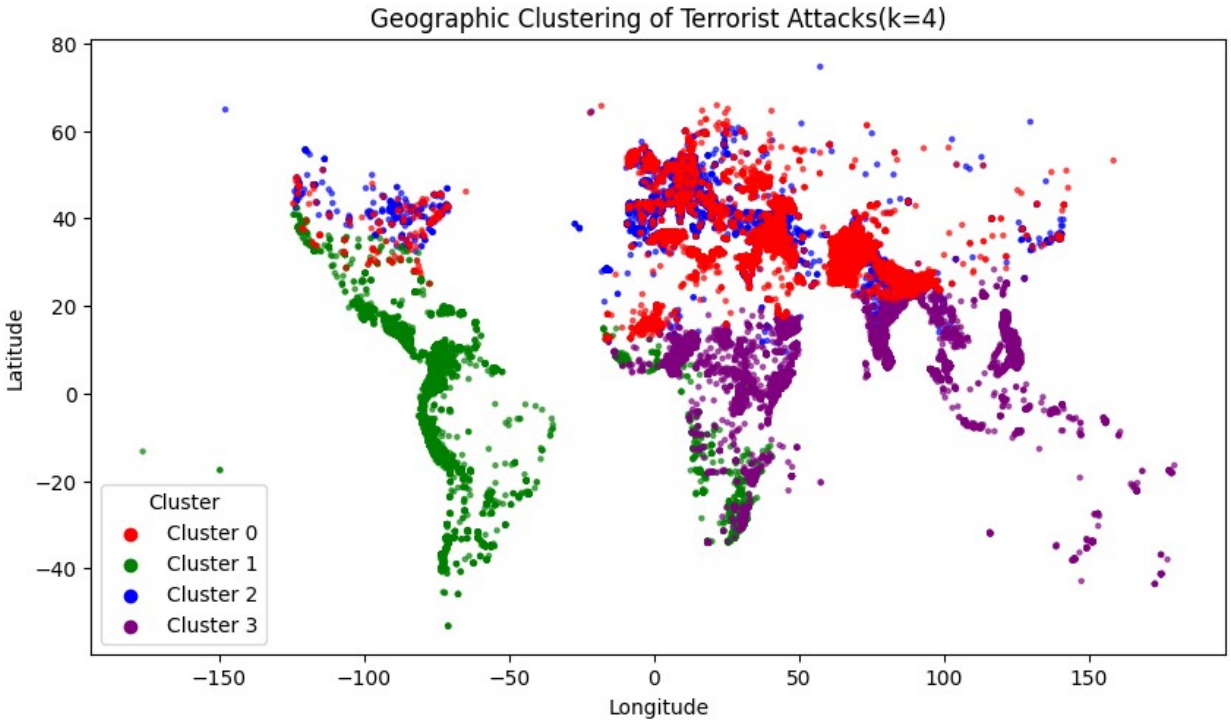
- se scegliessimo un k minore, cluster eterogenei verrebbero aggregati;
- se scegliessimo un k maggiore, cluster omogenei verrebbero divisi.

Osserviamo come i cluster vengono rappresentati nel grafico: se risultano già ben separati, manterremo 3 cluster; in caso contrario, prenderemo in considerazione 4 cluster.

```
kmeans = KMeans(n_clusters=3, random_state=42, n_init=10)
labels = kmeans.fit_predict(Xscaled)

df['cluster'] = labels
colors = np.array(['red','green','blue','purple'])

plt.figure(figsize=(10,12))
plt.subplot(2,1,1)
```

```
plt.scatter(df['longitude'], df['latitude'], c=colors[df['cluster']],
s=5, alpha=0.6)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Geographic Clustering of Terrorist Attacks(k=3)')
for cl in np.unique(df['cluster']):
    plt.scatter([], [], color=colors[cl], label=f"Cluster {cl}")
    plt.legend(title="Cluster")


kmeans = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = kmeans.fit_predict(Xscaled)

df['cluster'] = labels

plt.subplot(2,1,2)
s = plt.scatter(df['longitude'], df['latitude'],
c=colors[df['cluster']], s=5, alpha=0.6)
plt.xlabel('Longitude')
plt.ylabel('Latitude')
plt.title('Geographic Clustering of Terrorist Attacks(k=4)')
for cl in np.unique(df['cluster']):
    plt.scatter([], [], color=colors[cl], label=f"Cluster {cl}")
    plt.legend(title="Cluster")


plt.show()
```





Il raggruppamento in cluster distinti e l'utilizzo della legenda rende immediata l'interpretazione della mappa. Possiamo vedere come attraverso il raggruppamento in 3 cluster si riescono a distinguere:

- **Africa orientale**,**Asia** e **Europa**(CLUSTER 0);
- **Nord America**, **Europa** e **Asia Occidentale**(CLUSTER 2);

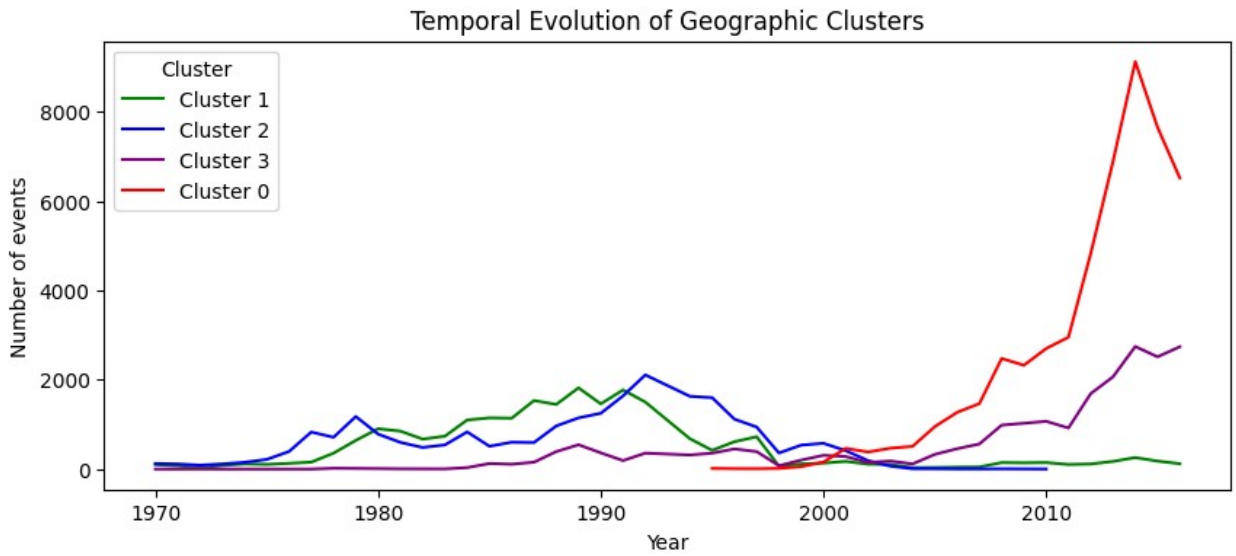
- **Centro** e **Sud America** e **Africa Occidentale**(CLUSTER 1);

Questo raggruppamento però non è ottimale perchè non fa delle separazioni molto nette. Per questo motivo aggiungendo un ulteriore cluster si ottiene:

- **Europa** e **Asia Occidentale**(CLUSTER 0);
- **Centro** e **Sud America**(inoltre una porzione di africa occidentale)(CLUSTER 1);
- **Europa** e **Nord America**(CLUSTER 2);
- **Africa** e **Asia**(CLUSTER 3);

Nota la separazione un po' più netta si sceglie di procedere con il clustering con **k=4** numero di cluster.

```
clusteryear =
df.groupby(['year','cluster']).size().reset_index(name='count')

plt.figure(figsize=(10,4))
for c in clusteryear['cluster'].unique():
    subset = clusteryear[clusteryear['cluster'] == c]
    plt.plot(subset['year'],
subset['count'],color=colors[c] ,label=f'Cluster {c}')
plt.xlabel('Year')
plt.ylabel('Number of events')
plt.title('Temporal Evolution of Geographic Clusters')
plt.legend(title="Cluster")
plt.show()

stats = df.groupby('cluster').agg(
    NumeroAttachi=('year','size'),
    Inizio=('year','min'),
    Fine=('year','max'),
    Picco=('year', lambda x: x.mode().iat[0]),
    MediaLatitudine=('latitude','mean'),
    MediaLongitudine=('longitude','mean')
)
print()
print("Statistiche generali per cluster:")
print(stats)
print()
print()



df['success'] = terrorism_cleaned.loc[df.index,'success']
succrate = df.groupby('cluster')['success'].mean()
print(succrate.round(2))
```




```
Statistiche generali per cluster:
         NumeroAttachi  Inizio  Fine  Picco  MediaLatitudine
MediaLongitudine
cluster

0                51240    1995  2016   2014        33.756088
52.169667
1                22561    1970  2016   1989        -2.548650        -
67.791591
2                23466    1970  2010   1992        39.769673
22.690623
3                22286    1970  2016   2014         9.127355
76.151825


cluster
0    0.85
1    0.92
2    0.86
3    0.88
Name: success, dtype: float64
```

L'analisi dei cluster mostra come il **terrorismo** si sia **spostato nel tempo** e **nello spazio**. Negli anni '70-'90 gli attacchi erano concentrati nelle Americhe e in Europa, mentre dal 2000 in poi il fenomeno si è spostato verso il Medio Oriente, Nord Africa e Asia meridionale.

Il picco più evidente si osserva nel 2014, con il cluster del Medio Oriente e Nord Africa che supera di gran lunga gli altri in termini di numero di eventi.

Questi risultati evidenziano chiaramente un cambiamento sia geografico sia temporale nell'intensità degli attacchi.

7.3 Stima della distribuzione degli attacchi per paese mediante KDE e modelli di densita': Analisi geografica e temporale

In questo paragrafo stimiamo la **distribuzione degli attacchi terroristici per paese e anno** utilizzando ***modelli di densità non parametrici e parametrici***.

L'obiettivo è ottenere una visualizzazione avanzata della frequenza e dei pattern degli attacchi, evidenziando gruppi di eventi simili e aree a bassa densità.

Questa analisi permette di osservare quali paesi hanno un numero di attacchi tipico e quali presentano eventi più rari o atipici.

Inoltre, per rendere l'analisi più sensibile ai cambiamenti nel tempo, oltre al numero di attacchi nel tempo, viene considerata anche la variazione anno-su-anno.

I nostri **modelli individueranno eventi atipici** se questi presentano:

- un numero di attacchi significativamente diverso dalla media dei paesi;
- variazioni nel numero di attacchi diverse rispetto alle variazioni degli altri paesi.

```
attackcounter = terrorism_cleaned.groupby(['country_txt',
'year']).size().reset_index(name='n_attacks')
attackcounter['delta'] = attackcounter.groupby('country_txt')
['n_attacks'].diff().fillna(0)
attackcounter['delta'] =  attackcounter['delta'].astype(int)
X = attackcounter[['n_attacks', 'delta']].values

scaler = StandardScaler()
Xscaled = scaler.fit_transform(X)

gmm1 = GaussianMixture(n_components=1, random_state=42)
gmm1.fit(Xscaled)

gmm3 = GaussianMixture(n_components=3, random_state=42)
gmm3.fit(Xscaled)

kde = KernelDensity(kernel='gaussian', bandwidth=0.4)
kde.fit(Xscaled)
print()
```

Prima di procedere si calcolano:

- **Numero di attacchi (`n_attacks`)**: quanti eventi sono avvenuti in quell'anno in quel paese;
- **Variazione anno-su-anno (`delta`)**: quanto il numero di attacchi è cambiato rispetto all'anno precedente.

Successivamente, standardizziamo questi valori per renderli comparabili e costruire modelli di densità che ci permettano di stimare la distribuzione dei dati nello spazio.

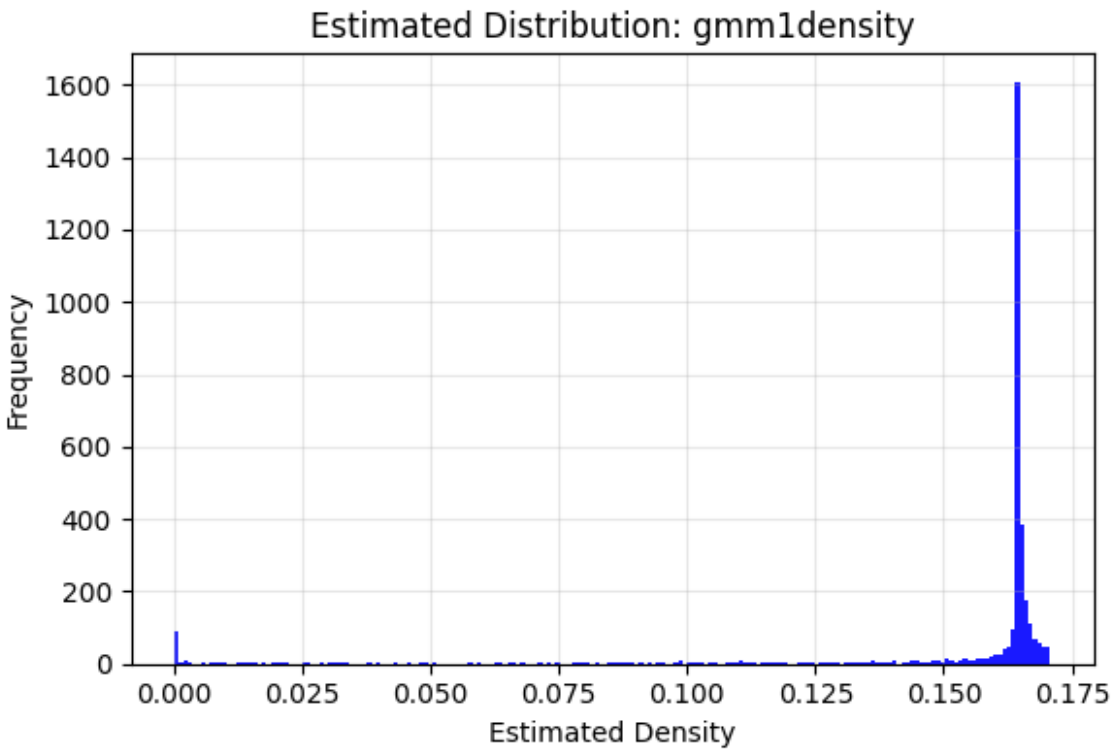
Abbiamo utilizzato tre modelli diversi:

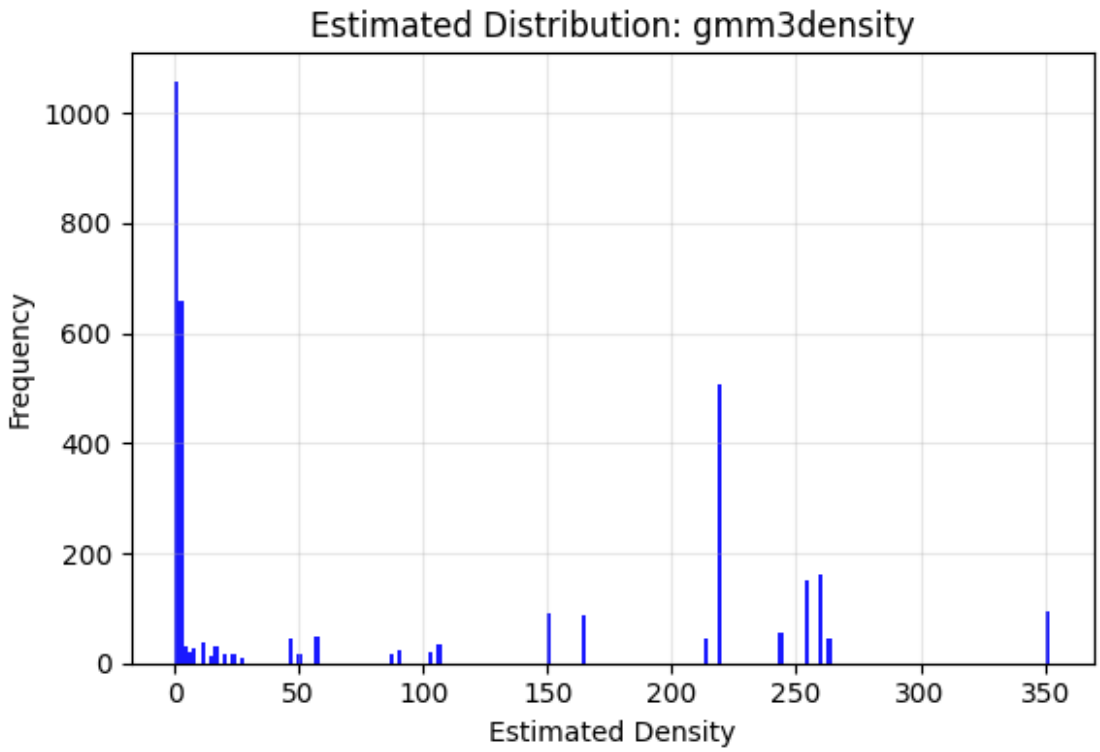
1. **Single Gaussian (GMM con K=1)**: fornisce una stima unimodale generale;

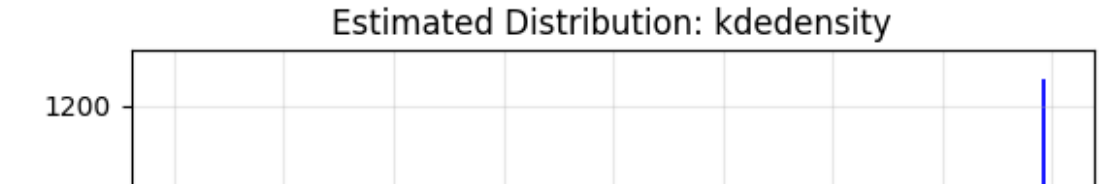
2. **Gaussian Mixture Model (GMM con K=3)**: cattura eventuali sottogruppi di paesi con comportamenti simili negli attacchi;
3. **Kernel Density Estimation (KDE)**: una stima non parametrica che ci permette di visualizzare la distribuzione dei dati senza assumere una forma particolare.

Questi modelli ci consentono di osservare come i dati si distribuiscono nello spazio considerato e di identificare pattern o valori atipici rispetto al comportamento medio.

```
attackcounter['gmm1density'] = np.exp(gmm1.score_samples(Xscaled))
attackcounter['gmm3density'] = np.exp(gmm3.score_samples(Xscaled))
attackcounter['kdedensity']  = np.exp(kde.score_samples(Xscaled))


plt.figure(figsize=(6, 12))

for i, col in enumerate(['gmm1density', 'gmm3density', 'kdedensity']):
    plt.subplot(3, 1, i+1)
    plt.hist(attackcounter[col], bins=200, density=False, alpha=0.9,
color='blue')
    plt.title(f"Estimated Distribution: {col}")
    plt.xlabel("Estimated Density")
    plt.ylabel("Frequency")
    plt.grid(alpha=0.3)

plt.tight_layout()
plt.show()

for model in ['gmm1density', 'gmm3density', 'kdedensity']:
    rare_points = attackcounter.sort_values(model).head(5)
    print(f"\nPunti con {model} più bassa (rari / bassa densità):")
    print(rare_points[['country_txt', 'year', 'n_attacks', 'delta',
model]])
```


Estimated Distribution: gmm1density
Frequency
Estimated Density
0
200
400
600
800
1000
1200
1400
1600
0.000
0.025
0.050
0.075
0.100
0.125
0.150
0.175


Estimated Distribution: gmm3density
Frequency
Estimated Density
0
200
400
600
800
1000
0
50
100
150
200
250
300
350


Estimated Distribution: kdedensity
1200

```
Punti con gmm1density più bassa (rari / bassa densità):
      country_txt  year  n_attacks  delta    gmm1density
1406         Iraq  2015       1408  -1107  4.355092e-162
2225     Pakistan  2015       1097   -805   8.887213e-90
1405         Iraq  2014       2515    694   2.015534e-89
1404         Iraq  2013       1821    755   3.975723e-60
23    Afghanistan  2012       1165    884   1.195213e-55

Punti con gmm3density più bassa (rari / bassa densità):
     country_txt  year  n_attacks  delta   gmm3density
1406        Iraq  2015       1408  -1107  2.151881e-29
1405        Iraq  2014       2515    694  6.719354e-19
2225    Pakistan  2015       1097   -805  2.876745e-17
1404        Iraq  2013       1821    755  1.065218e-12
2224    Pakistan  2014       1902     10  1.220792e-12

Punti con kdedensity più bassa (rari / bassa densità):
      country_txt  year  n_attacks  delta  kdedensity
3031      Ukraine  2014        705    700    0.000296
23    Afghanistan  2012       1165    884    0.000296
1404         Iraq  2013       1821    755    0.000296
1406         Iraq  2015       1408  -1107    0.000296
2225     Pakistan  2015       1097   -805    0.000296
```

Le variabili analizzate ci consentono sia di analizzare il volume degli attacchi e sia cambiamenti improvvisi e trend.

I tre modelli hanno restituito informazioni diverse dato che sono più o meno flessibili e fanno più o meno assunzioni.

I risultati sono:

- **Single Gaussian**
  - Assume che tutti i dati provengano da un unico gruppo con valori medi;
  - Nel grafico, la maggior parte dei dati si concentra attorno a un picco con densità circa 0.16 e con una lunga coda verso sinistra;
  - Gli eventi lungo la coda sono atipici, infatti le probabilità si avvicinano a 0;
- **Gaussian Mixture Model con 3 componenti**
  - Identifica fino a tre sottogruppi di comportamento tra i paesi;
  - Nel grafico emergono più picchi: ad esempio, intorno a 0 (tanti eventi rari), 225 e altri valori sparsi come 150 o 350;
  - Questo indica che ci sono cluster di paesi con livelli diversi di attività terroristica, mentre alcuni eventi sono unici o estremi;
- **Kernel Density Estimation (KDE)**
  - Si segue la forma dei dati;
  - Mostra un picco principale a densità elevata (eventi “normali”) e una coda verso densità bassa, dove risiedono gli eventi più rari;

Si ricorda che:

- i valori alti di densità indicano anni e paesi che si comportano come la maggior parte degli altri(numero attacchi e variazioni nella media);
- i valori bassi di densità indicano eventi **anomali o rari**.

Attraverso questi dati possiamo trovare delle anomalie e studiarle, infatti ad esempio: Iraq 2015 con 1408 attacchi e delta -1107; Pakistan 2015 con 1097 attacchi e delta -805.
Questi esempi mostrano come sia possibile usare i modelli di densità per **evidenziare pattern straordinari e potenziali anomalie** nel dataset.

CONCLUSIONI

Conclusioni sulla prima parte: Analisi Esplorativa e Inferenza Statistica

Analisi Esplorativa

Nell'analisi esplorativa, abbiamo proceduto suddividendo il lavoro in due fasi: analisi univariata, ovvero di una variabile per volta, e analisi multivariata di due o più variabili per volta, studiandone eventualmente le correlazioni.

Attraverso l'utilizzo di appositi grafici a barre, boxplot, density plot e un grafico a torta, l'analisi univariata ci ha mostrato ha permesso di risponderci a diverse domande, ad esempio **quali siano i paesi con il maggior numero di attacchi terroristici**, e giungiamo alla conclusione che è presente una forte concentrazione nei paesi mediorientali (Iraq circa 13,5%, Pakistan circa 8% e Afghanistan circa 7%), con **pochi attacchi che hanno causato molte vittime e la stragrande maggioranza che hanno causato 1 o 0 vittime**, nonché una minore concentrazione nel resto del mondo. Inoltre, la Skewness (asimmetria) ci dimostra un forte sbilanciamento verso destra (**5.67**), mentre la Kurtosis ci dimostra un forte appuntimento verso zero (**40.79**) come presenza di valore estremo. Come modalità più frequenti di attacco sorgono **Bombing/Explosion**, **Armed Assault**, **Assassination**, **Facility/Infrastructure Attack** e **Hostage Taking**, e la distribuzione di `attacktype1` suggerisce l'esistenza di strategie operative ricorrenti, orientate a massimizzare l'impatto con modalità consolidate. Infine, i bersagli più frequentemente colpiti secondo targtype1 sono **Private Citizens & Property**, **Government (General)**, **Police** e **Military e Business**; secondo l'EDA, il terrorismo colpisce prevalentemente bersagli ad alto valore simbolico o civile, con l'obiettivo di amplificare l'impatto psicologico. Per quanto riguarda l'analisi multivariata, abbiamo effettuato il confronto tra più tipi diversi di variabili tra loro: **Regione e Bersaglio (Categorica vs. Categorica)**, **Modalita' d'attacco e gruppi terroristici (Categorica vs. Categorica)**, **Uccisi, feriti, autori coinvolti e autori catturati (Numerica vs. Numerica)** e **Vittime per gruppo terroristico (Numerica vs. Categorica)**, con l'ausilio di strumenti come tabelle di contingenza, studi del **Test Chi-Quadrato**, **V di Cramer**, **p-value**, grafici a barre, boxplot e heatmap come forma di rappresentazione. Nel primo caso, dalla presenza di valori inferiori a **5** nella tabella di contingenza, abbiamo potuto osservare che la regione influisce sul tipo di bersaglio, ma non lo determina in modo netto; nel secondo caso, abbiamo visto che il **test del chi-quadrato** evidenzia che entrambe le associazioni sono statisticamente significative (p-value ≈ 0) ma che, con la V di Cramer, fossero presenti differenze

tra ***gname-attacktype1_txt*** e ***gname-targettype1_txt***; nel terzo caso, ***nkill***, ***nwound***, ***nperps*** e ***nperpcap*** sono risultate quasi indipendenti tra loro, trovando una spiegazione nel fatto che le variabili ***nkill*** e ***nwound*** contenessero un elevato numero di zeri, ovvero molti attacchi senza vittime o feriti; infine, nel quarto e ultimo caso abbiamo potuto notare che dalla tabella della **media** e della **mediana**, **Taliban** presenta la media più alta con **1.51 vittime per attacco** su **oltre 3.500 attacchi**, rendendolo sia il più attivo che il più letale, mentre **Boko Haram** e **LTTE** mostrano medie elevate (**~1.5** e **~1.3**), pur avendo un numero di attacchi inferiore, suggerendo una **strategia più distruttiva per singolo evento**.

Inferenza Statistica

Per effettuare l'inferenza statistica, ci siamo concentrati su tre domande inferenziali: **Media uccisioni in Middle East & North Africa**, **Intervallo di confidenza per la media di vittime per attacco** e **Confronto tra due campioni pre-2010 e 2010-2016**. Per la prima domanda inferenziale, abbiamo una media di 'nkill' del 0.82 più alta della media della popolazione, che risulta invece del 0.69, implicando che Middle East & North Africa sia l'area con il maggior numero di attacchi e anche una delle aree in cui ***ogni singolo evento terroristico risulta mediamente più letale***. Per la seconda domanda inferenziale, il test d'ipotesi fallisce, rendendo il campione uno **stimatore fedele della reale media di vittime globale**, attestata intorno a **0.69** vittime per attacco, con un **intervallo di confidenza** compreso tra **0.68** e **0.72**. Inoltre dalla varianza campionaria leggermente grande si notano **molti attacchi con poche vittime (0 o 1)** e **pochi attacchi con molte vittime (>1)**. Infine per la terza domanda, osservando le medie campionarie, ottenendo **$p<0.05$** rifiutando l'ipotesi nulla poiché esiste una differenza statisticamente significativa e notando che il numero medio di vittime per attacco risulta più alto **nel periodo 2010–2016** rispetto a tutto il periodo precedente, viene suggerito che nel periodo recente gli attacchi tendevano ad essere più letali, evidenziando un periodo di particolare **instabilità geopolitica** in cui l'attività di gruppi armati ha raggiunto livelli molto alti.

Come conclusione complessiva possiamo affermare che le **analisi esplorative** ci hanno permesso di identificare pattern temporali, geografici e operativi ricorrenti, mentre le **analisi inferenziali** mettono in luce i limiti interpretativi legati all'incertezza, all'attribuzione e alla definizione stessa di terrorismo.

Conclusioni sulla seconda parte: Modellazione statistica e Machine Learning

Modellazione e Regressione

Il **modello lineare** applicato al **logaritmo di nkill**, a cui è stata applicata la **backward elimination**, pur spiegando solo il **32%** circa della variabilità complessiva, mette in luce alcune relazioni statistiche. Evidenzia che le variabili **success**, **suicide** e **nwound** sono significativamente **associate** al numero di vittime. In particolare, gli *attacchi suicidi* mostrano un *impatto molto forte*, con un incremento stimato del **101%** rispetto alla baseline. Alcuni *target* specifici, come **Airports & Aircraft**, sono correlati a una **riduzione** del numero di vittime. Inoltre, la tipologia di attacco **Assassination** è l'**unica** con **effetto positivo** sul logaritmo di nkill, mentre le altre tendono a generare meno vittime.

Il **modello logistico** sugli **ostaggi**, a cui abbiamo applicato la **backward elimination**, **spiega bene** la probabilità di avere ostaggi (**58%**) e ottiene molti predittori con **p-value significativo** a

cui abbiamo calcolato gli **odds ratio**. Da questi ultimi notiamo che gli attacchi rivolti a **bersagli istituzionali o militari** tendono a **ridurre la probabilità** di ostaggi, mentre l'uso di **armi da fuoco o da taglio** la **aumenta**. Un numero maggiore di **vittime o feriti** è associato a una **minore probabilità** di presa di ostaggi, suggerendo dinamiche più rapide e letali. Al contrario, un numero più elevato di **autori** e un **successo operativo incrementano** sensibilmente il rischio. L'effetto più marcato riguarda gli **eventi estesi** oltre le 24 ore, che **amplificano drasticamente la probabilità** di ostaggi. Queste relazioni delineano i principali fattori che influenzano la dinamica degli attacchi con sequestro.

Il **modello logistico sugli attacchi estesi** oltre le 24h, soggetto a **backward elimination**, **spiega bene** la **probabilità** che un'attacco sia esteso (**70%**) e rileva molti **predittori significativi** ai quali abbiamo calcolato gli **odds ratio**. Da essi notiamo che il gli attacchi extended sono influenzati dal **rapimento di ostaggi**, dal **successo operativo**, dagli **attacchi contro i media** e dal **contesto geografico**. Questi fattori sono fortemente associati a operazioni complesse, negoziazioni o contesti geopolitici che favoriscono eventi prolungati. Inoltre ci sono *attacchi* rapidi come **esplosivi**, **assassinii**, **corpo a corpo** e *bersagli* come **trasporti**, **militari/polizia**, che **riducono drasticamente** la **probabilità** di durata oltre le 24 ore.

Analisi Predittiva e ML

Nel contesto applicativo in cui vogliamo intercettare **attacchi con vittime per poter intervenire tempestivamente**, il modello **Gaussian Naive Bayes** si distingue per l'**elevatissimo recall** sulla **classe letale**, pur sacrificando **precisione e accuratezza**. Tuttavia, la **curva ROC evidenzia che la Regresisone Logistica** mantiene un **miglior equilibrio globale tra sensibilità e specificità**, con un **AUC superiore**. In scenari operativi dove è cruciale **ridurre falsi negativi** ma anche **contenere falsi allarmi**, la **Regressione Logistica risulta più adatta** a supportare decisioni affidabili e sostenibili.

Il modello di **Regressione Logistica sui tipi di attacco**, ottimizzato tramite **Grid Search** e integrato in una **pipeline** con **preprocessing numerico e categorico**, raggiunge performance solide con un'accuratezza dell'**80%**. Le classi più frequenti di *attacktype1_txt*, come **Armed Assault** e **Bombing/Explosion**, vengono riconosciute con **elevata affidabilità**, rendendo il modello utile per scenari operativi che richiedono una *rapida classificazione degli attacchi più comuni*. Le prestazioni **calano** sulle categorie rare, come **Hijacking(dirottamento)** o **Hostage Taking (Barricade Incident)(barricamento con presa ostaggi)**, dove il **basso recall** riflette la scarsità di esempi nel dataset. Nonostante questo limite, offre una **buona capacità** di generalizzazione sulle tipologie di attacco più rilevanti.

Il confronto tra i **modelli predittivi** applicati a **gname (top10)** mostra che la **Linear Discriminant Analysis (LDA)** rappresenta la soluzione **più efficace ed equilibrata**. Pur utilizzando strumenti diversi come **Regressione Logistica**, **Naive Bayes** e **LDA**, quest'ultima garantisce prestazioni molto *vicine alla Regressione Logistica* ma con **tempi di esecuzione** nettamente **inferiori**. L'**accuratezza elevata**, unita alla quasi totale assenza di **falsi positivi** e **falsi negativi** nella **confusion matrix**, la rende particolarmente **affidabile** per classificare correttamente i **gruppi più attivi**. In uno scenario operativo dove si vogliono identificare anche gli attacchi con gname sconosciuto, la LDA offre un supporto concreto per evidenziare pattern sospetti e migliorare le attività di monitoraggio.

Conclusioni sulla terza parte: Unsupervised Learning

Questa parte segna un passaggio metodologico importante verso l'**apprendimento non supersionato**, ossia che non ci sono **variabili target** predefinite e si esplora la natura profonda del fenomeno terrorisitico.

Mendiante la selezioni di alcune variabili e il ridimensionamento dei dati mediante PCA, gli attacchi appaiono come una struttura continua che scivola da **eventi minori** a **catastrofi**.

Applicando algoritmi di clustering, lo studio fa emergere che attacchi più brutali sono meno efficienti e hanno un tasso di successo minore. Quindi la brutalità in un attacco come il suicidio(ad esempio con giubotto esplosivo), compromette la riuscita dell'operazione stessa. Inversamente, attacchi con molti perpretratori(ben organizzati) ma poche vittime(letalità inferiore), porta al successo dell'attacco terroristico ergo i perpretatori ottengono ciò che vogliono.

L'analisi che si è fatta successivamente riguarda l'**evoluzione storico geografica**. I cluster individuati evidenziano una migrazione del baricentro del terrorismo: se tra gli anni ’70 e ’90 il fenomeno era prevalentemente radicato nelle Americhe e in Europa, oggi risulta concentrato soprattutto in Medio Oriente, Nord Africa e Asia.

Infine, l'ultima sezione si occupa di distinguere il **comportamento normale dei paesi** da **eventi eccezionali**, analizzando non solo il numero di attacchi, ma la loro variazione rispetto l'anno precedente. L'analisi ha permesso di isolare matematicamente le anomalie strane. Ad esempio, l'Iraq nel 2015 (1408 attacchi) e il Pakistan nel 2015 sono emersi come casi straordinari, caratterizzati da densità probabilistiche vicine allo zero a causa dell'elevato numero di eventi e delle drastiche variazioni annuali.